

MOBILE WEB WRAPPERS WITH .NET MAUI

SUCCINCTLY[®]

BY **ALESSANDRO
DEL SOLE**

Mobile Web Wrappers with .NET MAUI Succinctly

Alessandro Del Sole

Foreword by Daniel Jebaraj



Copyright © 2026 by Syncfusion, Inc.

2501 Aerial Center Parkway

Suite 111

Morrisville, NC 27560

USA

All rights reserved.

ISBN: 978-1-64200-250-8

Important licensing information. Please read.

This book is available for free download from www.syncfusion.com on completion of a registration form.

If you obtained this book from any other source, please register and download a free copy from www.syncfusion.com.

This book is licensed for reading only if obtained from www.syncfusion.com.

This book is licensed strictly for personal or educational use.

Redistribution in any form is prohibited.

The authors and copyright holders provide absolutely no warranty for any information provided.

The authors and copyright holders shall not be liable for any claim, damages, or any other liability arising from, out of, or in connection with the information in this book.

Please do not use this book if the listed terms are unacceptable.

Use shall constitute acceptance of the terms listed.

SYNCFUSION, SUCCINCTLY, ESSENTIAL, ESSENTIAL STUDIO, BOLDDesk, BOLDSIGN, BOLD BI, and BOLD REPORTS are the registered trademarks of Syncfusion, Inc.

Technical Reviewer: James McCaffrey

Copy Editor: Courtney Wright

Acquisitions Coordinator: Graham High, content team lead, Syncfusion, Inc.

Proofreader: Naomi Spicer, content producer, Syncfusion, Inc.

Table of Contents

About the Author	10
Introduction.....	11
Target audience	11
What you will learn	11
Chapter 1 Introducing Mobile Web Wrappers	13
What is a mobile web wrapper?.....	13
Mobile wrappers vs. native apps	13
Key use cases and industry examples.....	14
Chapter summary.....	15
Chapter 2 Setting Up the Development Environment.....	16
Installing .NET workloads	16
Setting up an Azure subscription	18
Chapter summary.....	18
Chapter 3 Comparing WebView and HybridWebView	19
Introducing the HybridWebView	19
Differences between HybridWebView and WebView.....	19
Choosing between the WebView and HybridWebView	20
Chapter summary.....	21
Chapter 4 Embedding Web Content in .NET MAUI: HybridWebView	22
Creating a .NET MAUI solution.....	22
Structuring web content within the project.....	23
Adding sample data	23
Adding CSS styles	23
Adding JavaScript code	24

Adding HTML content	28
Defining the user interface	31
Code-behind implementation	33
Running the application	37
Chapter summary	40
Chapter 5 Wrapping Existing Websites	41
Creating a sample ASP.NET Core web application	41
Configuring SQL Server and migrations	42
Applying Entity Framework migrations	43
Configuring the project startup	44
Defining webpages	45
Displaying a PDF document.....	46
Sensor interaction: geolocation.....	47
Adding a user profile page	49
Printing page content.....	51
Implementing page navigation	52
Running the web application.....	53
Deploying the web application to Azure	57
Creating an Azure SQL database	58
Deploying the web application	62
Creating a .NET MAUI project	64
Implementing a custom WebView	65
Defining a ViewModel	66
Defining the startup code	67
Mobile app recap	68
Chapter summary	68

Chapter 6 Implementing the Client Infrastructure	69
Understanding the interaction between websites and .NET	69
Important considerations about JavaScript injection	69
Implementing bridge class and handlers on iOS.....	71
Implementing bridge class and handlers on Android	75
Defining the bridge class.....	76
Connecting bridge classes to the custom WebView	78
Extending the custom WebView to address iOS requirements.....	82
Completing the client infrastructure	83
Chapter summary.....	84
Chapter 7 Implementing a Navigation Bar	85
Defining the user interface.....	85
Controlling the navigation bar visibility.....	88
Running the application.....	89
Chapter summary.....	91
Chapter 8 Implementing Biometric Authentication	92
Installing NuGet packages.....	92
Extending the CustomWebView control.....	92
Handling authentication events.....	94
Defining methods for the biometric flow	95
Extending functionalities with extension methods	99
Injecting JavaScript code	102
Adding manifest permissions.....	106
Running the application.....	108
Chapter summary.....	111
Chapter 9 Handling Mobile Input Properly	112

Defining the scripts.....	112
Injecting the scripts.....	113
Running the application	114
Chapter summary.....	115
Chapter 10 Printing and Sharing Documents	116
Implementing printing capabilities.....	116
Defining the platform-specific scripts.....	116
Accessing the platform-specific printing services	118
Injecting the scripts	121
Sharing documents	122
Defining the platform-specific scripts.....	122
Handling document URLs	124
Injecting the scripts	125
Displaying the OS sharing user interface	126
Running the application	127
Testing the printing service	127
Testing the document sharing feature.....	129
Chapter summary.....	130
Chapter 11 Working with Sensors and Permissions	131
Intercepting web geolocation requests with JavaScript.....	131
Defining the Android scripts	131
Defining the iOS scripts	132
Requesting user permissions	133
Declaring permissions on Android.....	133
Declaring permissions on iOS.....	134
Requesting permissions in C#	134

Injecting JavaScript code.....	136
Managing location data in the bridge classes	136
Android implementation	136
iOS implementation	137
Handling events and managing location data	138
Running the application	140
A common approach for sensors	142
Chapter summary	142
Chapter 12 Handling Network Connections	143
Handling network connections: Understanding the flow	143
Defining and implementing a network service	143
Designing a user interface for network status change.....	145
Handling connection events.....	146
Running the application	149
Future enhancements.....	151
Hints about submitting apps to Google Play and the Apple App Store	151
Chapter summary	151
Conclusion	152

The *Succinctly* Series of Books

Daniel Jebaraj
CEO of Syncfusion, Inc.

When we published our first *Succinctly* series book in 2012, *jQuery Succinctly*, our goal was to produce a series of concise technical books targeted at software developers working primarily on the Microsoft platform. We firmly believed then, as we do now, that most topics of interest can be translated into books that are about 100 pages in length.

We have since published over 200 books that have been downloaded millions of times. Reaching more than 2.7 million readers around the world, we have more than 70 authors who now cover a wider range of topics, such as Blazor, machine learning, and big data.

Each author is carefully chosen from a pool of talented experts who share our vision. The book before you and the others in this series are the result of our authors' tireless work. Within these pages, you will find original content that is guaranteed to get you up and running in about the time it takes to drink a few cups of coffee.

We are absolutely thrilled with the enthusiastic reception of our books. We believe the *Succinctly* series is the largest library of free technical books being actively published today. Truly exciting!

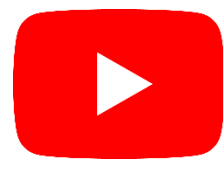
Our goal is to keep the information free and easily available so that anyone with a computing device and internet access can obtain concise information and benefit from it. The books will always be free. Any updates we publish will also be free.

Let us know what you think

If you have any topics of interest, thoughts, or feedback, please feel free to send them to us at succinctlyseries@syncfusion.com.

We sincerely hope you enjoy reading this book and that it helps you better understand the topic of study. Thank you for reading.

Please follow us on social media and help us spread the word about the *Succinctly* series!



About the Author

Alessandro Del Sole is a Xamarin Certified Mobile Developer and a former Microsoft MVP. Awarded MVP of the Year in 2009, 2010, 2011, 2012, and 2014, he is internationally considered a Visual Studio expert and a .NET authority. Alessandro has authored many printed books and ebooks on programming with Visual Studio, including *.NET MAUI Succinctly*, *Visual Basic 2015 Unleashed*, and *Xamarin.Forms Succinctly*. He has written tons of technical articles about .NET, Visual Studio, and other Microsoft technologies in Italian and English for many developer portals. He has been a frequent speaker at Italian conferences, and he has also produced many instructional videos in both English and Italian. Alessandro works as a senior software engineer for Fresenius Medical Care, focusing on building mobile apps with .NET MAUI in the healthcare market. You can follow him on [LinkedIn](#), and support him with a [coffee](#).

Introduction

Many companies have websites, and they also want to offer mobile app counterparts to bring their business to the palms of users' hands. In most cases, the best option is creating native mobile apps designed to deliver the best and most appropriate experience, taking advantage of the platform's APIs and device hardware and of design principles that adhere to each platform's standards.

However, building native mobile apps is not always possible for many reasons, including timing or budget. Plus, a company's website can be quite simple and can be easily displayed as it is inside the shell of a mobile app.

The point is that neither Google Play nor the Apple App Store accepts submissions of mobile apps that simply display web content without providing features that are typical of the mobile experience. These can include, but are not limited to, biometric authentication and sensor integration, a navigation bar, network connection handling, and showing the appropriate keypad based on the data type of an input field. Also, if the website has features that use default services, such as printing, these must be rewritten for mobile platforms.

This book shows you how to build mobile apps using .NET MAUI that display web content and can be safely submitted to and approved by Google Play and the Apple App Store. It's a practical guide to creating *web wrappers*, which let you use your existing websites or web applications as mobile apps. You'll learn how to make these web-based apps feel more like native mobile apps by adding features specific to phones and tablets.

Target audience

This book is intended for mobile developers who have intermediate knowledge of .NET MAUI and are looking to extend their target to displaying existing web content in mobile apps. If you need guidance about .NET MAUI, it is recommended that you first read [.NET MAUI Succinctly](#) from the author of this ebook. The assumption is that you are familiar with creating Visual Studio solutions, that you know how to add pages and debug applications, and that you have at least a basic knowledge of development platforms that use XAML to define the user interface, such as .NET MAUI, Xamarin.Forms, or Universal Windows Platform.

Knowing JavaScript, HTML, and CSS is not mandatory but is absolutely beneficial, as you'll be working a lot with web code and connecting it to your C# app code. By the way, you will always receive the appropriate explanations where necessary.

What you will learn

This book will help you:

- Understand mobile web wrappers and when to use them.
- Set up your development environment for building these apps.

- Use .NET MAUI's new **HybridWebView** to display in-app web content.
- Use .NET MAUI's **WebView** controls to display external websites.
- Make your web content talk to your native C# app.
- Add native features like navigation bars, biometric login, and access to device sensors.
- Handle common mobile challenges like internet connectivity issues.

You will combine and integrate all these features into a sample application that is compliant with the Google Play and Apple App Store requirements for mobile web wrappers. This includes creating a sample ASP.NET web application. You will also learn how to display in-app web content via the new **HybridWebView** control, recently introduced to MAUI in .NET 9.

The companion source code is [available on GitHub](#).

Chapter 1 Introducing Mobile Web Wrappers

In today's fast-paced software landscape, organizations often face the challenge of delivering high-quality mobile experiences without duplicating their efforts across platforms. If you already have a well-designed and responsive web application, you may be wondering how to bring that experience to iOS and Android users without rewriting the entire app using platform-specific code. This is where the concept of a mobile web wrapper comes into play.



Note: *.NET MAUI allows for targeting multiple operating systems from a single, shared codebase, including Android, iOS, Windows 10 and higher, macOS, and Tizen. However, since the target of this ebook is mobile applications, the main focus will be on Android and iOS.*

What is a mobile web wrapper?

A mobile web wrapper is a native mobile application that primarily displays web content, typically by embedding a web application inside a container. This approach allows you to reuse your existing website or web app while still distributing it as a native app through app stores. At its core, a mobile web wrapper is a thin shell of native code that hosts a web view, a control that renders web content inside a native app. With this setup, the actual logic and UI can remain on the server or within your web application, while the wrapper handles interactions such as navigation, authentication, and access to native features like biometric login or file sharing.

However, Google Play and the Apple App Store will not accept submissions of apps that only display web content without providing users with an appropriate mobile experience. This implies adding native device features and custom interactions to enhance the user experience beyond what a mobile browser would allow. This can include implementing biometric authentication, leveraging sensors, sending files via the share user interface of each operating system, printing web content by invoking the platform's API, and so on.

.NET MAUI is a cross-platform development technology, which means that you can implement mobile features over web wrappers from a single, shared codebase, targeting Android, iOS, Windows, macOS, and Tizen devices. This also means improved productivity and faster customer reach.

Mobile wrappers vs. native apps

There are both advantages and limitations to consider when opting for a mobile wrapper. Pros include:

- **Reusability:** Leverage your existing web assets and back-end services.
- **Faster development:** Avoid building and maintaining separate native apps.

- **Cross-platform delivery:** Share one codebase with both iOS and Android users.
- **Native enhancements:** Add native features like biometrics, file sharing, and navigation without rewriting the entire app.

Cons include:

- **Performance limitations:** Web views may not perform as smoothly as fully native apps and might have feature restrictions imposed by the producer.
- **App store restrictions:** Apple and Google have strict guidelines for web-based apps.
- **Offline support:** Wrappers rely on the availability of a network connection unless extra work is done for caching.
- **Limited UI flexibility:** You may not fully match native UI/UX expectations.

By understanding these tradeoffs, you can make an informed decision based on your project's goals, team skills, and delivery timeline.

Key use cases and industry examples

Mobile web wrappers are especially useful in scenarios where time-to-market and code reuse are crucial. Some common use cases include:

- **Internal business tools:** Enterprise dashboards or forms used by employees.
- **Event and conference apps:** Quickly spin up branded apps for events that use an existing web portal.
- **Customer portals:** Let users access a web-based account or service through an app.
- **Educational platforms:** Online courses or documentation tools that are already web-based.

Well-known companies have adopted this strategy to various degrees. For example, retailers might embed their mobile web store to ensure feature parity across platforms. Even tech companies sometimes release MVPs (minimum viable products) as wrappers to validate ideas before committing to full native builds.

As you read through this book, you will build your own mobile web wrapper using .NET MAUI and .NET 9. You will learn how to integrate native features such as biometric authentication, a navigation bar, print functionality, file sharing, and permissions. By the end, you will have a solid understanding of how to balance the flexibility of the web with the power of native apps.



Note: Obviously, it is not possible to cover every possible scenario related to website wrapping in native mobile apps, but the approach followed in the next

chapters will give you the method to understand how you can customize the mobile experience based on your own website's features.

Chapter summary

A mobile web wrapper allows you to bring your existing web applications to iOS and Android with minimal effort by wrapping them in a native shell. This approach lets you reuse code, speed up development, and enrich the experience with native capabilities. While it comes with tradeoffs in performance and app store compliance, it remains a powerful strategy for many business cases.

In the next chapter, you will set up your development environment and begin building the foundation for your hybrid mobile app with .NET MAUI.

Chapter 2 Setting Up the Development Environment

Before you start building your mobile wrapper app, you need to set up a proper development environment. In this chapter, you will install the necessary tools to create a sample ASP.NET Core website that will act as the wrapped content and to set up a new .NET MAUI project using Visual Studio 2022 and .NET 9.

Installing .NET workloads

In this book, you will create a sample website that is wrapped in a mobile app built with .NET MAUI. There is a plethora of technologies that allow for building websites, but because you work with Visual Studio, and in order to keep things simple and organized, the example will be based on an ASP.NET Core web application. Web applications created with Razor pages are also *responsive*, which means that their user interface automatically and proportionally adapts to the device and screen size.

To get started, open the Visual Studio Installer tool and make sure that the following workloads are installed:

- **.NET Multi-platform App UI development:** This provides the environment to build mobile apps with .NET MAUI.
- **ASP.NET and web development:** This workload includes everything you need to create web applications with .NET.

Figure 1 shows how the workloads appear in the Visual Studio Installer tool.

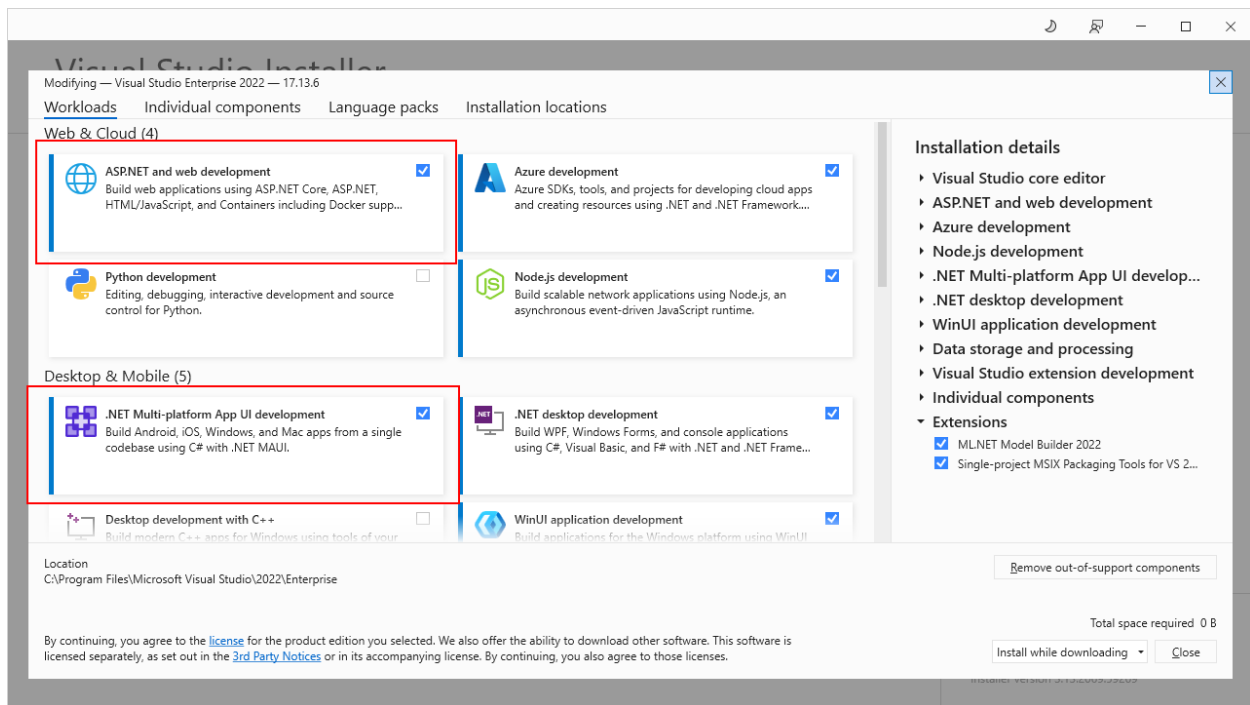


Figure 1: Installing the required .NET workloads

The last step is to ensure that the .NET 9 runtime is installed on your machine. Open the **Individual components** tab and make sure that **.NET 9 Runtime** and **.NET 9 Web Assembly Tools** are both selected, as shown in Figure 2.

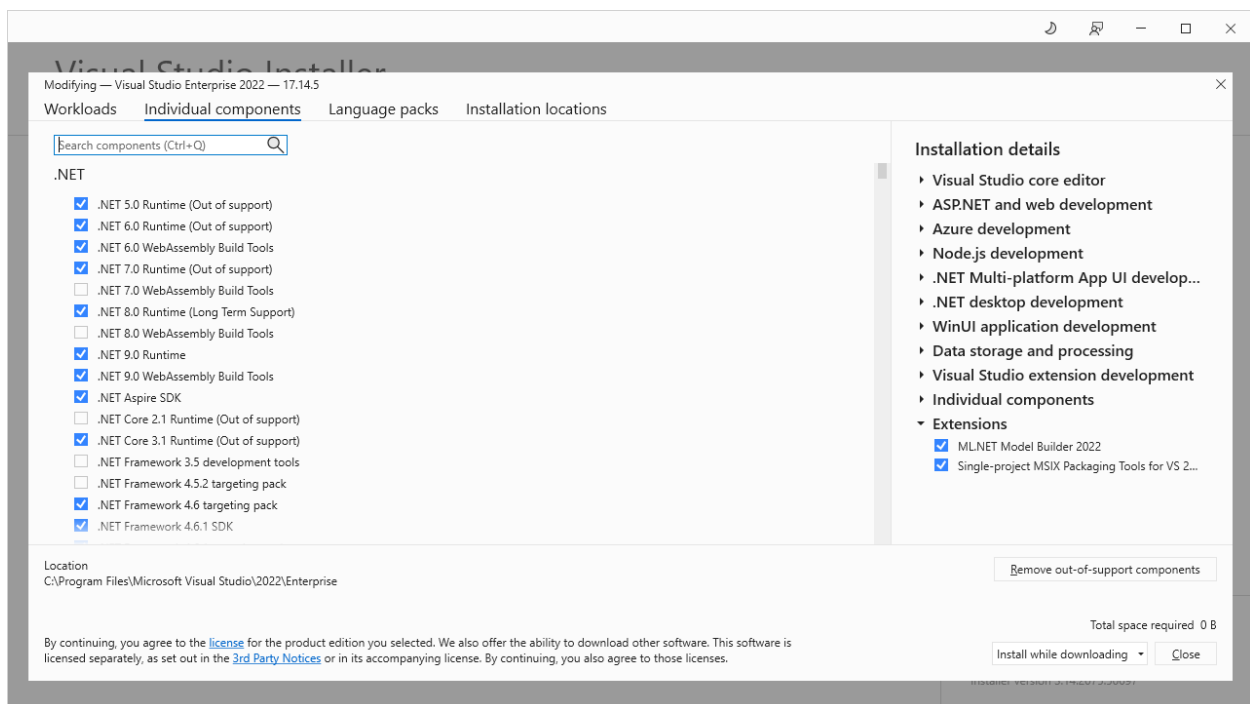


Figure 2: Ensuring that .NET 9 runtime tools are installed

Once you have installed or updated the necessary tools, you can move to creating a sample website.

Setting up an Azure subscription

To deploy your ASP.NET Core web application (which you will build in Chapter 5) and make it accessible from your .NET MAUI wrapper, a quick option is to deploy it to Microsoft Azure. You are totally free to deploy it to your own server or to a different cloud service, but since the content of this book is based on the Microsoft technology stack, Azure is the environment of choice.

If you do not already have an Azure subscription, Microsoft offers a free trial that provides credits and free services, perfect for getting started with development and testing without incurring costs. Follow these steps to create an Azure subscription:

1. Navigate to the [Azure Free Account](#) page.
2. Select **Start free** or **Create your free account**.
3. Sign in with your Microsoft account.
4. Complete the registration form. You'll need to provide some personal information, verify your identity via phone, and enter credit card details. Note: The credit card is for identity verification only. You won't be charged unless you explicitly upgrade to a paid subscription or exceed the free service limits.
5. Follow the on-screen prompts to complete the setup. Once activated, you'll have access to your Azure portal, where you can manage all your cloud resources.

The Azure free account typically includes a generous credit (for example, \$200 for 30 days) and free access to popular services for 12 months. This is more than enough for developing and testing the web application and deploying it to Azure App Services and Azure SQL Database, as demonstrated in Chapter 5. Always monitor your usage in the Azure portal to ensure you stay within the free tier limits.

With your environment successfully set up and an Azure subscription ready, you are now prepared to explore the core components of mobile web wrappers in .NET MAUI and deploy your web content to the cloud.

Chapter summary

In this chapter, you set up your development environment using Visual Studio 2022 and .NET 9. Additionally, you learned how to create an Azure free trial subscription, which is a good option for deploying your web application to the cloud later in this eBook.

With your local and cloud development environments ready, Chapter 3 will delve into a crucial comparison of **HybridWebView** and **WebView** controls.

Chapter 3 Comparing WebView and HybridWebView

When embarking on the journey of embedding web content within your .NET MAUI applications, you'll encounter two primary controls: **WebView** and **HybridWebView**. While both are designed to display web content, they serve distinct purposes, particularly concerning the level of interaction and communication required between your web content and the native .NET MAUI application. Understanding their differences is crucial for choosing the right control for your specific web wrapping scenario, and this chapter provides the appropriate explanation.

Introducing the HybridWebView

The **HybridWebView** is a new control introduced with .NET 9, which extends the functionality of a basic **WebView** by enabling seamless two-way communication between JavaScript running in the embedded web content and C# code in your .NET MAUI application. This capability is often referred to as *JavaScript interoperability* or *JavaScript/C# bridging*.

Traditional web views are excellent for displaying webpages, much like a miniature browser, as you will discover in the next chapters. However, they typically lack an easy, direct mechanism for the web content to interact with the native app code, or for the native app to directly invoke JavaScript functions within the web content.



Note: *Lacking direct and easy mechanisms for interaction between app code and website code does not mean that the **WebView** control cannot do this. In fact, starting with Chapter 5, you will see how to implement a complete mobile wrapping solution based on a regular **WebView** and specific C# and JavaScript code.*

The **HybridWebView** finally bridges this gap by exposing specific APIs that allow:

- **C#-to-JavaScript interaction:** C# code can execute JavaScript functions within the loaded web content and pass arguments to them.
- **JavaScript-to-C# interaction:** JavaScript code running in the web content can call C# methods exposed by your .NET MAUI application, allowing web actions to trigger native functionality (such as accessing device sensors, saving data to a native database, or showing a native alert).

This makes the **HybridWebView** ideal for building rich hybrid applications where the web content is not just a static display, but an interactive part of the native application, deeply integrated with device capabilities and native UI elements.

Differences between HybridWebView and WebView

For the sake of clarity, Table 1 summarizes the key differences between the two controls.

Table 1: Differences between the `WebView` and `HybridWebView`

Feature	<code>WebView</code>	<code>HybridWebView</code>
Primary purpose	Display isolated web content (HTML, CSS, JS)	Display in-app web content with robust two-way JavaScript/C# communication
JavaScript/C# bridge	No direct, built-in bridging mechanism; requires development efforts (see Chapter 5 and on)	Built-in support for two-way communication; allows JavaScript to call C# methods and C# to invoke JavaScript functions
Use cases	Displaying external websites, showing static local HTML files, and embedding web content that doesn't need to interact with native features	Building rich hybrid apps where web components need to trigger native actions, manipulating native UI from web content, and passing data seamlessly between web and native layers
Complexity	Simpler to use for basic web display	More complex setup due to the communication mechanisms
Underlying technology	Platform-specific web rendering components (<code>WKWebView</code> on iOS, native <code>WebView</code> on Android, <code>WebView2</code> on Windows)	Often built on top of the platform's native web view, but with added layers for the communication bridge

Choosing between the `WebView` and `HybridWebView`

Choose the `WebView` when your primary goal is displaying web content like external websites or displaying local HTML files, and if you need more complex native interaction between the external website and your app, as you will discover starting with Chapter 5.

Instead, choose the `HybridWebView` when your web content can be an integral part of your mobile application, requiring the ability to invoke native functionalities or for the native app to control elements within the web view.

Chapter summary

This chapter provided a detailed comparison of **WebView** and **HybridWebView**, highlighting their distinct purposes, use cases, and particularly the crucial two-way communication capabilities of the latter. Understanding these differences is key to making informed decisions when building your mobile web wrapper.

In Chapter 4, you will dive into the practical implementation of web content embedded in a .NET MAUI app using **HybridWebView**, complete with interactive JavaScript and C# communication.

Chapter 4 Embedding Web Content in .NET MAUI: HybridWebView

The purpose of embedding web content within a .NET MAUI application, specifically through the **HybridWebView** control, is to leverage existing web technologies (HTML, CSS, JavaScript) while still providing a rich, native application experience. This approach allows you to reuse web-based UI components or business logic within a mobile application, reducing development time and effort. The key strength of **HybridWebView** lies in its ability to facilitate two-way communication, enabling JavaScript code running in the web view to call C# methods in your .NET MAUI application, and vice versa.

This chapter provides guidance on creating hybrid apps, walking you through an example available on [GitHub](#) created by Beth Massi, Program Manager at Microsoft on .NET MAUI, which serves as a reliable and well-structured foundation for understanding cross-platform web-to-native communication using the **HybridWebView**. By using this example, we ensure the implementation aligns with Microsoft's intended usage patterns and best practices, offering a practical and realistic frame of reference for anyone looking to integrate **HybridWebView** functionality into their own applications.



Note: *The working example used in this chapter covers, with a general approach, the communication possibilities between JavaScript and C#, and vice versa. The official example provided in the documentation has some issues that prevent it from fully working. With this in mind, either clone, browse, or download the code from GitHub so that it is easier to follow the discussion.*

Creating a .NET MAUI solution

You will now recreate the sample app. You need to create a new .NET MAUI application in Visual Studio 2022. To do so:

1. Open Visual Studio 2022 and select **Create a new project**.
2. In the project templates list, search for **MAUI** and select the **.NET MAUI App** template. Select **Next**.
3. Provide a suitable project name, for example, **HybridWebViewDemo**, and choose a location for your project. Select **Next**.
4. Select **.NET 9** as the framework version and then select **Create**.

Once the project is created, you will need to add the web assets to your solution.

Structuring web content within the project

The source sample project is made of two parts: one part based on a user interface created with XAML, and one part based on a user interface created with React and embedded into the **HybridWebView**. However, this chapter only focuses on the first part. The second part with React will not be covered, since it is a very specific scenario.

Before diving into the C# and XAML implementation, it is important to first understand how to properly organize the web content files. The **HybridWebView** control relies on a specific folder within your .NET MAUI project to locate its web assets. By convention, all web-related files, such as HTML, CSS, JavaScript, and others, should be placed in the `Resources\Raw` directory.

For this example, the folder structure includes a subdirectory named `HybridSamplePage`, which contains the main HTML file (`index.html`), a JSON data file (`asyncdata.txt`), a docs folder with a sample PDF (`sample.pdf`), a scripts folder with the interop library (`HybridWebView.js`), and a styles folder with the stylesheet (`app.css`). This organization is essential because the **HybridRoot** property of **HybridWebView** will be set to **HybridSamplePage**, instructing the control to load `index.html` from that specific location.



Tip: Most examples about the *HybridWebView*, including the one from the official documentation, show how to use a folder called *wwwroot* by convention, located under *Resources\Raw*, to add web files and subfolders. The current example is instead using a folder called *HybridSamplePage*. The folder name does not follow any particular rule, so feel free to use a folder name that is convenient for you.

Now, you will add the specific files to these folders.

Adding sample data

You will add a file called `asyncdata.txt`, which is a simple JSON file that will be fetched asynchronously by JavaScript in your `index.html` file. This demonstrates how the web content can interact with local files served by the **HybridWebView**. To add the file:

1. Right-click the **HybridSamplePage** folder, select **Add > New Item**.
2. Choose **Text File** and name it **asyncdata.txt**.
3. Replace its content with the following:

```
{
  "key1": "value1",
  "key2": "value2"
}
```

The source example also includes a file called `sample.pdf`, which you can copy from the original solution into a new subfolder called `HybridSamplePage\docs`.

Adding CSS styles

CSS files can be used to provide styling for visual elements; in the current example, basic styling for the buttons within the `index.html` file will be provided. To accomplish this:

1. Right-click the **HybridSamplePage\styles** folder and select **Add > New Item**.
2. Choose **Stylesheet File** and name it **app.css**.
3. Replace its content with the following:

```
button {
    background-color: dodgerblue;
    border-radius: 5px;
    border-width: 1px;
    color: white;
    cursor: pointer;
    margin: 6px;
    text-align: center;
    font-size: x-large;
}

body {
    background-color: darksalmon;
    font-size: x-large;
}
```

You can obviously provide more sophisticated styles, but for the current example, this is enough.

Adding JavaScript code

Adding JavaScript code is crucial for enabling the communication bridge between your web content and the native .NET MAUI application. .NET MAUI provides a standard JavaScript library for the **HybridWebView**, which provides utility functions for sending messages to C# and invoking C# methods from JavaScript. Follow these steps to add the library file:

1. Right-click the **HybridSamplePage\scripts** folder and select **Add > New Item**.
2. Choose **JavaScript File** and name it **HybridWebView.js**.
3. Replace its content with Code Listing 1.

Code Listing 1

```
window.HybridWebView = {
    "Init": function Init() {
        function DispatchHybridWebViewMessage(message) {
            const event = new CustomEvent("HybridWebViewMessageReceived", {
                detail: { message: message } });
            window.dispatchEvent(event);
        }

        if (window.chrome && window.chrome.webview) {
            // Windows WebView2
            window.chrome.webview.addEventListener('message', arg => {
                DispatchHybridWebViewMessage(arg.data);
            });
        }
    }
};
```



```

    }
    else if (window.webkit && window.webkit.messageHandlers && window.w
ebkit.messageHandlers.webwindowinterop) {
        // iOS and MacCatalyst WKWebView
        window.external = {
            "receiveMessage": message => {
                DispatchHybridWebViewMessage(message);
            }
        };
    }
    else {
        // Android WebView
        window.addEventListener('message', arg => {
            DispatchHybridWebViewMessage(arg.data);
        });
    }
},

"SendRawMessage": function SendRawMessage(message) {
    window.HybridWebView.__SendMessageInternal('__RawMessage', message)
;
},

"InvokeDotNet": async function InvokeDotNetAsync(methodName, paramValue
s) {
    const body = {
        MethodName: methodName
    };

    if (typeof paramValues !== 'undefined') {
        if (!Array.isArray(paramValues)) {
            paramValues = [paramValues];
        }

        for (var i = 0; i < paramValues.length; i++) {
            paramValues[i] = JSON.stringify(paramValues[i]);
        }

        if (paramValues.length > 0) {
            body.ParamValues = paramValues;
        }
    }

    const message = JSON.stringify(body);

    var requestUrl = `${window.location.origin}/__hvwInvokeDotNet?data=
${encodeURIComponent(message)}`;

    const rawResponse = await fetch(requestUrl, {

```

```

        method: 'GET',
        headers: {
            'Accept': 'application/json'
        }
    });
    const response = await rawResponse.json();

    if (response) {
        if (response.IsJson) {
            return JSON.parse(response.Result);
        }

        return response.Result;
    }

    return null;
},

"__SendMessageInternal": function __SendMessageInternal(type, message)
{
    const messageToSend = type + '|' + message;

    if (window.chrome && window.chrome.webview) {
        // Windows WebView2
        window.chrome.webview.postMessage(messageToSend);
    }
    else if (window.webkit && window.webkit.messageHandlers && window.w
ebkit.messageHandlers.webwindowinterop) {
        // iOS and MacCatalyst WKWebView
        window.webkit.messageHandlers.webwindowinterop.postMessage(mess
ageToSend);
    }
    else {
        // Android WebView
        hybridWebViewHost.sendMessage(messageToSend);
    }
},

"__InvokeJavaScript": async function __InvokeJavaScript(taskId, methodN
ame, args) {
    try {
        var result = null;
        if (methodName[Symbol.toStringTag] === 'AsyncFunction') {
            result = await methodName(...args);
        } else {
            result = methodName(...args);
        }
        window.HybridWebView.__TriggerAsyncCallback(taskId, result);
    } catch (ex) {

```

```

        console.error(ex);
        window.HybridWebView.__TriggerAsyncFailedCallback(taskId, ex);
    },
    },
    "__TriggerAsyncCallback": function __TriggerAsyncCallback(taskId, result) {
        const json = JSON.stringify(result);
        window.HybridWebView.__SendMessageInternal('__InvokeJavaScriptCompleted', taskId + '|' + json);
    },
    "__TriggerAsyncFailedCallback": function __TriggerAsyncCallback(taskId, error) {
        if (!error) {
            json = {
                Message: "Unknown error",
                StackTrace: Error().stack
            };
        } else if (error instanceof Error) {
            json = {
                Name: error.name,
                Message: error.message,
                StackTrace: error.stack
            };
        } else if (typeof (error) === 'string') {
            json = {
                Message: error,
                StackTrace: Error().stack
            };
        } else {
            json = {
                Message: JSON.stringify(error),
                StackTrace: Error().stack
            };
        }

        json = JSON.stringify(json);
        window.HybridWebView.__SendMessageInternal('__InvokeJavaScriptFailed', taskId + '|' + json);
    }
}

window.HybridWebView.Init();

```

The **HybridWebView** JavaScript object is a cross-platform bridge that enables interactive communication between a web view and a .NET MAUI native application. As soon as the script loads, it triggers the **Init()** function, which sets up different messaging pipelines tailored to the hosting platform. These include: **window.chrome.webview** for Windows (**WebView2**), **window.webkit.messageHandlers.webwindowinterop** for iOS and macOS (**WKWebView**), and a standard **window.addEventListener('message')** fallback for Android.

Each of these mechanisms listens for incoming messages from the native side and passes them to a centralized handler that dispatches a custom JavaScript event (**HybridWebViewMessageReceived**) containing the message payload in **detail.message**. This makes it easy for other parts of the front-end app to subscribe and react to native-side communications.

For sending messages back to the native layer, the **SendRawMessage()** function is used. Under the hood, it relies on **__SendMessageInternal()**, which combines the message type and content into a single string using a delimiter and posts it to the native endpoint using the appropriate bridge based on the current platform. This abstraction handles the differences between environments so that developers don't need to tailor their logic per platform.

A more powerful capability is found in **InvokeDotNet()**, an asynchronous function that allows JavaScript to invoke C# methods directly. It wraps the method name and any parameters into an object, serializes them to JSON, and then constructs a special URL (**__hvwInvokeDotNet**) with the encoded payload passed as a query parameter. The request is dispatched using the **fetch()** API, and the response, expected in JSON form, is parsed and returned to the calling JS code. This function ensures input flexibility by detecting whether parameters are undefined, singular, or an array, and it guarantees consistent serialization. If the .NET response indicates the result is JSON, the method parses it into an object; otherwise, it returns the raw string.

Going the other direction, the native .NET layer can initiate calls into JavaScript using the internal function **__InvokeJavaScript()**. It accepts a task ID, method name, and arguments, then attempts to execute the corresponding JS function, either synchronously or asynchronously, depending on whether it's tagged as an async function. The result, on success or failure, is sent back to C# using either **__TriggerAsyncCallback()** or **__TriggerAsyncFailedCallback()**. These functions wrap the result or error (complete with message and stack trace) and transmit it back via the internal message-sending mechanism, maintaining consistency with the cross-platform pipeline.

Altogether, **HybridWebView** functions as a bi-directional messaging and function-invocation framework, insulating developers from the quirks of platform-specific integration and giving hybrid apps the feeling of tight, native-level integration between UI and business logic. It supports not just static messaging, but robust remote procedure calls, complete with async/await semantics, structured data serialization, and graceful error propagation, all triggered simply from JavaScript or C# in a highly extensible and adaptable form.

Adding HTML content

This is the main HTML file that will be displayed within the **HybridWebView**. It includes the **app.css** and **HybridWebView.js** files and contains various buttons to demonstrate the two-way communication.

1. Right-click the **HybridSamplePage** folder and select **Add > New Item**.
2. Choose HTML Page and name it **index.html**.
3. Replace its content with Code Listing 2.

Code Listing 2

```
<!DOCTYPE html>

<html lang="en" xmlns="http://www.w3.org/1999/xhtml">
<head>
    <meta charset="utf-8" />
    <title></title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no, viewport-fit=cover" />
    <link rel="icon" href="data:,">
    <link rel="stylesheet" href="styles/app.css">
    <script src="scripts/HybridWebView.js"></script>
    <script>
        window.addEventListener(
            "HybridWebViewMessageReceived",
            function (e) {
                DisplayMessage(e.detail.message);
            });

        function DisplayMessage(msg) {
            var messageFromCSharp = document.getElementById(
                "messageFromCSharp");
            messageFromCSharp.value += '\r\n' + msg;
        }

        function AddNumbers(a, x, b, y) {
            var result = {
                "result": a + b,
                "operationName": "Addition"
            };
            return result;
        }

        function SendAlert(msg) {
            window.alert(msg);
            DisplayMessage(msg);
            return undefined;
        }

        async function CallApiAsync() {
            var apiUrl = "https://restcountries.com/v3.1/name/aruba";

            const response = await fetch(apiUrl);
            if (!response.ok) {
                throw new Error(`HTTP error: ${response.status}`);
            }
        }
    </script>

```

```

    }
    var jsonData = await response.json();
    var msg = "API call returned: " + JSON.stringify(jsonData).
        substring(0, 100) + "...";
    window.HybridWebView.SendRawMessage(msg);

    return jsonData;
}

async function InvokeDotnetAdd() {
    var result = await window.HybridWebView.
        InvokeDotNet("AddNumbers", [1, 2]) ;
    DisplayMessage('Result from C#: ' + result);
}

var count = 0;

async function EvaluateMeWithParamsAndAsyncReturn(s1, s2) {
    const response = await fetch("/asyncdata.txt");
    if (!response.ok) {
        throw new Error(`HTTP error: ${response.status}`);
    }
    var jsonData = await response.json();

    jsonData[s1] = s2;

    const msg = 'JSON data is available: ' +
        JSON.stringify(jsonData);
    window.HybridWebView.SendRawMessage(msg)

    return jsonData;
}
</script>
</head>
<body>
    <div>
        Hybrid sample!
    </div>
    <div>
        <button onclick="window.HybridWebView.
            SendRawMessage('Message from JS! ' +
                (count++))">Send message to C#</button>
        <button onclick="InvokeDotnetAdd()">Invoke C# Add function</button>

    </div>
    <div>
        Message from C#: <textarea readonly id="messageFromCSharp"
            style="width: 80%; height: 10em;"></textarea>
    </div>

```

```

        <div>
            Consider checking out this PDF: <a href="docs/sample.pdf">sample.pdf
        f</a>
        </div>
    </body>
</html>

```

The page includes basic styling via the external CSS file (app.css) and loads the JavaScript library (HybridWebView.js) that facilitates communication between JavaScript and C#. Within the `<script>` block, several functions are defined to handle this two-way interaction.

The `HybridWebViewMessageReceived` event listener waits for messages from the C# side and appends them to a read-only `<textarea>` element using the `DisplayMessage` function. The `AddNumbers` function performs a basic addition operation and returns a result object. The `SendAlert` function shows a browser alert with a message and logs it to the text area. The `CallApiAsync` function performs an asynchronous fetch request to the REST Countries API, retrieves JSON data about Aruba as an example, and sends a truncated version of the response back to C# using the `SendRawMessage` function. Similarly, the `InvokeDotnetAdd` function demonstrates invoking a C# method named `AddNumbers` from JavaScript and displaying the result.

Another function, `EvaluateMeWithParamsAndAsyncReturn`, fetches the local JSON file (asyncdata.txt), modifies its content based on parameters, and sends the updated data back to C#. A global counter variable (`count`) is used to track how many times the **Send message to C#** button is clicked, appending the count to each message. The body of the page includes several interactive elements: buttons that trigger JavaScript functions to communicate with C#, a text area to display incoming messages, and a link to a sample PDF file.

 **Tip:** For all the files that you added so far, you need to assign the *Build Action* property, in the *Properties* window, with the *MauiAsset* value, and the *Copy to Output Directory* property with *Do not Copy*.

Defining the user interface

Now, you will define the user interface for the sample application in the `MainPage.xaml` file. This XAML file will contain the `HybridWebView` control and native .NET MAUI UI elements for interaction. Having said this, open the file and integrate its content with Code Listing 3.

Code Listing 3

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="HybridMauiApp.MainPage"
              Title="HybridWebView Basics">

```

```

<Grid ColumnDefinitions="2*,1*" RowDefinitions="Auto,1*">

    <Editor
        Grid.Row="0"
        Grid.Column="0"
        Text="HybridWebView here"
        IsReadOnly="True"
        MinimumHeightRequest="200"
        x:Name="statusText" />

    <VerticalStackLayout
        Grid.Row="0"
        Grid.Column="1">

        <Button
            Margin="10"
            Text="Send message to JS"
            Clicked="SendMessageButton_Clicked" />

        <Button
            Margin="10"
            Text="Invoke JS w/ null return"
            Clicked="InvokeJSNullMethodButton_Clicked" />

        <Button
            Margin="10"
            Text="Call external API from JS"
            Clicked="InvokeJSApiMethodButton_Clicked" />

        <Button
            Margin="10"
            Text="Invoke JS"
            Clicked="InvokeJSMethodButton_Clicked" />

        <Button
            Margin="10"
            Text="Invoke Async JS"
            Clicked="InvokeAsyncJSMethodButton_Clicked" />

    </VerticalStackLayout>

    <HybridWebView
        x:Name="hwv"
        Grid.Row="1"
        Grid.ColumnSpan="3"
        HybridRoot="HybridSamplePage"
        RawMessageReceived="hwv_RawMessageReceived"/>

</Grid>

```



```
</ContentPage>
```

The user interface layout for the main page of the hybrid application combines native MAUI elements with the **HybridWebView** to facilitate communication between C# and JavaScript.

The top section contains an **Editor** named **statusText** to log messages from both C# and JavaScript. Next is a **VerticalStackLayout** with buttons to initiate different interactions. The core of the page is the **HybridWebView** control itself, which loads our local web content. Notice how the **HybridRoot** property specifies the project folder where to look for content, and how the **RawMessageReceived** event handler is declared to intercept messages from JavaScript. This will be implemented in the next section about the C# logic.

Code-behind implementation

The last step is implementing the logic in the MainPage.xaml.cs code-behind file to handle interactions with the **HybridWebView**, including sending messages to JavaScript and receiving messages/invoking JavaScript methods from C#. Open the file and replace its content with Code Listing 4.

Code Listing 4

```
using System.Text.Json.Serialization;

namespace HybridMauiApp
{
    public partial class MainPage : ContentPage
    {
        public MainPage()
        {
            InitializeComponent();

            hwv.SetInvokeJavaScriptTarget<ComputeJSInvokeTarget>(
                new ComputeJSInvokeTarget(this));
        }

        int count;
        private void SendMessageButton_Clicked(object sender, EventArgs e)
        {
            hwv.SendRawMessage($"Hello from C#! #{count++}");
        }

        private void InvokeJSNullMethodButton_Clicked(object sender,
            EventArgs e)
        {
            hwv.InvokeJavaScriptAsync<string>("SendAlert",
                null,
```

```

        ["Hello from C#! #{count++}"],
        [SampleInvokeJsContext.Default.String]);
    }

    private void InvokeJSApiMethodButton_Clicked(object sender,
        EventArgs e)
    {
        hmv.InvokeJavaScriptAsync<string>("CallApiAsync", null);
    }

    private async void InvokeJSMethodButton_Clicked(object sender,
        EventArgs e)
    {
        var statusResult = "";

        var x = 123d;
        var y = 321d;

        var result = await hmv.
            InvokeJavaScriptAsync<ComputationResult>(
                "AddNumbers",
                SampleInvokeJsContext.Default.ComputationResult,
                [x, null, y, null],
                [SampleInvokeJsContext.Default.Double, null,
                SampleInvokeJsContext.Default.Double, null]);

        if (result is null)
        {
            statusResult += Environment.NewLine +
                $"Got no result for operation with {x} and {y}";
        }
        else
        {
            statusResult += Environment.NewLine +
                $"Used operation {result.operationName} with numbers {x}
                and {y} to get {result.result}";
        }

        Dispatcher.Dispatch(() => statusText.Text += statusResult);
    }

    private async void InvokeAsyncJSMethodButton_Clicked(object sender,
        EventArgs e)
    {
        var statusResult = "";

        var asyncFuncResult = await hmv.InvokeJavaScriptAsync

```

```

        <Dictionary<string, string>>(
            "EvaluateMeWithParamsAndAsyncReturn",
            SampleInvokeJsContext.Default.DictionaryStringString,
            ["new_key", "new_value"],
            [SampleInvokeJsContext.Default.String,
             SampleInvokeJsContext.Default.String]);

        if (asyncFuncResult == null)
        {
            statusResult += Environment.NewLine +
                $"Got no result from EvaluateMeWithParamsAndAsyncReturn ";
        }
        else
        {
            statusResult += Environment.NewLine +
                $"Got result from EvaluateMeWithParamsAndAsyncReturn:
                {string.Join(", ", asyncFuncResult)}";
        }

        Dispatcher.Dispatch(() => statusText.Text += statusResult);
    }

    private void hwv_RawMessageReceived(object sender,
        HybridWebViewRawMessageReceivedEventArgs e)
    {
        Dispatcher.Dispatch(() => statusText.Text +=
            Environment.NewLine + e.Message);
    }

    public class ComputationResult
    {
        public double result { get; set; }
        public string? operationName { get; set; }
    }

    // This type's attributes specify JSON serialization info to
    // preserve type structure for trimmed builds.
    [JsonSourceGenerationOptions(WriteIndented = true)]
    [JsonSerializable(typeof(ComputationResult))]
    [JsonSerializable(typeof(double))]
    [JsonSerializable(typeof(string))]
    [JsonSerializable(typeof(Dictionary<string, string>))]
    internal partial class SampleInvokeJsContext : JsonSerializerContext
    {
    }

    private sealed class ComputeJSInvokeTarget
    {
        private MainPage _mainPage;
    }

```

```

        public ComputeJSInvokeTarget(MainPage mainPage)
        {
            _mainPage = mainPage;
        }

        public int AddNumbers(int number1, int number2)
        {
            var result = number1 + number2;

            _mainPage.Dispatcher.Dispatch(() => _
            mainPage.statusText.Text +=
            Environment.NewLine + $"Got number {result}");
            return result;
        }
    }
}
}
}

```

Upon initialization, the page sets up a target class, **ComputeJSInvokeTarget**, which exposes C# methods that can be invoked from JavaScript. The page includes several **Clicked** event handlers, each showcasing different forms of communication.

The **SendMessageButton_Clicked** event handler sends a simple raw message from C# to JavaScript, incrementing a counter with each click. The **InvokeJSNullMethodButton_Clicked** event handler calls a JavaScript function named **SendAlert**, passing a message from C# while testing behavior introduced in a specific .NET update.

The **InvokeJSApiMethodButton_Clicked** event handler triggers a JavaScript function that performs an API call, demonstrating asynchronous data retrieval. More complex logic is handled in the **InvokeJSMethodButton_Clicked** event handler, which calls the **AddNumbers** JavaScript function with numeric parameters and expects a structured result. If the result is null, a fallback message is displayed; otherwise, the result and operation name are shown on the page. Similarly, **InvokeAsyncJSMethodButton_Clicked** calls a JavaScript function that fetches and modifies JSON data, returning a dictionary that is displayed in the app.

All methods are executed on the main thread via **Dispatcher.Dispatch**. This ensures that the **statusText** element is updated inside the same calling thread. The **hmv_RawMessageReceived** event handler method listens for incoming messages from JavaScript and appends them to the status text. The nested **ComputationResult** class defines the structure of data exchanged between C# and JavaScript. The **SampleInvokeJSContext** class derives from **JsonSerializerContext** that acts as a precompiled registry for supported JSON shapes. By registering types like **double**, **string**, and **Dictionary<string, string>** with **[JsonSerializable]** attributes, the project ensures robust and fast serialization even under AOT compilation.



Note: AOT stands for ahead-of-time compilation and, in the context of .NET MAUI, it refers to the process of compiling code into native machine code before the

application runs, rather than at runtime (as with Just-in-Time or JIT compilation). This approach improves startup performance, reduces memory usage, and is essential for platforms that don't support runtime code generation—like iOS or when using full linking and trimming for smaller app sizes.

Finally, the `ComputeJSInvokeTarget` class exposes a method called `AddNumbers`, which performs a simple addition and sends the result back to the UI, demonstrating how JavaScript can invoke native C# logic. Altogether, this code provides a robust example of hybrid communication in a MAUI app, blending asynchronous operations, structured data exchange, and UI updates across the JavaScript-C# boundary.

Running the application

In Visual Studio, press F5 to start debugging the application in the target device (physical or emulator) of choice. Once the application is displayed (see Figure 3), make sure that the `HybridWebView` has rendered the web content by checking for the **Hybrid sample!** heading and visible buttons. When ready, select **Send** at the top, and you will see a message appear inside the `HybridWebView`, as shown in Figure 3.

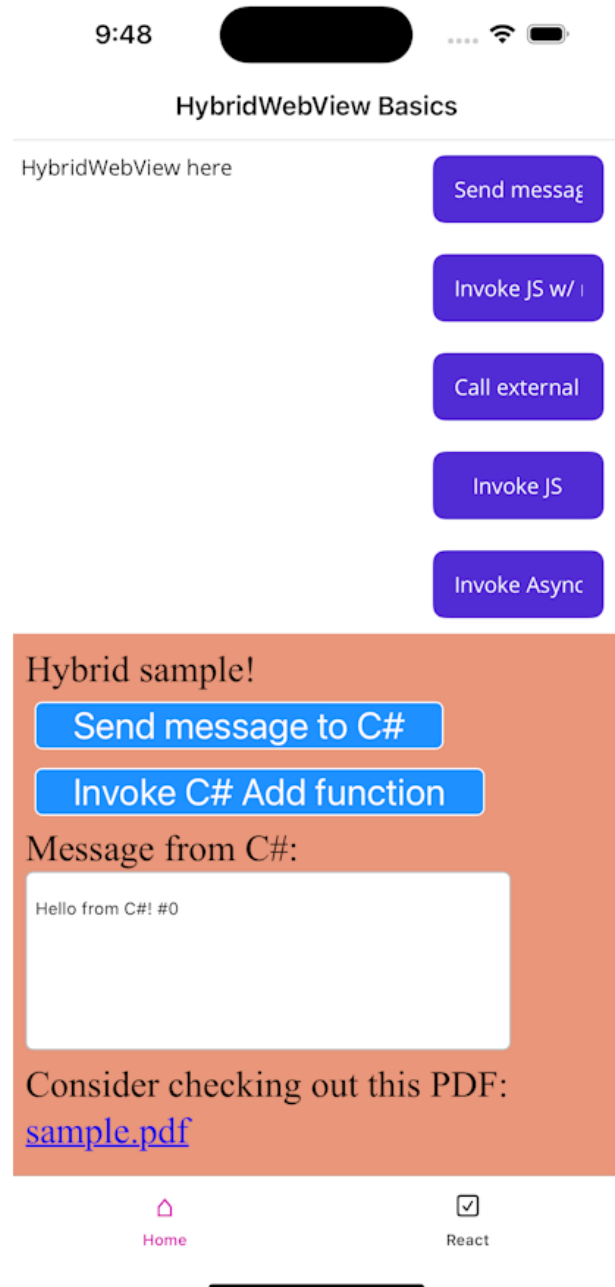
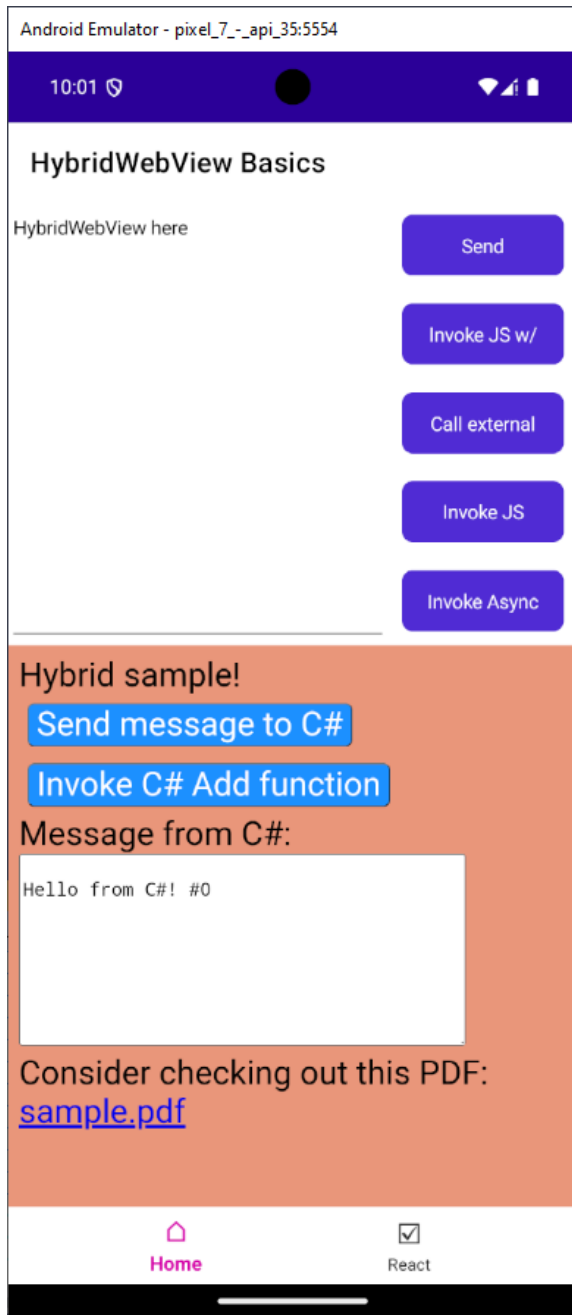


Figure 3: Running the hybrid app

Now select **Invoke JS** and ensure that the computed result appears in the **Editor**, as shown in Figure 4.

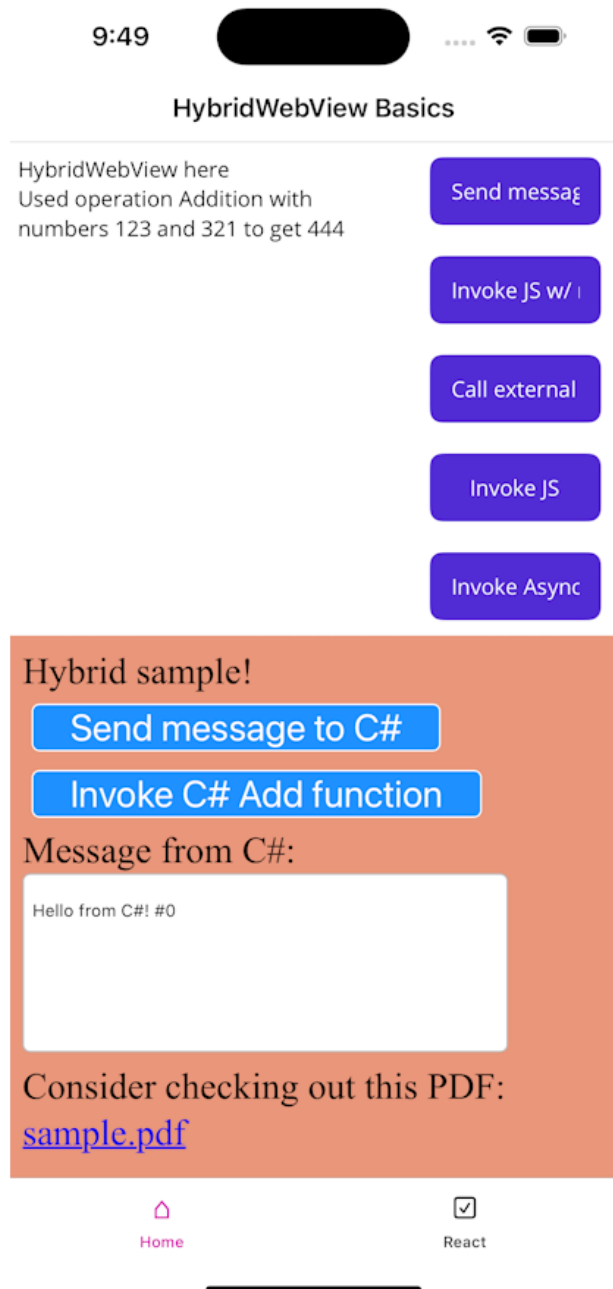
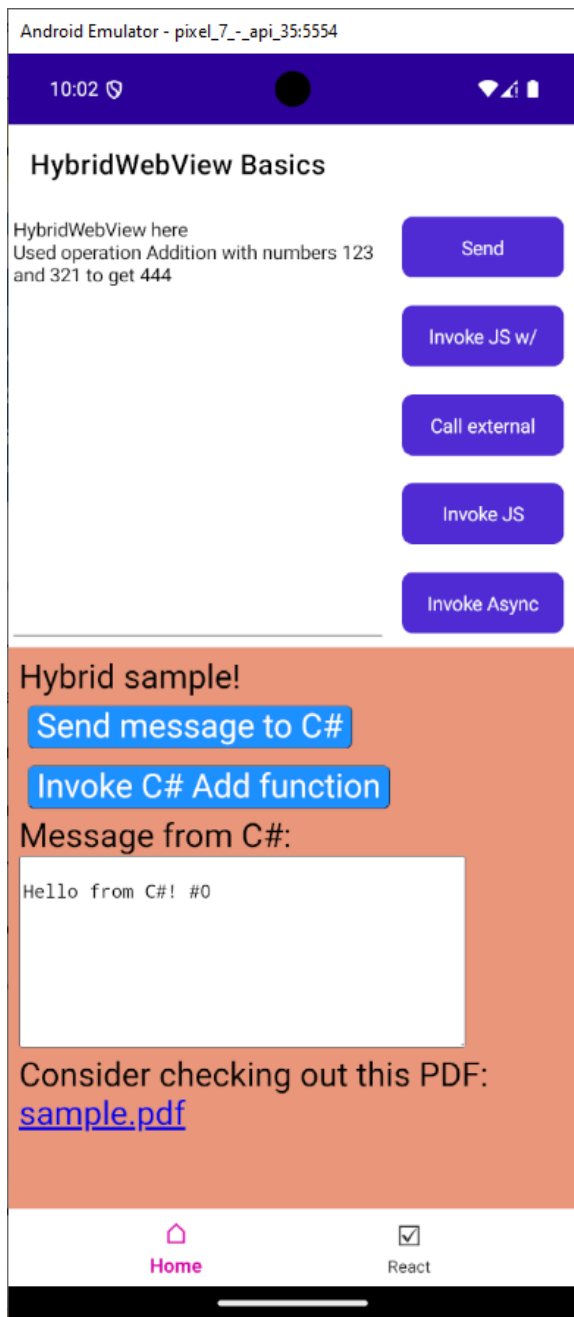


Figure 4: Testing the app functionalities

Each interaction confirms the app's message bridge is functioning smoothly across both layers. It is left to your curiosity to try all the other features. The key concept is that you have seen how to leverage the new **HybridWebView** control to provide built-in interaction between C# and web content inside hybrid apps.

Chapter summary

This chapter explained the structure and functionality of a hybrid .NET MAUI app that enables seamless communication between C# and JavaScript through the new **HybridWebView** control. You have gone through each key component, from the interop logic in the `HybridWebView.js` script, to the HTML structure of `index.html`, and the native layout and back-end logic in the XAML and code-behind. By examining message flows, method invocations, and return value handling across both layers, we demonstrated how web and native code can work together in real time.

In the next chapter, you will lay the foundation for a more advanced scenario—interacting with existing websites.

Chapter 5 Wrapping Existing Websites

Wrapping an existing website into a mobile app is very common, but simply displaying a website in a mobile web viewer is not enough. In fact, both Apple and Google require developers to implement mobile experience features (such as biometric login or sensor integration); otherwise, the application will be rejected from the stores' submission processes.

In .NET MAUI, you use the **WebView** control to display an existing website, and then incrementally add mobile experience features, as you will see in the next chapters. But first, you need an existing website that will be used as the base. Here, you will create a sample ASP.NET Core web application that includes various functionalities, such as displaying a PDF document, interacting with device sensors via JavaScript, handling user profiles with different input types, enabling page printing, and managing file uploads. You will also deploy the website to Azure to demonstrate in-cloud deployment. Finally, you will create a new .NET MAUI project, adding the minimum code required to display a website, and that will be incrementally developed in the next chapters.

Creating a sample ASP.NET Core web application

To simulate the web content that the app will wrap, you will create a simple ASP.NET Core Razor Pages website with a login form and a few additional pages. The sample web app will implement features that will be later wrapped into a native mobile experience. This includes page navigation, biometric authentication, file sharing, document printing, sensor interaction, and, in general, JavaScript connection with C# via bridge classes. To accomplish this:

1. Open Visual Studio 2022.
2. Select **Create a new project**.
3. In the search bar of the **Create a new project** dialog, type **ASP.NET Core web app** and select the **ASP.NET Core Web App (Razor Pages)** template, as shown in Figure 5, and click **Next**.
4. For **Project name**, enter **WrapperWebApp**. Choose a suitable **Location** for your project and select **Next**.
5. On the **Additional information** screen, select **Individual Accounts** for authentication. This will set up the necessary scaffolding for user registration, login, and identity management.
6. On the same screen, make sure that **Configure for HTTPS** is checked and that **Enable Docker** is unchecked.
7. Select **Create**.

Visual Studio will now create the project with the basic structure for an ASP.NET Core Razor Pages application, including identity management.

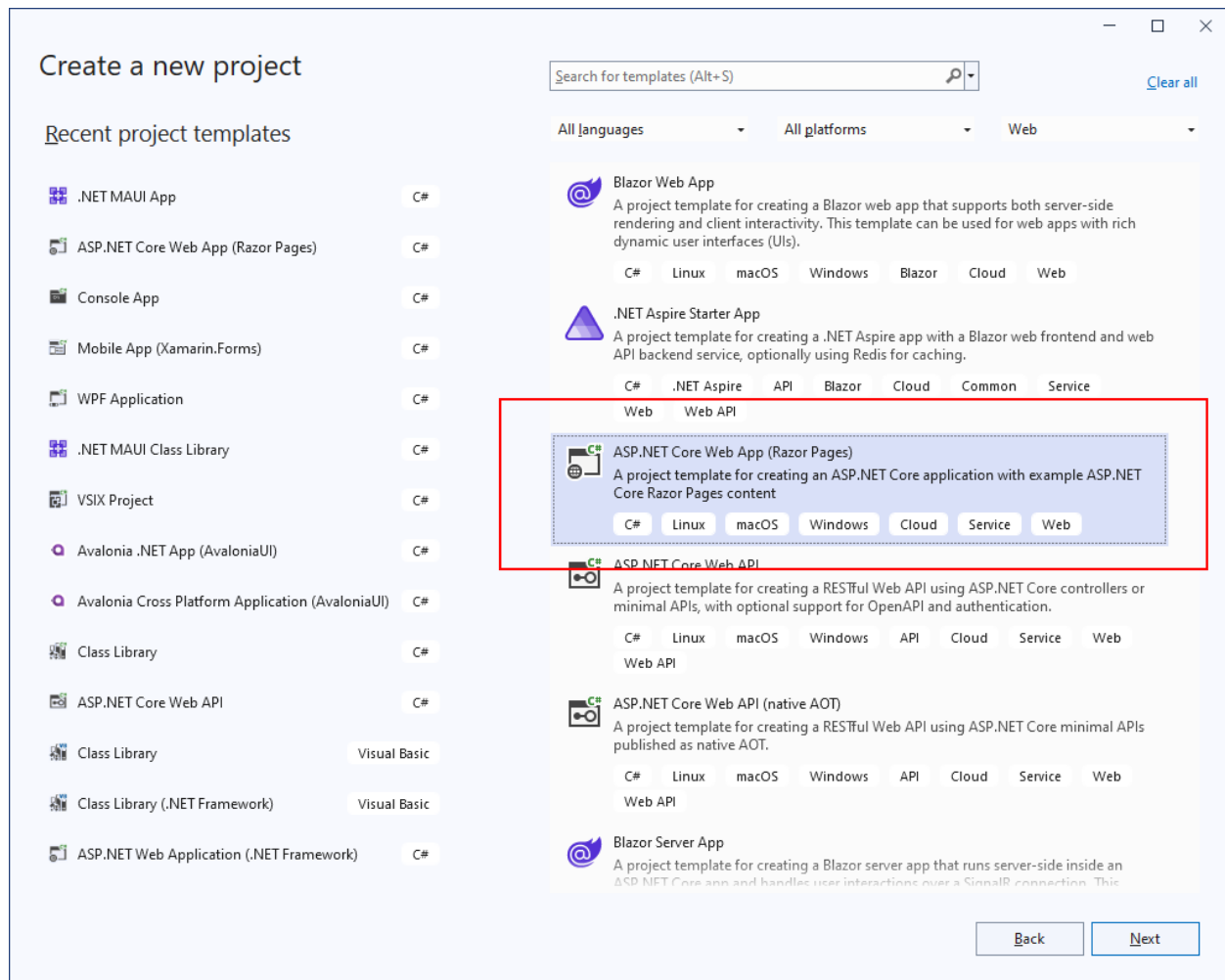


Figure 5: Creating an ASP.NET Core web application

Before adding pages to the website, you will now make some edits to support SQL Server as the database used for authentication.

Configuring SQL Server and migrations

By default, ASP.NET Core Identity uses SQL LocalDB for development. However, you will now configure the project to use a full SQL Server instance, which is more typical in real-world scenarios, and more suitable for Azure deployments. First, you will need to install the following NuGet packages:

- Microsoft.AspNetCore.Identity.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools (for migrations)

To install a package, right-click on your project in Solution Explorer, and **select Manage NuGet Packages**. Go to the **Browse** tab, search for the package, and click **Install**.

The next step is updating the `appsettings.json` file, which contains, among other things, the connection string. Open the **appsettings.json** file in your project and modify the **DefaultConnection** string to point to your SQL Server instance. Replace **(localdb)\mssqllocaldb** with your local SQL Server instance name (e.g., **YOUR_SERVER_NAME**):

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"YOUR_SERVER_NAME;Database=WrapperWebAppDb;Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```



Note: Updating the connection string now is required for development and debugging. However, once you are ready to deploy the application, you will need to update it again to point to the SQL Azure database hosted in the cloud.

Replace **YOUR_SERVER_NAME** with the actual name of your SQL Server instance. If you are using SQL Server Express, it might be **.\\SQLEXPRESS**. For a full SQL Server instance, it could be just the server name. **TrustServerCertificate=True** is often used for development with self-signed certificates; for production, you should use proper certificate validation.

Applying Entity Framework migrations

Because you changed the target database, you will need to apply Entity Framework Core migrations to create the database schema for Identity management. To accomplish this, follow these steps:

1. Open the Package Manager Console in Visual Studio (**Tools > NuGet Package Manager > Package Manager Console**).
2. Ensure **WrapperWebApp** is selected as the **Default project**.
3. Execute the following commands:

> **Add-Migration InitialIdentitySetup**

> **Update-Database**

The **Add-Migration** command creates a new migration file that defines the database schema based on your **ApplicationDbContext** class. The **Update-Database** command applies these changes to your configured SQL Server database, creating the necessary tables for ASP.NET Core Identity.

Configuring the project startup

Since you have configured a SQL Server database to manage authentication, you will also need to update the project startup accordingly. This is accomplished by modifying the Program.cs file as shown in Code Listing 5.

Code Listing 5

```
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using WrapperWebApp.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
var connectionString = builder.Configuration.GetConnectionString("DefaultCo
nnection") ?? throw new InvalidOperationException("Connection string 'Defau
ltConnection' not found.");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));
builder.Services.AddDatabaseDeveloperPageExceptionFilter();

builder.Services.AddDefaultIdentity<IdentityUser>(options => options.SignIn
.RequireConfirmedAccount = true)
    .AddEntityFrameworkStores<ApplicationDbContext>();
builder.Services.AddRazorPages();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseMigrationsEndPoint();
}
else
{
    app.UseExceptionHandler("/Error");
    // The default HSTS value is 30 days. You may want to change this for p
roduction scenarios, see https://aka.ms/aspnetcore-hsts.
    // HTTP Strict Transport Security

    app.UseHsts();
}

app.UseHttpsRedirection();

app.UseRouting();
```

```
app.UseAuthorization();

app.MapStaticAssets();
app.MapRazorPages()
    .WithStaticAssets();

app.Run();
```

The **ApplicationDbContext** class, which inherits from **DbContext**, is the .NET access point to the database and is registered using **AddDbContext<T>**, and it's configured with **UseSqlServer(connection string)**. This tells Entity Framework Core to connect to a SQL Server database defined by the connection string. Entity Framework Core handles the underlying SQL generation and data tracking for you.

The application also configures ASP.NET Core Identity through **AddDefaultIdentity<IdentityUser>**, which uses EF Core behind the scenes via **AddEntityFrameworkStores<ApplicationDbContext>** to manage user accounts and roles in the same database. Identity stores and retrieves user info like usernames, hashed passwords, and claims using Entity Framework's data access layer.

To control access and security, the middleware pipeline includes the **UseAuthentication** and **UseAuthorization** methods. These rely on the Identity system and EF Core's backing store to verify users and enforce access rules. Finally, the **MapStaticAssets** method enables serving static content like CSS, JavaScript, and images from the `wwwroot` directory. In this case, it will be used to serve a PDF document.

Defining webpages

You will now add a few webpages to the application, and each page will be the target of the mobile experience implementation in the .NET MAUI app that you will create in the next chapters. For every page, you will follow these steps:

1. In the Solution Explorer, right-click the **Pages** folder and select **Add > New Item**.
2. In the **Add New Item** dialog, select the **Razor Page – Empty** item template (see Figure 6).
3. Enter the page name and select **Add**.

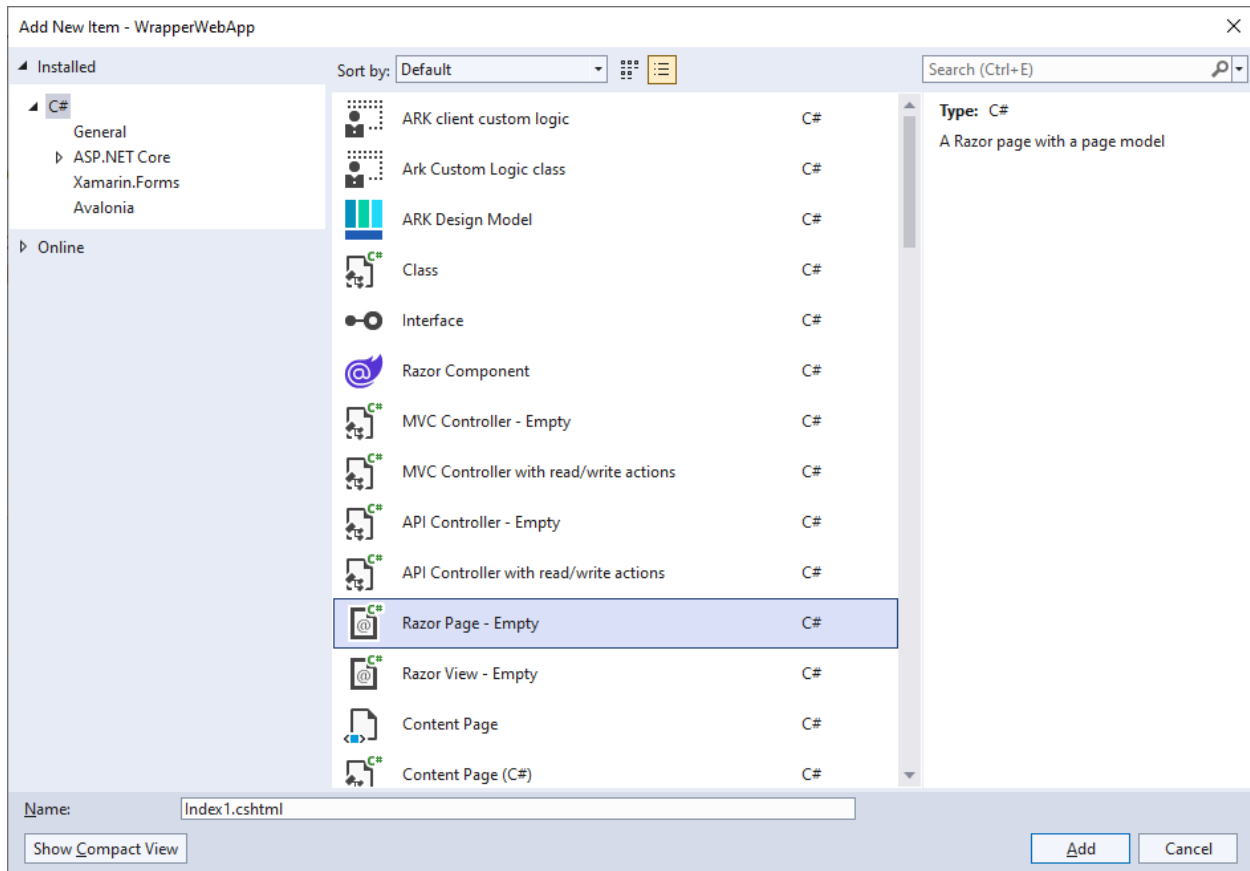


Figure 6: Adding new Razor pages to the project

Page names will be specified in the next sections, when you create new ones. Remember that the Log in and Registration pages are already defined by Visual Studio when creating the solution, and you do not need to make changes to them. For this reason, they will not be discussed from an implementation point of view, and they will be used when testing the application.

Displaying a PDF document

The first page that you add allows for displaying a PDF document. In the web application, you will see the PDF document opened in a browser tab. In the mobile app, you will need to override this behavior and invoke each operating system printing API.

Place a PDF document of your choice into the **wwwroot/files** folder of the project and make sure its name is **sample.pdf**. The companion solution already contains a sample PDF document that you can use. Add a new page called **Brochure.cshtml** and change its code as shown in Code Listing 6.

Code Listing 6

```
@page
@model WrapperWebApp.Pages.BrochureModel
```

```
@{
    ViewData["Title"] = "Product Brochure";
}

<h1>@ViewData["Title"]</h1>

<p>You can view or download our brochure by clicking the link below:</p>

<a href="/files/brochure.pdf" target="_blank">📄 View the PDF Brochure</a>
```

The relevant part of this page's code is the `<a href>` element, which opens the PDF document in a separate browser tab (`target="_blank"`). The path is relative to the web application's root. In the code-behind file, make sure the class name is decorated with the **Authorize** attribute as follows:

```
[Authorize]
public class BrochureModel : PageModel
```

This ensures that the page will be accessible to logged-in users only.

Sensor interaction: Geolocation

The next page you want to add is called **Geolocation.cshtml**, and it demonstrates how to display the current location in the browser. The code is shown in Code Listing 7.

Code Listing 7

```
@page
@model WrapperWebApp.Pages.GeoLocationModel
@{
    ViewData["Title"] = "Geolocation";
}

<h1>@ViewData["Title"]</h1>

<button onclick="getLocation()">Get Location</button>

<p id="status"></p>
<p id="location"></p>

@section Scripts {
    <script>
        function getLocation() {
            const status = document.getElementById('status');
            const locationDisplay = document.getElementById('location');

            if (!navigator.geolocation) {
```

```

        status.textContent = 'Geolocation is not supported by your
browser.';
        return;
    }

    status.textContent = 'Locating...';

    navigator.geolocation.getCurrentPosition(
        (position) => {
            const latitude = position.coords.latitude.toFixed(6);
            const longitude = position.coords.longitude.toFixed(6);

            status.textContent = 'Location found!';
            locationDisplay.textContent = `Latitude: ${latitude}, L
ongitude: ${longitude}`;
        },
        (error) => {
            switch (error.code) {
                case error.PERMISSION_DENIED:
                    status.textContent = 'Permission denied. Please
allow location access.';
                    break;
                case error.POSITION_UNAVAILABLE:
                    status.textContent = 'Location information is u
navailable.';
                    break;
                case error.TIMEOUT:
                    status.textContent = 'Location request timed ou
t.';
                    break;
                default:
                    status.textContent = 'An unknown error occurred
.';
                    break;
            }
        }
    );
}
</script>
}

```

The core of this page is the **getLocation** JavaScript function that retrieves and displays the user's current position using the browser's Geolocation API. It starts by referencing two HTML elements, **status** and **locationDisplay**, which are used to show real-time messages and coordinate data.

The function first checks if geolocation is supported via `navigator.geolocation`. If it isn't, it updates the status to let the user know and exits. If supported, `navigator.geolocation.getCurrentPosition` is called. While the location is being retrieved, a **Locating...** message is shown. On success, the browser returns a position object, from which `position.coords.latitude` and `position.coords.longitude` are extracted and rounded to six decimal places using `.toFixed(6)`. These values are then shown on the page.

In case of failure, the error callback handles different **PositionError** types using a switch statement:

- **PERMISSION_DENIED**: The user refused to share their location, either by declining the browser prompt or through settings.
- **POSITION_UNAVAILABLE**: The browser couldn't determine the location, often due to GPS being off or the device lacking sufficient signal.
- **TIMEOUT**: The attempt to retrieve the location took too long and was automatically canceled.

Each of these outcomes updates the status element with a helpful message, ensuring users understand what happened and why. The code of this page will also be overridden in the mobile wrapper app because it's not supported on all platforms and because you want to retrieve and manipulate the location values on the mobile side.

Like for the previous page, make sure that the **[Authorize]** attribute is applied to the class definition so that the page is accessible only after the user logs in.

Adding a user profile page

The next page you want to add is called **MyProfile.cshtml**, and its purpose is to collect basic user information, such as the full name, date of birth, and phone number. Later in this book, you will make sure that the proper on-screen keyboard is displayed in the mobile app, depending on the data type. Though the most recent mobile operating systems' WebView components are able to automatically detect the proper keyboard based on the data type, this is not necessarily true on older devices and system versions, so you will want to ensure that the behavior stays consistent.

Once you have added the page, specify the user interface code as shown in Code Listing 8.

Code Listing 8

```
@page
@model WrapperWebApp.Pages.MyProfileModel
@using Microsoft.AspNetCore.Authorization
@{
    ViewData["Title"] = "My Profile";
}

<h1>@ViewData["Title"]</h1>

<form method="post">
```

```

<div class="form-group">
    <label asp-for="FullName"></label>
    <input asp-for="FullName" class="form-control" />
    <span asp-validation-for="FullName" class="text-danger"></span>
</div>

<div class="form-group">
    <label asp-for="DateOfBirth"></label>
    <input asp-for="DateOfBirth" class="form-control" type="date" />
    <span asp-validation-for="DateOfBirth" class="text-danger"></span>
</div>

<div class="form-group">
    <label asp-for="PhoneNumber"></label>
    <input asp-for="PhoneNumber" class="form-control" id="user-phone-
number" />
    <span asp-validation-for="PhoneNumber" class="text-danger"></span>
</div>

<button type="submit" class="btn btn-primary">Save</button>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

This page is designed to collect user input, including the date of birth and phone number, using HTML5-enhanced fields for a better user experience. The date input field uses **type="date"**, allowing modern browsers to present a calendar picker or native date selector. For the phone number input, a custom **id="user-phone-number"** is applied. This ID can be leveraged by additional scripts or mobile platform wrappers to trigger a numeric keypad or attach custom behavior like formatting or validation, as you will see in the next chapters.

At the bottom of the form, the Submit button allows users to send their input for processing. Client-side validation is also enabled via the Razor directive **@section Scripts { <partial name="_ValidationScriptsPartial" /> }**, which injects a reusable script into the page. This typically leverages jQuery Validation, ensuring that users receive immediate feedback on input errors before the data is ever sent to the server. The C# code-behind file of the page defines the properties and the page behavior as follows:

```

[Authorize]
public class MyProfileModel : PageModel
{
    [BindProperty]
    [Required]
    public string FullName { get; set; }

    [BindProperty]
    [DataType(DataType.Date)]

```

```

    [Required]
    public DateTime? DateOfBirth { get; set; }

    [BindProperty]
    [Required]
    [Phone]
    public string PhoneNumber { get; set; }

    public void OnGet()
    {
        // Optionally load preexisting user data
    }

    public IActionResult OnPost()
    {
        if (!ModelState.IsValid)
        {
            return Page();
        }

        // Save data (e.g., to DB or user claims)
        TempData["Message"] = "Profile saved successfully!";
        return RedirectToPage(); // Or stay on the page
    }
}

```

Notice how the **OnPost** method performs a data model validation before sending the data to the server.

Printing page content

Another feature that you will need to specifically handle in a mobile app, and which wraps an existing website, is printing. For this purpose, you can edit the existing **Privacy.cshtml** page, adding the following line as the last one on the page:

```
<button id="printButton" onclick="window.print()">Print this page</button>
```

This button will invoke the printing API so that the page can be printed. It's not necessary to add the **[Authorize]** attribute to the class definition here, because a privacy policy page is usually visible without needing to log in.



Note: While printing a webpage in the browser requires one line of code, you will see that overriding this behavior in a mobile wrapper app requires more effort with platform-specific code.

Implementing page navigation

All the pages that you have created so far must be added to the common navigation stack, which is provided by the `_Layout.cshtml` file, located under `Pages\Shared`. Edit the file content by changing the central `<div>` element as follows:

```
<div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
    <ul class="navbar-nav flex-grow-1">
        <li class="nav-item">
            <a class="nav-link text-dark" asp-area="" asp-page="/Index">Home</a>
        </li>
        <li class="nav-item">
            <a class="nav-link text-dark" asp-area="" asp-page="/Privacy">Privacy</a>
        </li>
        <li class="nav-item">
            <a class="nav-link text-dark" asp-page="/Brochure">Brochure</a>
        </li>
        <li class="nav-item">
            <a class="nav-link text-dark" asp-page="/Upload">Upload</a>
        </li>
        <li class="nav-item">
            <a class="nav-link text-dark" asp-page="/Geolocation">Geolocation</a>
        </li>
        <li class="nav-item">
            <a class="nav-link text-dark" asp-page="/MyProfile">My
Profile</a>
        </li>
    </ul>

    <partial name="_LoginPartial" />
</div>
```

All page hyperlinks will be visible from the registration and login pages, but those marked with **[Authorize]** will redirect to the login page if the user attempts to open them before logging in.

At this point, rebuild the solution and make sure that it builds without any errors.

Running the web application

Before deploying the application to a remote web server, you will now test the application locally. Press F5 to start debugging, and when the browser displays the home page, open the **Privacy** page and click **Print this page** (see Figure 7).

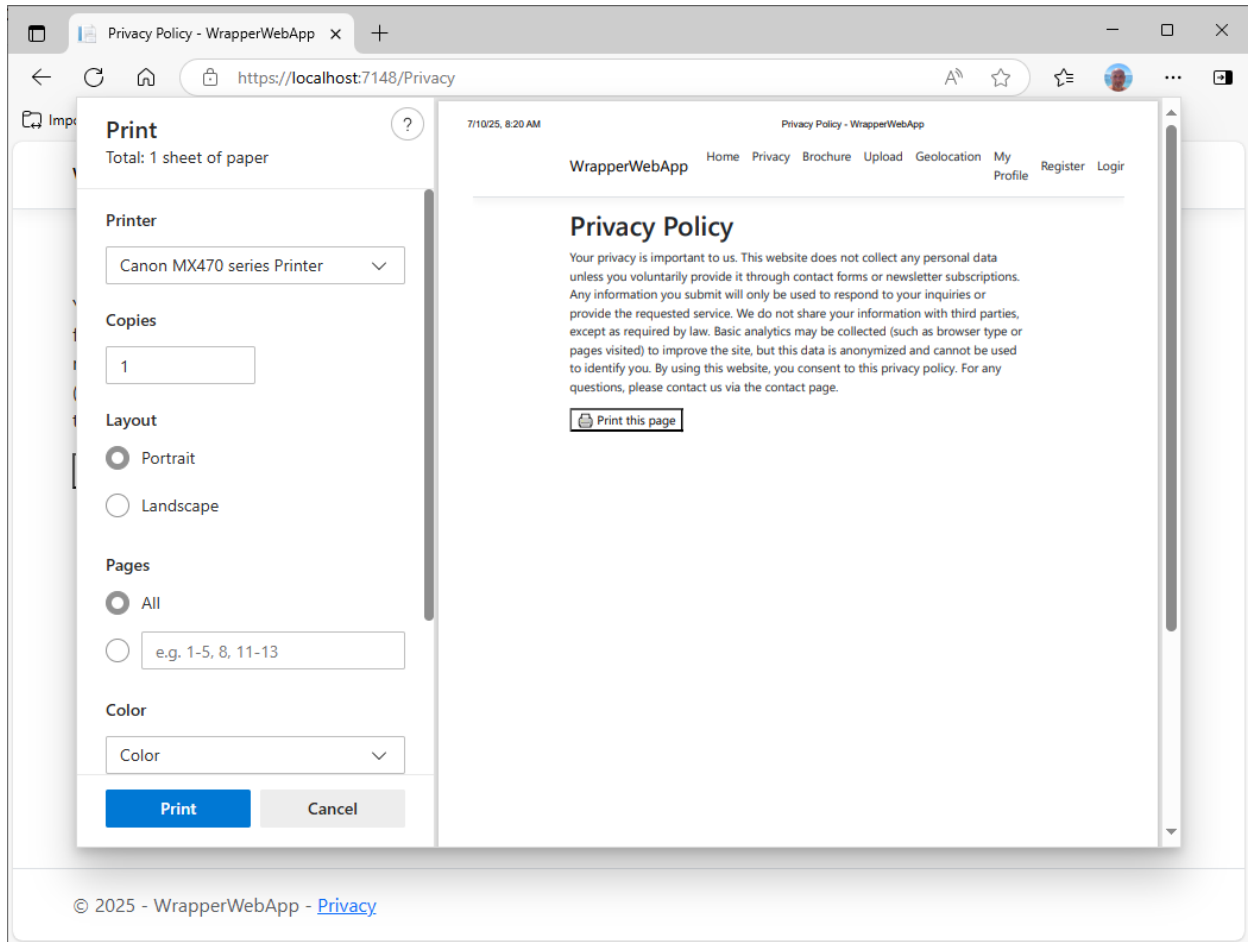


Figure 7: Printing a webpage

It is not necessary to print the page; this is just to demonstrate the feature. Close the print dialog, select **Register**, and supply an email address and password for registration (you can use a fake email address for this purpose), as shown in Figure 8.

Register - WrapperWebApp

https://localhost:7148/Identity/Account/Register

WrapperWebApp Home Privacy Brochure Upload Geolocation My Profile Register Login

Register

Create a new account.

Email
alessandro.delsole@something.com

Password
.....

Confirm Password
.....

Register

Use another service to register.

There are no external authentication services configured. See this [article about setting up this ASP.NET application to support logging in via external services.](#)

© 2025 - WrapperWebApp - [Privacy](#)

Figure 8: Registering a new account

You should be automatically logged in; if not, open the **Log in** page and enter the newly created credentials. At this point, the remaining webpages will be accessible. If you go to the **Brochure** page and select **View the PDF brochure**, a new browser tab will display a sample PDF document (see Figure 9). This feature will be useful to demonstrate how to intercept separate tab requests and how to handle PDF documents properly in a MAUI project.

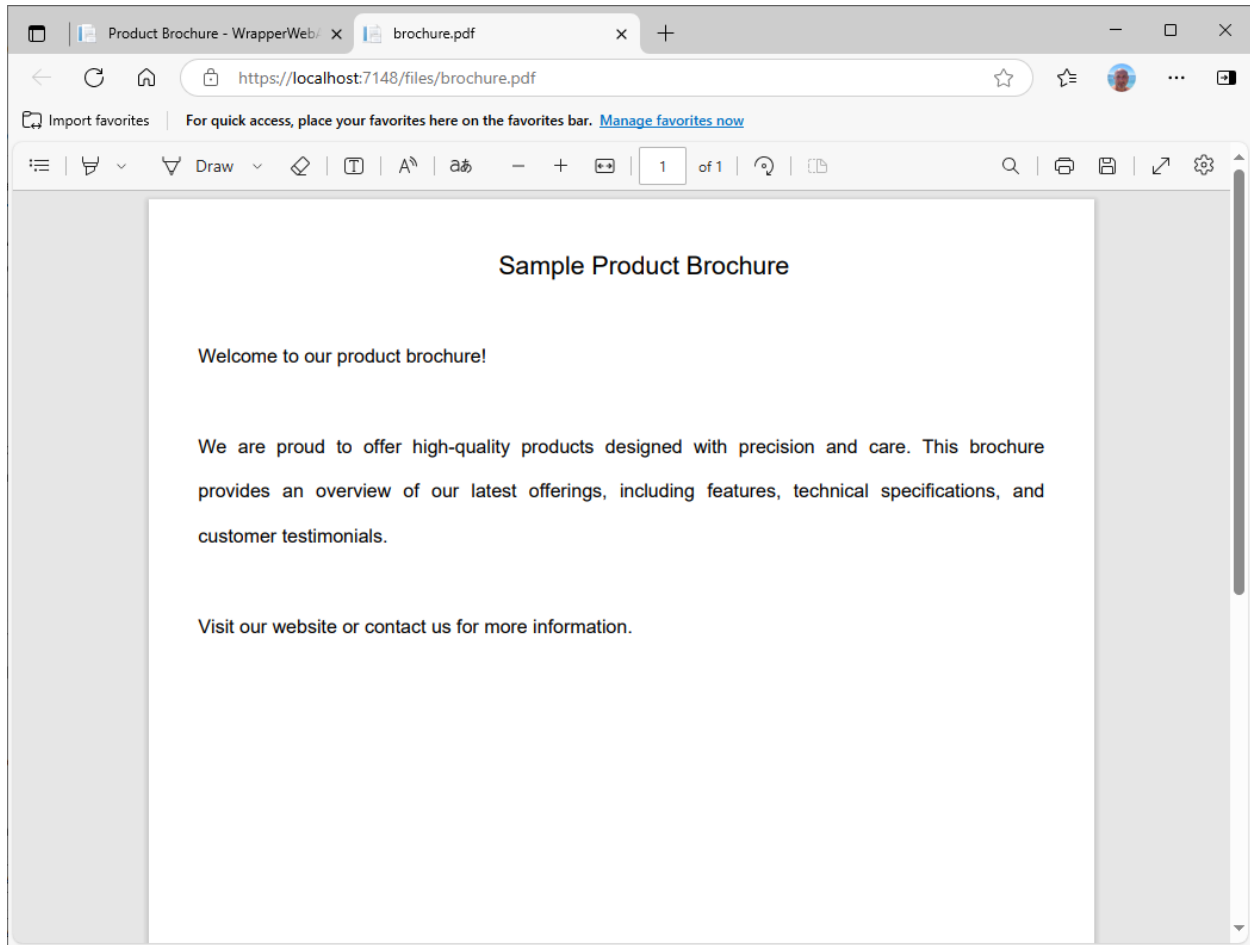


Figure 9: Displaying a PDF document

Close the **Document** tab, open the **Geolocation** page in the web application (see Figure10), and select **Get Location** to display the current location. You will need to grant your browser permissions to retrieve it.

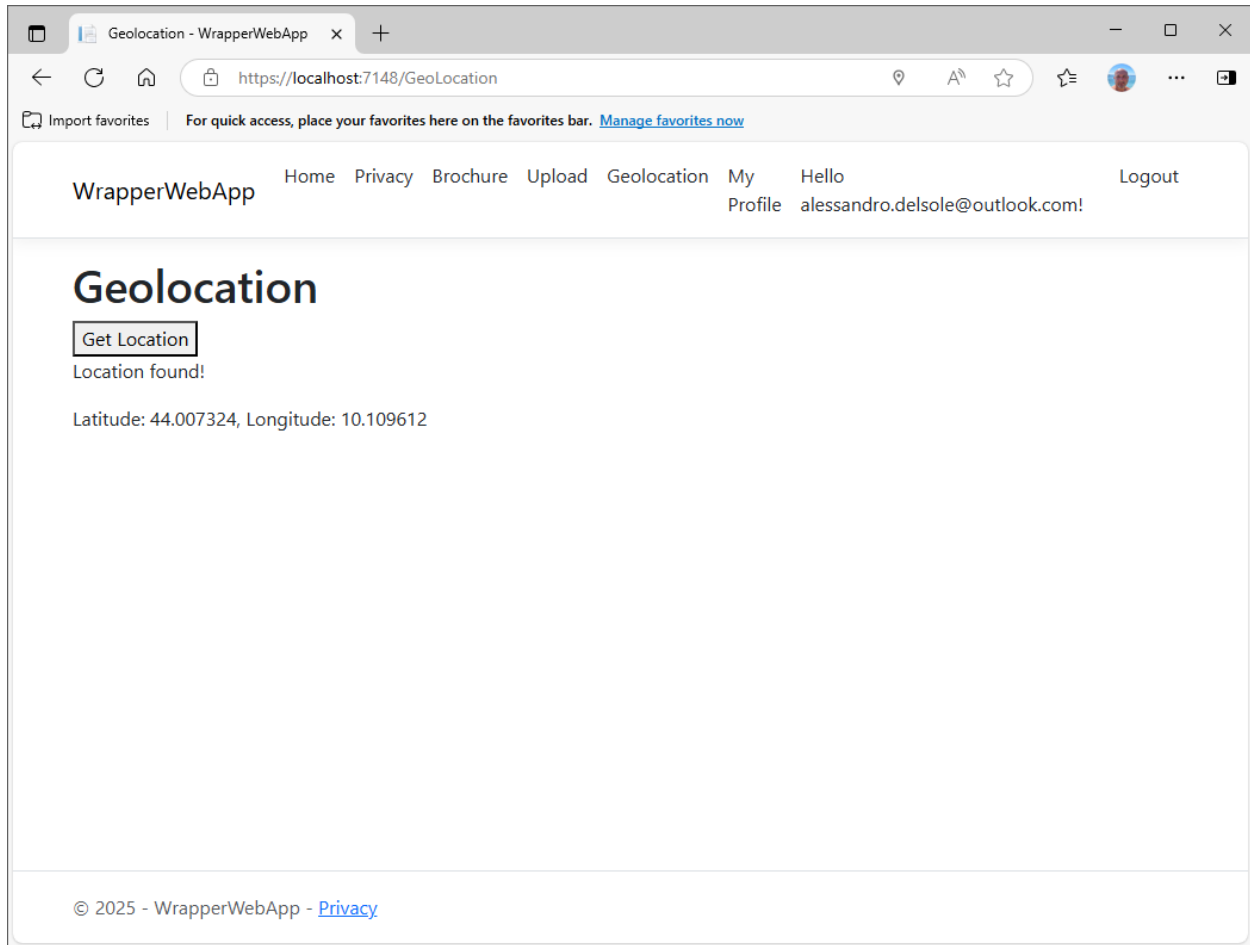


Figure 10: Retrieving the current location

This feature allows for demonstrating sensor interaction in a MAUI project. Once you select **Upload**, the selected file will be uploaded to the web app's **uploads** subfolder for demonstration purposes.

The last step is opening the **My Profile** page (see Figure 11), where you can supply some contact information. This will be used to demonstrate how to ensure that a proper on-screen keyboard is displayed in a mobile app according to the field's data type.

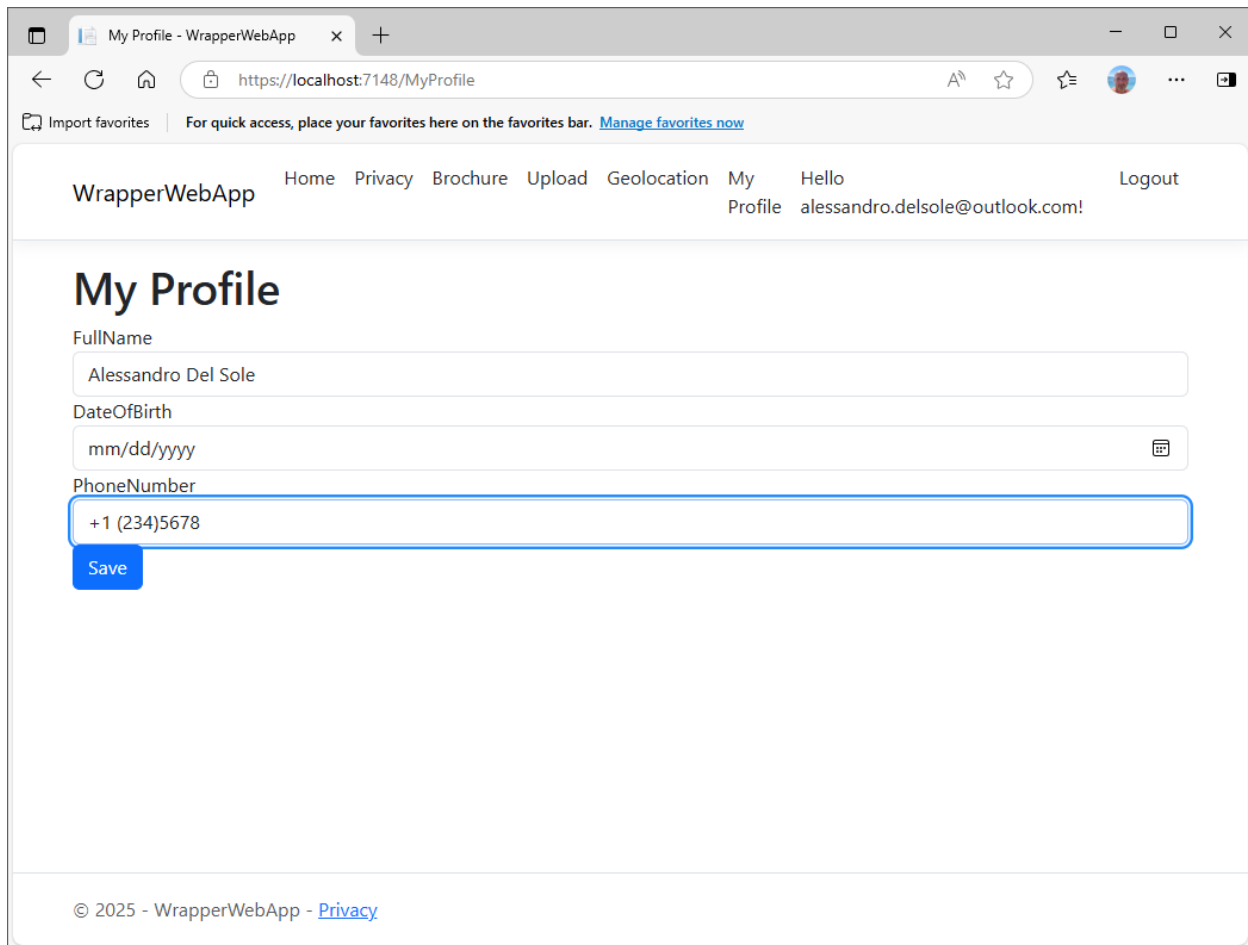


Figure 11: Adding sample contact data

Now that you have verified that the application is properly running on your development machine, you can deploy it to a web server.

Deploying the web application to Azure

To make your web application accessible to a .NET MAUI mobile wrapper, you must deploy it to a publicly accessible web server. For the sake of simplicity, we will use Azure App Service, which provides a robust, scalable platform for hosting web applications. We will deploy the WrapperWebApp to an Azure Web App and set up a SQL Azure database for identity management. If you do not have an Azure subscription, you can create one by leveraging the [free trial](#) option.

Creating an Azure SQL database

Before deploying your web application, you must provision an Azure SQL Database to host your application's data, including the ASP.NET Core Identity tables. You will first need to create a server, if you don't have one already, and then you will need to create a database. To accomplish this, log in to the [Azure Portal](#) and follow the steps described in the following paragraphs.

Creating a database server

In the Azure Portal search bar at the top, type **SQL Servers** and select **SQL servers** from the results. When the **SQL servers** page appears (see Figure 12):

1. Click the **+ Create** button.
2. On the **Create SQL Database Server** page, specify a resource group, which is required to logically organize Azure resources. In the **Resource group** dropdown, select an existing resource group or click **Create new**. You can optionally change the autogenerated name. A resource group is a logical container for Azure resources.
3. In the **Server name** text box, enter a globally unique name for your SQL server (e.g., **webserversuccinctly**). This name must be unique across all of Azure.
4. In the **Location** box, choose a region close to you or your target audience.
5. In the **Authentication method** option set, select **Use SQL authentication**.
6. In the **Server admin login** text box, enter a username for the server administrator.
7. In the **Password** text box, enter a strong password and confirm it. Save these credentials, as you will need them shortly.

Create SQL Database Server - Mic

+

←

↺

🏠

🔒

https://portal.azure.com/#view/SqlAzureExtension/CreateServerV3Blade/_provisioningContex

Microsoft Azure

🔍 Search

[Home](#) > [SQL servers](#) >

Create SQL Database Server

...

Microsoft

Basics

Networking

Security

Additional settings

Tags

Review + create

SQL database server is a logical container for managing databases and elastic pools. Complete the Basic tab, then go to Review + Create to provision with smart defaults, or visit each tab to customize. [Learn more](#)

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Visual Studio Enterprise Subscription

Resource group *

WrapperWebApp20250529074211ResourceGroup

[Create new](#)

Server details

Enter required settings for this server, including providing a name and location.

Server name *

Enter server name

.database.windows.net

Location *

(Europe) Italy North

Authentication

📘

Azure Active Directory (Azure AD) is now Microsoft Entra ID. [Learn more](#)

Select your preferred authentication methods for accessing this server. Create a server admin login and password to access your server with SQL authentication, select only Microsoft Entra authentication [Learn more](#) using an existing Microsoft Entra user, group, or application as Microsoft Entra admin [Learn more](#), or select both SQL and Microsoft Entra authentication.

Authentication method

☒

Use Microsoft Entra-only authentication

Review + create

Next : Networking >

Figure 12: Creating a new database server

Finally, select **Review + Create** and then **Create** to deploy the SQL server. This process may take a few minutes. When the operation is completed, you need one more step before creating a database, which is to allow connections to the server from specific IP address ranges. To accomplish this, click on the newly created server name and, under the **Security** group, select **Networking** (see Figure 13).

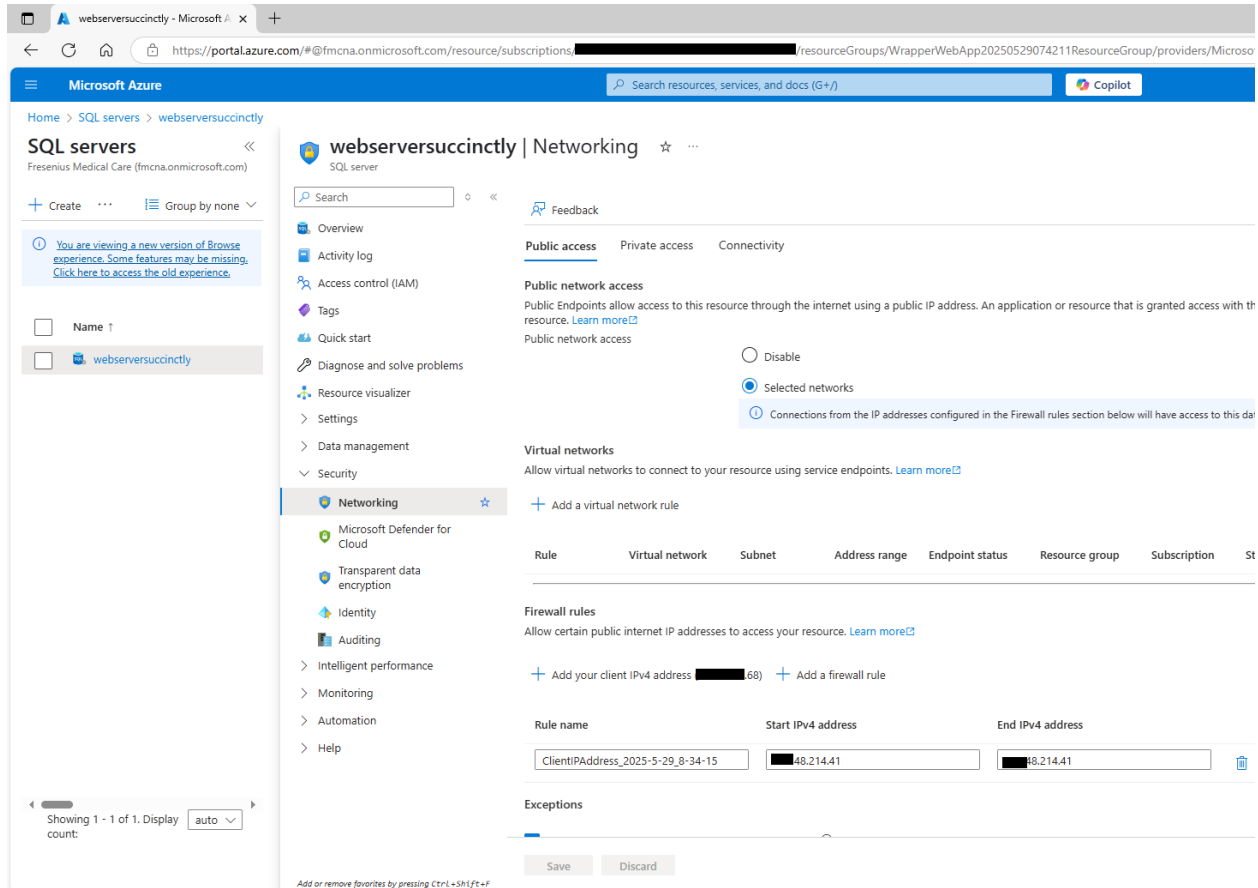


Figure 13: Configuring IP address ranges

Select **Save** when you're ready to move on to creating the database.

Creating a SQL database

Once the SQL server deployment is complete, type **SQL databases** in the search bar and select **SQL databases** from the results. When the **SQL databases** page appears (see Figure 14):

1. Click **+ Create**.
2. On the **Create SQL Database** page, provide the following details, and select the same resource group you created for your SQL server.
 - a. In the **Database name** text box, enter a name for your database.
 - b. In the **Server** text box, select the SQL server you created previously.
 - c. In the **Workload environment** option set, select **Development**, which is appropriate for instructional and testing purposes.
 - d. In the **Backup storage redundancy** option set, select **Locally-redundant** backup storage.

3. Select **Compute + storage** to configure the pricing tier. Given that you have a \$200 credit per month on a free Azure subscription, the **Basic** tier represents the cheapest option.

Microsoft Azure

Home > SQL databases >

Create SQL Database

Microsoft

Basics Networking Security Additional settings Tags Review + create

Create a SQL database with your preferred configurations. Complete the Basics tab then go to Review + Create to provision with smart defaults, or visit each tab to customize. [Learn more](#)

SQL Database Hyperscale: Low price, high scalability, and best feature set. [Learn more](#)

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * Visual Studio Enterprise Subscription

Resource group * WrapperWebApp20250529074211ResourceGroup [Create new](#)

Database details

Enter required settings for this database, including picking a logical server and configuring the compute and storage resources

Database name * webappdatabase ✓

Server * webserver succinctly (Italy North) [Create new](#)

Want to use SQL elastic pool? ☐ Yes ☒ No

Workload environment ☒ Development ☐ Production

Default settings provided for Development workloads. Configurations can be modified as needed.

[Review + create](#) [Next: Networking >](#)

Cost summary

General Purpose (GP_S_Gen5_1)
Cost per GB (in EUR) 0.13
Max storage selected (in GB) x 41.6

ESTIMATED STORAGE COST / MONTH 5.33 EUR
COMPUTE COST / VCORE SECOND ¹ 0.000149 EUR

NOTES
¹ Serverless databases are billed in vCore seconds based on a combination of CPU and memory utilization. [Learn more about serverless billing](#)

Figure 14: Creating a new SQL database

When ready, select **Review + create** and then **Create** to deploy the SQL database and wait for the operation to be completed.

Configuring the database schema

Now that you have an Azure SQL database, you need to copy your local database schema and any existing data (specifically the Identity tables created by Entity Framework Core) to the remote database. This can be accomplished using the Entity Framework Core tools. The first step is modifying the **DefaultConnection** string in the **appsettings.json** file of the **WrapperWebApp** project to point to the new SQL Azure Database, like in the following JSON:

```
"ConnectionStrings": {
```

```

    "DefaultConnection": "Server=tcp:your-server-
name.database.windows.net,1433;Initial Catalog=your-db-name;Persist Security
Info=False;User ID=your-user-name;Password=your-
password;MultipleActiveResultSets=False;Encrypt=True;TrustServerCertificate=
False;Connection Timeout=30;"
  }

```

Replace the placeholders with your actual Azure SQL server name, database name, and the administrator username and password you set previously. **TrustServerCertificate=False** is the recommended setting for production environments when connecting to Azure SQL Database, as Azure provides valid SSL certificates.

The next step is applying Entity Framework data migrations to update the remote schema. Open the Package Manager Console in Visual Studio (**Tools > NuGet Package Manager > Package Manager Console**) and execute the following command:

```
> dotnet ef database update
```

This command will use the connection string in appsettings.json to connect to your Azure SQL Database and apply all pending Entity Framework Core migrations, creating the necessary tables for ASP.NET Core Identity.

Hints for production environments

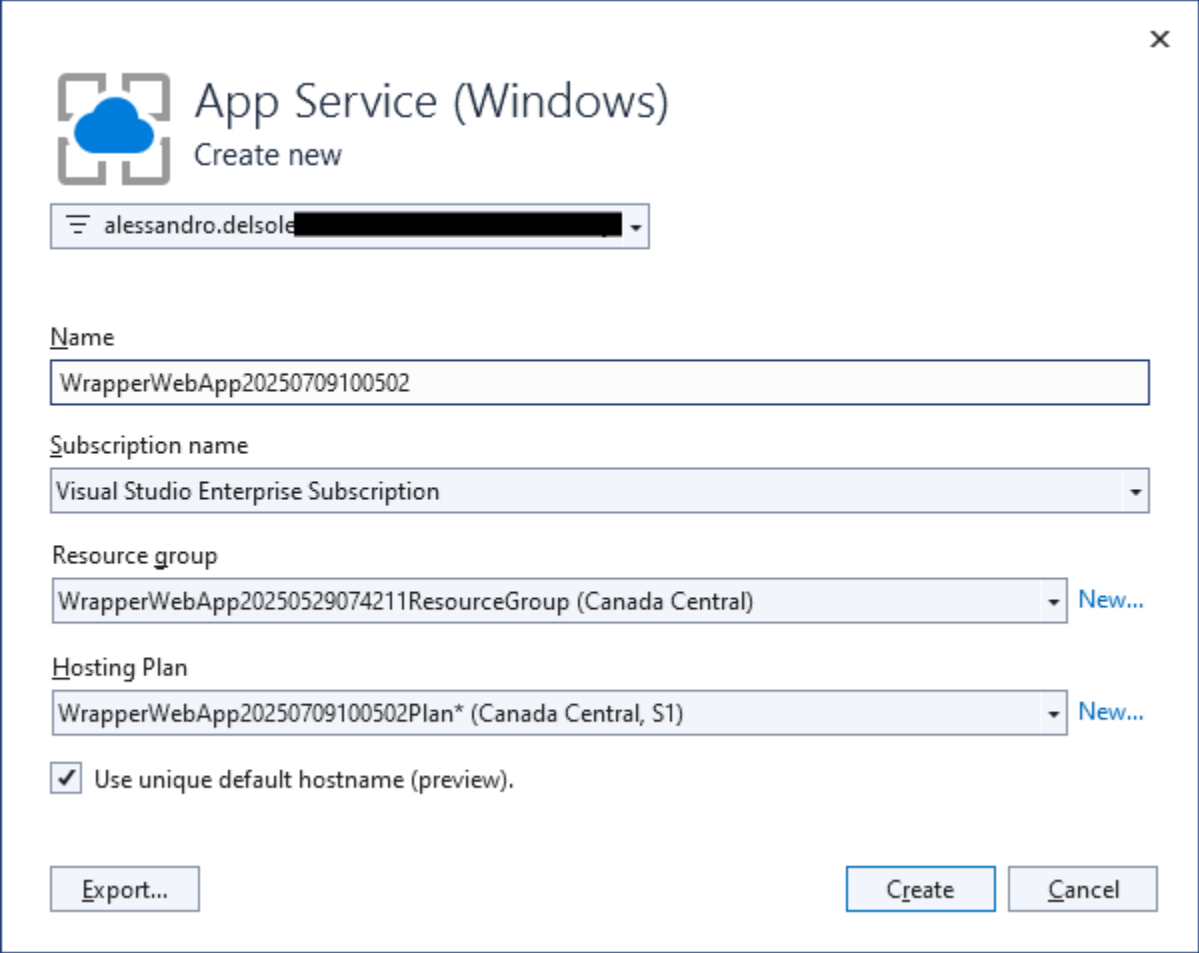
For security best practices, you should not hardcode sensitive information like database passwords directly into your appsettings.json file when deploying to production. Instead, you will use Azure App Service's application settings to securely store the connection string. Though this is not covered here, since you are using the development settings options, the official documentation provides a [comprehensive guide](#) on how to set up a connection string properly for production.

Deploying the web application

Now, you will publish your ASP.NET Core web application to Azure App Service using Visual Studio's built-in publishing tools. To accomplish this, follow these steps:

1. Right-click your **WrapperWebApp** project in the Solution Explorer and select **Publish**.
2. In the **Publish** dialog, choose **Azure** as the target. Select **Next**.
3. Select **Azure App Service (Windows)** as the specific target. Click **Next**.
4. On the App Service screen, select **Create new**. When the **Create App Service** dialog appears (see Figure 15), select the same resource group you used for your SQL database.
5. Enter a globally unique name for your web app (for example, **wrapperwebapp-unique-id**) or leave the autogenerated name unchanged. This name will become part of your application's URL (for example, <https://wrapperwebapp-unique-id.azurewebsites.net>). In my example, I left the autogenerated name unchanged (wrapperwebapp20250529074211).

6. For the **Hosting Plan**, choose an existing plan or select **New** to create a new one. Choose the closest location to you and the free tier (this is enough for testing purposes).
7. Select **Create** so that Visual Studio will provision the Azure App Service.



The screenshot shows the 'App Service (Windows)' dialog box with the following details:

- Title:** App Service (Windows) Create new
- User:** alessandro.delsole
- Name:** WrapperWebApp20250709100502
- Subscription name:** Visual Studio Enterprise Subscription
- Resource group:** WrapperWebApp20250529074211ResourceGroup (Canada Central) [New...]
- Hosting Plan:** WrapperWebApp20250709100502Plan* (Canada Central, S1) [New...]
- Options:** ☒ Use unique default hostname (preview).
- Buttons:** Export..., Create, Cancel

Figure 15: Preparing the Web application for Azure deployment

Once the app service is provisioned, you will be returned to the **Publish** dialog. Select **Finish** to create the publish profile, and then select **Publish** to initiate the deployment. At this point, Visual Studio will:

- Build your application in Release configuration.
- Package the compiled application.
- Deploy the application files to the newly created Azure app service.

You can monitor the deployment progress in the Visual Studio Output window.

Once the deployment is complete, you can open the application URL defined previously and check that the application is running in the browser as expected. You will need to register again because you are now using the remote database and not the development database.

After this check, you are ready to move to the core topic of the book, wrapping an existing website into a .NET MAUI app and implementing the necessary mobile experience.

Creating a .NET MAUI project

The last part of this crucial chapter covers creating a .NET MAUI project and implementing a **WebView** control that displays the website that you developed previously. In the next chapters, you will incrementally implement features that satisfy the mobile experience requirements of Google Play and the Apple App Store.

In Visual Studio, select **File > New > Project**. In the **Create a New Project** dialog, select the **.NET MAUI App** template (see Figure 16), and then select **Next**.

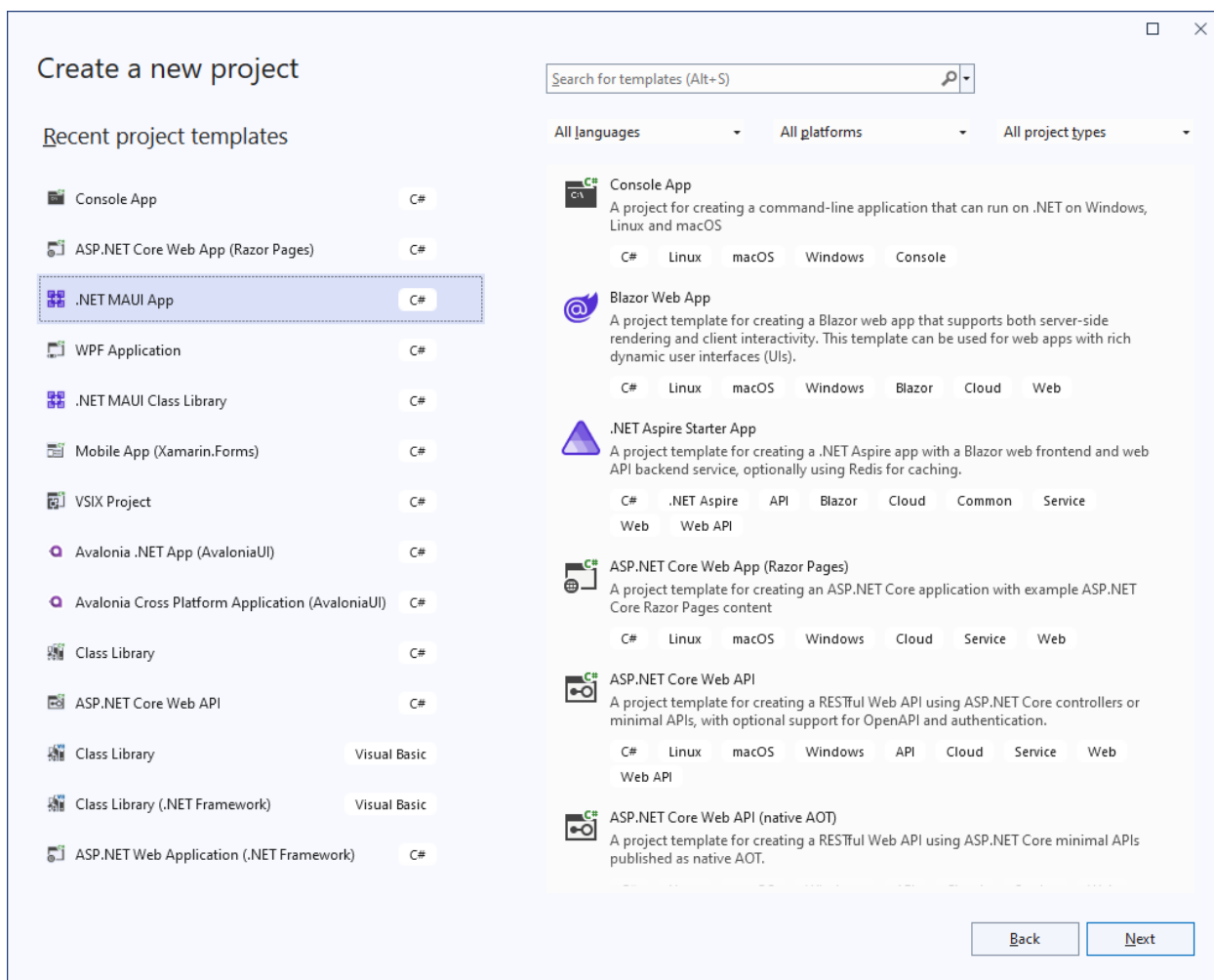


Figure 16: Creating a .NET MAUI project

In the **Configure your new project** dialog, specify **MobileAppWrapper** as the project name and click **Next**. In the final **Additional information** dialog, choose **.NET 9.0 (Standard Term Support)** as the development platform and select **Create**.

When the project is ready, make sure you install the Newtonsoft.Json NuGet package. Though .NET includes an API to manipulate JSON content, without a doubt, the Newtonsoft.Json library has become a *de facto* standard for JSON management over the years.

Implementing a custom WebView

The first step in the new MAUI project is creating a subclass of **WebView**. This is required because you will need to define custom event handlers to detect actions performed on the website. With this in mind, create a **Views** folder in the project and add a **CustomWebView.cs** code file whose content is the following:

```
public class CustomWebView: WebView
{
}
```

No more code is needed now, and you will build features incrementally, starting with the next chapter. In the **MainPage.xaml** file, start editing the user interface by adding the following code:

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              xmlns:views="clr-namespace:MobileAppWrapper.Views"
              x:Class="MobileAppWrapper.MainPage">

    <Grid>
        <Grid x:Name="LayoutRoot">
            <Grid.RowDefinitions>
                <RowDefinition />
                <RowDefinition Height="40"/>
            </Grid.RowDefinitions>
            <views:CustomWebView BindingContext="{Binding}"
                               Source="{Binding Source}" x:Name="RootWebView"/>
            <ActivityIndicator Color="Blue" x:Name="SpinnerGrid"
                               IsVisible="False" IsRunning="True"
                               WidthRequest="60" HeightRequest="60"/>
        </Grid>
    </Grid>
</ContentPage>
```

The user interface defines a **Grid** with two rows. The first row contains the custom **WebView** that displays the website. The second row will be used to host a navigation bar and is empty at the moment. Notice how the **Source** property is not assigned with the website URL directly; instead, it is bound to the **Source** property of a **ViewModel** that you will define shortly. The reason for this is to make it work properly on iOS, which is very different from Android in its native behavior.

Finally, an **ActivityIndicator** will be displayed while the **WebView** is loading pages.

Defining a ViewModel

ViewModels need to raise a property change notification every time one of their property values changes. You could implement the **INotifyPropertyChanged** interface or, more simply, you can leverage the **CommunityToolkit.Mvvm** NuGet package, which provides a ready-to-use series of objects tailored to the MVVM pattern.

In the Solution Explorer, right-click the project name and select **Manage NuGet Packages**. When the NuGet user interface appears (see Figure 17), search for the **CommunityToolkit.Mvvm** library.

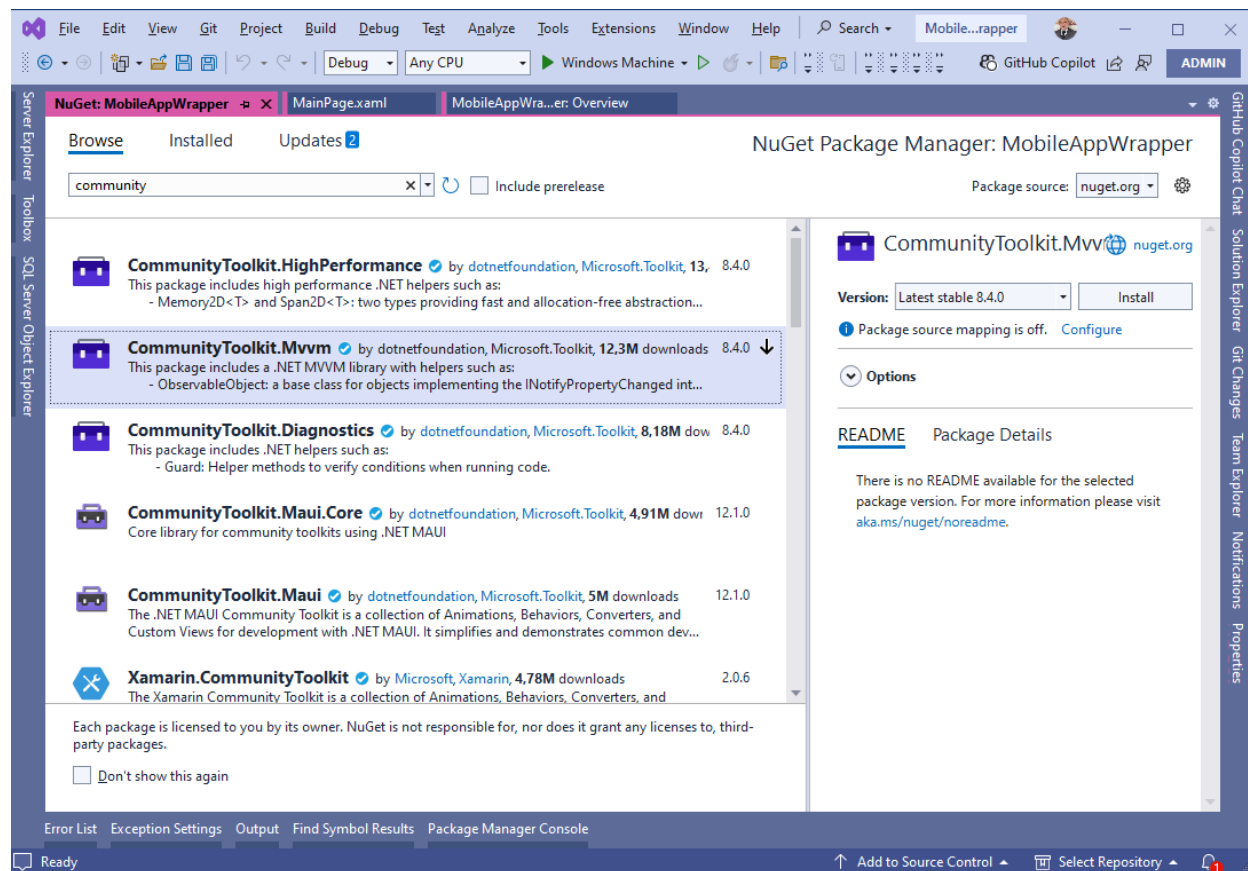


Figure 17: Installing the **CommunityToolkit.Mvvm** package

Select **Install**. When ready, add a **ViewModels** subfolder to the project and, inside this, a new class called **MainPageViewModel.cs**, defined as follows:

```
public class MainPageViewModel : ObservableObject
{
    private WebViewSource source;

    public WebViewSource Source
    {
        get => source;
        set
```

```

    {
        if (!object.Equals(source, value))
        {
            source = value;
            OnPropertyChanged();
        }
    }
}

```

Remember to add a using `CommunityToolkit.Mvvm.ComponentModel`; directive. The `ObservableObject` class already implements property change notification, so it allows for cleaner code. Notice how the `Source` property is of type `WebViewSource`, which is how you assign a value to the `Source` property of your `WebView`.

Defining the startup code

In the `MainPage.xaml.cs` file, add the following fields and change the constructor as follows:

```

private MainPageViewModel _viewModel;
private readonly string baseUrl =
    "https://your-wrapperwebapp-url.azurewebsites.net/";
public MainPageViewModel ViewModel
{ get => _viewModel;
  set { _viewModel = value; OnPropertyChanged(); }
}

public MainPage()
{
    InitializeComponent();

    ViewModel = new MainPageViewModel();

    this.BindingContext = ViewModel;
    ViewModel.Source = new UrlWebViewSource() { Url =
        $"{baseUrl}Identity/Account/Login" };
}

```

This code creates and binds an instance of the `MainPageViewModel` class, setting the startup address for the website and pointing to the login page directly.



Tip: When the user correctly logs in, the web application redirects to the base URL. Detecting this redirection is a simplified way to understand when to display the

navigation bar, which must appear only after successful logins. This is not a reliable approach in real-world applications, but it is enough in this context.

Mobile app recap

If you now build the application and run it on your favorite device, you will see how it displays the website, and you will be able to interact with it. In the next chapter, you will start adding platform-specific code and implementing mobile experience features.

Chapter summary

In this chapter, you have created an ASP.NET Core web application that will serve as an existing website to be wrapped by a .NET MAUI mobile application. You configured the project to use individual user accounts with SQL Server for authentication and data storage, and you explored the detailed implementation of various Razor pages demonstrating common web functionalities such as PDF display, geolocation, user profile management with specific input types, page printing, and file uploads.

You deployed the web application to Azure App Service, including the necessary steps for provisioning resources and applying database migrations post-deployment. In the last section, you created a basic .NET MAUI app capable of displaying the website, which you will develop in the next chapters by adding the necessary mobile experience features.

Chapter 6 Implementing the Client Infrastructure

As you have read multiple times, a mobile wrapper is not just an app that displays a website as it is. This would be rejected by the app stores. Instead, it needs to give value to users by providing features that are typical of a native mobile app. In this chapter, you will set up all the necessary platform-specific code to support feature implementation, so that you will be able to add the necessary mobile experience starting with the next chapter.

Understanding the interaction between websites and .NET

The way you implement mobile features that interact with a website relies on injecting JavaScript code in the website. In .NET MAUI, you can inject JavaScript using the **WebView.EvaluateJavaScriptAsync** method. This method lets you call functions, manipulate the Document Object Model (DOM), or extract values directly from the page. If you want to communicate back from JavaScript to your .NET code, which is often needed to retrieve values from objects in the page, you need to implement a bridge class, which is a C# class that acts as a communication channel between your mobile app and the web content.

On Android, you expose this bridge by implementing a C# class decorated with the **[JavaScriptInterface]** attribute and then invoking the **WebView.AddJavascriptInterface** method. Assuming the bridge class is registered as **jsBridge**, calls from JavaScript typically go through the following syntax: **jsBridge.MethodName()**.

On iOS, the setup differs. You use **WKScriptMessageHandler** to handle JavaScript messages sent via **window.webkit.messageHandlers.functionName.postMessage()**. You register your handler by injecting a custom **WKUserContentController** into the native web view. Unlike Android, iOS does not directly expose C# methods, so you handle messages as events and route them appropriately in your .NET code.

This cross-platform approach lets you build rich interactions between your app's native logic and embedded web content, maintaining smooth communication regardless of platform differences. You will now implement C# bridge classes for both iOS and Android, providing all the necessary customizations.

Important considerations about JavaScript injection

In all the coming chapters, you will implement mobile features that require injecting JavaScript code to the various webpages displayed in the **CustomWebView** control. Especially when you build your own mobile app wrappers, remember that there is no fixed rule or pattern about the scripts you write. In fact, you might try to reuse JavaScript code that you get from this book, and that might not work on different websites, because websites can be built with different development technologies and have different behavior.

For example, in the next chapters you will see how HTML element instances are sometimes retrieved using **XPath** expressions, and other times via **getElementById** functions, and the choice has been made depending on which of the two worked the best way against the selected page.

So, the only general rule is understanding the approach and concepts about interaction between the website and your C# code, and this will make it easier to write the appropriate JavaScript code. As a developer, you can always test your scripts in the JavaScript console of your favorite browser's developer tools, so you can see what the best script is and then bring it to your app code more safely.

Figure 18 shows an example, where you can see the Privacy Policy page and then console from the browser's developer tools, which you enable by pressing F12.

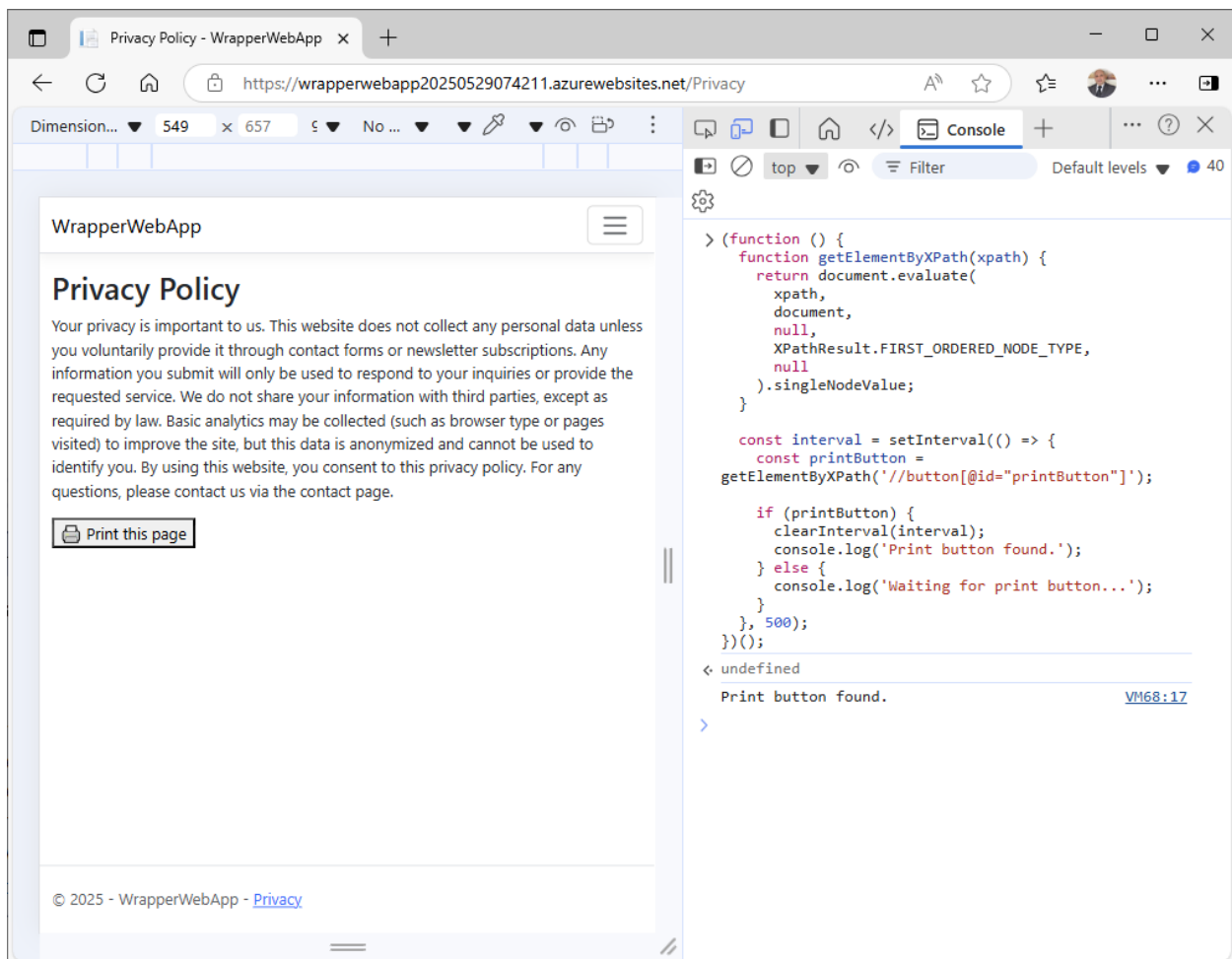


Figure 18: Testing JavaScript code in the developer console

As you can see in Figure 18, the script in the console is used to retrieve the instance of the button on the page. If the script you write does not work as expected, you can test different code and then promote it to your app's scripts.

JavaScript learning resources

As you can imagine, it is not possible to explain how JavaScript works in this book, because it would be completely out of scope. However, Microsoft offers [free educational videos](#) that you can watch to get started and get the minimum knowledge that you need to build mobile web wrappers.

Implementing bridge class and handlers on iOS

On iOS, rich interaction between a mobile app and a website loaded in a WebView can be implemented by extending the native platform handler with a custom class that derives from **WebViewHandler** and that exposes and configures the underlying native **WKWebView**. This native control supports advanced features, including message passing between web content and the mobile app. To enable such communication, the handler constructs a **WKUserContentController**, which manages scripts and message handlers for the web view.

In the **Platforms/iOS** folder of the project, add a new code file called **CustomWebViewHandler.cs** and add the code shown in Code Listing 9.

Code Listing 9

```
using CoreGraphics;
using Foundation;
using Microsoft.Maui.Handlers;
using Microsoft.Maui.Platform;
using MobileAppWrapper.Platforms.iOS;
using MobileAppWrapper.Views;
using Newtonsoft.Json;
using System.Diagnostics;
using UIKit;
using WebKit;
namespace MobileAppWrapper.Platforms.iOS;
public class CustomWebViewHandler : WebViewHandler
{
    const string JavaScriptFunction = "function invokeCSharpAction(data){window.webkit.messageHandlers.invokeAction.postMessage(data);}";
    private WKUserContentController userController;
    private JSBridge jsBridgeHandler;
    static SynchronizationContext sync;

    public CustomWebViewHandler()
    {
        sync = SynchronizationContext.Current;
    }

    protected override WKWebView CreatePlatformView()
    {
        sync = sync ?? SynchronizationContext.Current;
```

```

        jsBridgeHandler = new JSBridge(this);
        userController = new WKUserContentController();

        var script = new WKUserScript(new NSString(JavaScriptFunction), WKU
serScriptInjectionTime.AtDocumentEnd, false);

        userController.AddUserScript(script);
        userController.AddScriptMessageHandler(jsBridgeHandler, "invokeActi
on");

        var config = new WKWebViewConfiguration
        {
            UserContentController = userController,
            MediaTypesRequiringUserActionForPlayback = WKAudiovisualMediaTy
pes.None,
            AllowsInlineMediaPlayback = true,
        };
        config.Preferences.JavaScriptEnabled = true;
        var webView = new WKWebView(CGRect.Empty, config);
        return webView;
    }

    protected override void ConnectHandler(WKWebView platformView)
    {
        if (VirtualView.Source != null)
        {
            LoadSource(((WebViewSource)VirtualView.Source), PlatformView);
        }

        base.ConnectHandler(platformView);
    }

    protected override void DisconnectHandler(WKWebView platformView)
    {
        base.DisconnectHandler(platformView);

        userController.RemoveAllUserScripts();
        userController.RemoveScriptMessageHandler("invokeAction");

        jsBridgeHandler?.Dispose();
        jsBridgeHandler = null;
    }

    public static void LoadSource(WebViewSource source, WKWebView control)
    {
        if (source is HtmlWebViewSource html)
        {
            control.LoadHtmlString(html.Html, new NSURL(html.BaseUrl ?? "ht
tp://localhost", true));
        }
    }

```



```

    }
    else if (source is UrlWebViewSource url)
    {
        control.LoadRequest(new NSUrlRequest(new NSUrl(url.Url)));
    }
}

```

A JavaScript function that by convention we name `invokeCSharpAction(data)` is injected into the webpage through a `WKUserScript`, using the injection time `AtDocumentEnd` so that the function becomes available once the page has loaded. This function uses `window.webkit.messageHandlers.invokeAction.postMessage(data)` to send a message to the native side.

The `CreatePlatformView` method returns a new instance of `WKWebView`, initialized with a custom `WKWebViewConfiguration`. This configuration allows inline media playback and enables JavaScript explicitly. A synchronization context is captured to ensure certain operations are marshaled appropriately to the main thread. Additionally, the handler includes a static helper method `LoadSource`, which interprets either HTML- or URL-based sources and loads them into the view accordingly.

Now you need the actual bridge class. Add a new code file called **JSBridge.cs** to the **Platforms\iOS folder** and write the code shown in Code Listing 10.

Code Listing 10

```

public class JSBridge : NSObject, IWKScriptMessageHandler
{
    private string documentName = string.Empty;
    readonly CustomWebViewHandler _handler;
    WKWebView? webView;

    public JSBridge(CustomWebViewHandler handler)
    {
        _handler = handler;
    }

    public void WebViewNavigationBack()
    {
        var webView = _handler.PlatformView;
        if (webView == null)
            return;

        Foundation.NSUrl url = new NSUrl("");

        WKBackForwardList HistoryList = webView.BackForwardList;
        var lastItem = HistoryList.BackList.Count();
    }
}

```

```

        if (lastItem > 0)
        {
            url = HistoryList.BackList[lastItem - 1].Url;
        }
        else
        {
            url = HistoryList.BackList[0].Url;
        }

        webView.LoadRequest(new Foundation.NSURLRequest(url));

    }
    public void DidReceiveScriptMessage(WKUserContentController userContent
Controller, WKScriptMessage message)
    {
        try
        {
            var body = message.Body.ToString();

            JavascriptData _data = JsonConvert.DeserializeObject<Javascript
Data>(body);

            if (_data.function == "invokeCSharpMSNavigationBack")
            {
                WebViewNavigationBack();
            }
        }
        catch (Exception ex)
        {
            Debug.WriteLine(ex.Message);
        }
    }
}

```

By inheriting from **NSObject** and implementing **IWKScriptMessageHandler**, the **JSBridge** class becomes a fully functional Objective-C-compatible message handler. It can be registered with the **WKUserContentController** and safely invoked from JavaScript running in the embedded webpage. This setup is essential for enabling hybrid app features like navigation, data exchange, and UI updates triggered from web content.

The **DidReceiveScriptMessage** method is triggered whenever a message is posted from JavaScript to the registered handler name, in this case, **invokeAction**. Within this method, JSON-formatted data received from the webpage is parsed and dispatched into native logic, such as triggering navigation or other app-specific actions. For example, a message containing a custom field such as function: **invokeCSharpMSNavigationBack** may be handled by invoking **WebViewNavigationBack**, which examines the **WKBackForwardList** and navigates accordingly. This implementation enables robust bi-directional communication between

embedded web content and native application logic, using platform-native iOS APIs wrapped seamlessly in the .NET MAUI architecture.

Implementing bridge class and handlers on Android

The standard **WebView** control in .NET MAUI provides basic support for rendering and navigation. However, especially on Android, if the web content needs to interact with device hardware, a higher level of control is required. This is because such interactions trigger permission requests from the webpage that must be explicitly handled in the native Android layer.

To manage these requests appropriately, the **WebView** (as well as derived objects) must be configured with a custom implementation of the **WebChromeClient** class, which extends the default behavior of how permissions and media-related operations are handled. A **WebChromeClient** is an Android class responsible for advanced browser-like features inside a **WebView**, including JavaScript dialogs, media playback, and permission requests. By subclassing **WebChromeClient**, you gain control over these interactions, and you can respond to requests for device access initiated by JavaScript in the embedded page.

With this in mind, add a new code file called **CustomWebViewClient.cs** to the **Platforms\Android** folder and add the code shown in Code Listing 11.

Code Listing 11

```
using Android;
using Android.App;
using Android.Content;
using Android.Content.PM;
using Android.OS;
using Android.Webkit;
using AndroidX.Core.App;
using AndroidX.Core.Content;

namespace MobileAppWrapper.Platforms.Android
{
    public class CustomWebViewClient : WebChromeClient
    {
        private readonly Activity _activity;

        public CustomWebViewClient(Activity activity)
        {
            _activity = activity;
        }

        public override void OnPermissionRequest(PermissionRequest? request
        )
        {
            if (Build.VERSION.SdkInt >=
```

```

        BuildVersionCodes.Lollipop && request != null)
    {
        MainThread.BeginInvokeOnMainThread(() =>
        {
            try
            {
                request.Grant(request.GetResources());
            }
            catch (Exception ex)
            {
                Console.WriteLine($"OnPermissionRequest: {ex}");
                request.Deny();
            }
        });
    }
}
}
}
}
}

```

The **CustomWebViewClient** class overrides the **OnPermissionRequest** method, which is called when a webpage requests access to protected device resources. This class examines the requested permissions and checks with the Android system whether those permissions have already been granted to the app. If not, it invokes the **Grant** method to display a prompt to the user and ensure proper access is obtained before allowing the webpage to proceed.

Notice that this is done for Android versions later than 5.1 (Lollipop), which introduced some more restrictions. Without this customization, web-based features that depend on device access may silently fail, creating a poor user experience or restricting functionality. Therefore, implementing a dedicated client like **CustomWebViewClient** is essential when building mobile web wrappers.



Tip: Requesting permissions in the custom *WebChromeClient* is actually a requirement, but you still have to request for user permissions in the shared code. This will be explained in the next chapters.

Defining the bridge class

In Android applications, enabling communication between JavaScript running in a **WebView** and native C# code involves establishing a bridge class. To implement one in the sample project, add a new code file called **JSBridge.cs** to the **Platforms\Android** folder, and then add the code shown in Code Listing 12.

Code Listing 12

```

using Android.Content;
using Android.Print;

```

```

using Android.Webkit;
using Java.Interop;
using Java.IO;
using Microsoft.Maui.Handlers;
using MobileAppWrapper.Views;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.Json;
using System.Threading.Tasks;
using static Android.Media.MediaRouter;

namespace MobileAppWrapper.Platforms.Android
{
    public class JSBridge : Java.Lang.Object
    {
        readonly IWebViewHandler _handler;

        public JSBridge(IWebViewHandler handler)
        {
            _handler = handler;
        }

        [JavascriptInterface]
        [Export("placeholderFunctionName")]
        public void PlaceholderFunctionName(string userValue,
            string pwdValue)
        {
            MainThread.BeginInvokeOnMainThread(() =>
            {
                // Run C# code here...
            });
        }
    }

    public class ValueCallback : Java.Lang.Object,
        global::Android.Webkit.IValueCallback
    {
        private readonly Action<string> _callback;

        public ValueCallback(Action<string> callback)
        {
            _callback = callback;
        }

        public void OnReceiveValue(Java.Lang.Object value)
        {
            _callback?.Invoke(value.ToString());
        }
    }
}

```

```

    }
}

```

This **JSBridge** class extends **Java.Lang.Object**, which is a requirement for interoperability between C# and the Java-based Android runtime. By inheriting from **Java.Lang.Object**, the class becomes eligible for marshaling across the Java Native Interface, allowing JavaScript calls to be routed into C# methods.

The core mechanism for exposing C# methods to JavaScript relies on the use of the **[JavascriptInterface]** attribute. This attribute marks specific methods as callable from JavaScript once the bridge is registered with the **WebView**. Additionally, the **[Export("methodName")]** attribute defines the name by which JavaScript can invoke the method. In this implementation, a method named **PlaceholderFunctionName** is exposed, and at the moment it serves as a placeholder and illustrates the pattern for future methods that will be used to interact with user data or trigger business logic from JavaScript. When invoked, its body executes logic on the main UI thread using **MainThread.BeginInvokeOnMainThread**, ensuring thread-safe interaction with visual components or UI-bound operations.

The bridge class is registered with the **WebView** via the **AddJavascriptInterface** method, assigning it a name such as **jsBridge**. Once registered, JavaScript can call it using the **window.jsBridge.placeholderFunctionName(value1, value2)** syntax, passing parameters from the webpage into the native handler.

Alongside **JSBridge**, the **ValueCallback** class is included to handle asynchronous responses from JavaScript evaluation or other **WebView**-related callbacks. It implements the **IValueCallback** interface from Android's **WebKit** namespace, allowing processed values to be received and passed into an **Action<string>** delegate. This pattern is commonly used when evaluating JavaScript from native code and waiting for a result, such as a computed value or a status flag. Together, these classes form the foundation for hybrid communication in Android within a MAUI context.

Connecting bridge classes to the custom WebView

Now that you have defined the bridge classes and understood their purpose, it is time to connect them to the **CustomWebView** control, fine-tuning the behavior according to each platform. To accomplish this, open the **MauiProgram.cs** file and edit the **MauiProgram** class, as shown in Code Listing 13.

Code Listing 13

```

public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
    }
}

```

```

        .UseMauiApp<App>()
        .ConfigureFonts(fonts =>
        {
            fonts.AddFont("OpenSans-
Regular.ttf", "OpenSansRegular");
            fonts.AddFont("OpenSans-
Semibold.ttf", "OpenSansSemibold");
        }).ConfigureMauiHandlers(handlers =>
        {
#if ANDROID
            Microsoft.Maui.Handlers.WebViewHandler.Mapper.
AppendToMapping(
"WebViewNavigationCustomization",
(handler, view) => {
                if (view is CustomWebView)
                {
                    var platformView = handler.PlatformView;
                    platformView.Settings.SetSupportMultipleWindows
(false);
                    platformView.Settings.JavaScriptCanOpenWindowsA
utomatically = true;
                    platformView.Settings.JavaScriptEnabled = true;
                    platformView.AddJavascriptInterface(
new Platforms.Android.JSBridge(handler), "jsBri
dge");
                    platformView.Settings.SetSupportZoom(true);
                    platformView.Settings.MediaPlaybackRequiresUser
Gesture = false;
                    platformView.Settings.AllowFileAccessFromFileUR
Ls = true;
                    platformView.Settings.AllowUniversalAccessFromF
ileURLs = true;
                    platformView.SetWebChromeClient(new Platforms.A
ndroid.
CustomWebViewClient(Platform.CurrentActivity));
                }
            });
#elif IOS
            handlers.AddHandler<CustomWebView,
MobileAppWrapper.Platforms.iOS.CustomWebViewHandler>();

            Microsoft.Maui.Handlers.WebViewHandler.Mapper.
AppendToMapping("WebViewNavigationCustomization",
(handler, view) =>
            {
                if (view is CustomWebView)
                {

```

```

View;
CustomWebView customWebView = view as CustomWeb
try
{
    handler.PlatformView.NavigationDelegate =
    new CustomWebViewNavigationDelegate(customW
ebView);
}
catch (Exception ex)
{
    Console.WriteLine("Error at WebViewHandler:
"
    + ex.Message);
}
});
#endif
builder.Logging.AddDebug();
#endif
return builder.Build();
}
}

```

This code customizes the behavior of handlers for the **CustomWebView** control. In .NET MAUI, handlers are responsible for mapping cross-platform controls to their native platform counterparts, and they allow for implementing custom behaviors. In this particular case, the **WebViewHandler** is being extended to provide custom functionality for the **CustomWebView** control.

On Android, the mapping extension checks whether the MAUI control is a **CustomWebView**, and if so, accesses the underlying **Android.Webkit.WebView** through the **PlatformView** property of the handler. This native control's **Settings** object is then configured with several fine-grained options: multiwindow support is disabled to keep the browsing experience simple; JavaScript execution and automatic pop-up window behavior are enabled to allow interactive sites to function properly; and zooming and autoplay capabilities are allowed to enhance usability and media experience.

One crucial feature is the injection of a **JSBridge** object into the **WebView**'s JavaScript context using **AddJavascriptInterface**, which enables the webpage to invoke C# methods on the native side. This is critical for applications where the web layer needs to exchange data or trigger native workflows. File access policies are relaxed to accommodate hybrid scenarios where local content might be loaded, and the **CustomWebViewClient** defined previously is attached via **SetWebChromeClient**, allowing the developer to intercept UI-related browser events such as permission requests. This client is instantiated with the current activity context, granting it access to Android lifecycle and UI structures.

On iOS, the registration of a custom handler via **AddHandler** ensures that the **CustomWebView** is processed with specific native rendering logic defined in **CustomWebViewHandler**. In addition, the mapping customization accesses the native **WKWebView** and assigns a new instance of **CustomWebViewNavigationDelegate** to its **NavigationDelegate** property. Code Listing 14 defines such a custom navigation delegate, and you can add the following code to the definition of the **MauiProgram** class.

Code Listing 14

```
#if IOS

    public class CustomWebViewNavigationDelegate : WebKit.WKNavigationDelegate
    {
        private CustomWebView _customWebView;

        public CustomWebViewNavigationDelegate(CustomWebView customWebView)
        {
            _customWebView=customWebView;
        }
        public override async void DidCommitNavigation(WebKit.WKWebView webView,
        WebKit.WKNavigation navigation)
        {
            WebNavigationEvent navigationEvent = new WebNavigationEvent();
            WebViewSource source = _customWebView.Source;

            string url = webView.Url.ToString().ToLower();
            WebNavigationResult result = new WebNavigationResult();

            _customWebView.OnWebNavigating(navigationEvent, source, url);
        }
        public override async void DidFinishNavigation(WebKit.WKWebView webView,
        WebKit.WKNavigation navigation)
        {
            WebNavigationEvent navigationEvent = new WebNavigationEvent();
            WebViewSource source = _customWebView.Source;

            string url = webView.Url.ToString().ToLower();
            WebNavigationResult result = new WebNavigationResult();

            _customWebView.OnWebNavigated(navigationEvent, source, url, result);
        }
    }

#endif
```

```
}
```

This delegate adheres to Apple's `WKNavigationDelegate` protocol, enabling the app to respond to navigation lifecycle events. When the `CustomWebView` begins loading a page, `DidCommitNavigation` is fired, passing the URL and `WebView` source to a handler method called `OnWebNavigating` on the MAUI-side `CustomWebView`.

When the page finishes loading, `DidFinishNavigation` triggers `OnWebNavigated`, completing the navigation cycle and potentially allowing UI updates, logging, or cancellation.



Tip: *Though the `WebView` (and derived classes such as our `CustomWebView`) can fire and handle events such as `Navigating` and `Navigated`, when you customize the iOS handler, you must also manually fire (and handle in the cross-platform code) the `WebNavigating` and `WebNavigated` events from the native `WKWebView`; otherwise, the navigation events will be ignored.*

This design abstracts native navigation events into cross-platform callbacks, allowing you to hook into them without writing platform-specific logic throughout their application.

Extending the custom `WebView` to address iOS requirements

In the previous section, you saw that the custom navigation handler for iOS explicitly fires two events, `OnWebNavigating` and `OnWebNavigated`. These must both be handled by the `CustomWebView` class, so that when they are triggered by the native control, the cross-platform code is able to intercept them. Code Listing 15 demonstrates this.

Code Listing 15

```
namespace MobileAppWrapper.Views
{
    public class CustomWebView : WebView
    {
        public event EventHandler<WebNavigatedEventArgs>
            WebNavigated;
        public event EventHandler<WebNavigatingEventArgs>
            WebNavigating;

        public void OnWebNavigated(WebNavigationEvent navigationEvent,
            WebViewSource source, string url, WebNavigationResult result)
        {
            WebNavigated?.Invoke(this,
                new WebNavigatedEventArgs(navigationEvent,
                    source, url, result));
        }
        public void OnWebNavigating(WebNavigationEvent navigationEvent,
            WebViewSource source, string url)
        {

```

```

        WebNavigating?.Invoke(this,
            new WebNavigatingEventArgs(navigationEvent,
                source, url));
    }
}

```

This code implements two new client-side events that map the native iOS navigation events and redirects them to the user interface. You will see how to implement control over navigation events very shortly.

Completing the client infrastructure

The final step in the client infrastructure is to extend the main page with navigation event handlers. To accomplish this, in the **MainPage.xaml** file, extend the **CustomWebView** definition as follows:

```

<views:CustomWebView BindingContext="{Binding}"
    Source="{Binding Source}"
    x:Name="RootWebView"
    WebNavigated="RootWebView_Navigated"
    WebNavigating="RootWebView_Navigating"
    Navigated="RootWebView_Navigated"
    Navigating="RootWebView_Navigating"/>

```

This allows for intercepting both the client navigation events and the native iOS navigation events. In the code-behind files, add the following event handler definitions:

```

private async void RootWebView_Navigating(object sender,
    WebNavigatingEventArgs e)
{
    // Add some transparency while the spinner is shown
    RootWebView.Opacity = 0.3;
    SpinnerGrid.IsVisible = true;
}
private async void RootWebView_Navigated(object sender,
    WebNavigatedEventArgs e)
{
    RootWebView.Opacity = 1; // full page visibility
    SpinnerGrid.IsVisible = false;
}

```

For now, both event handlers manage the visibility of the **ActivityIndicator** (and its parent **Grid**) depending on the navigation status. In the next chapters, you will extend them according to the feature implementation.

You have now completed the implementation of all the necessary client-code, and you are ready to start working on mobile features. Running the application now would not be of great value, because it would just display the target website and nothing else—so it will be left for later (though you are free to try if you are curious).

Chapter summary

This chapter explained how to connect native platform **WebView** controls to .NET MAUI using bridge classes, which allow for interaction between JavaScript and C#, and handlers that customize the behavior of native **WebView** controls to support the addition of mobile features. You have completed the infrastructure code for the client side in the cross-platform code. Now you have a fully functional wrapper ready to be extended with the mobile experience features. This starts with the next chapter.

Chapter 7 Implementing a Navigation Bar

When wrapping a website into a mobile application, the goal is not merely to replicate the web experience, because this would not even be accepted by the stores. The goal is to elevate the experience. One of the most common ways to achieve this is by incorporating a navigation bar built with .NET MAUI components. By implementing a navigation bar, you simplify the user experience by providing shortcuts to relevant pages of the website.

This chapter explains how to implement a navigation bar in .NET MAUI, and how to take advantage of it for improved interaction between the target website and the mobile app.

Defining the user interface

The navigation bar for the sample application will include shortcuts to the Home, Brochure, Geolocation, and My Profile pages of the website. You will need a .png icon for each shortcut, each representing the target page in a meaningful way.

 **Tip:** For simplicity, you can use the .png icons included in the companion solution. They are named *home.png*, *document.png*, *profile.png*, and *location.png*, and they are located under the *Resources\Images* folder of the project. When you add image files, do not forget to set the *Build Action* property as *MauiImage*.

Having said that, open the **MainPage.xaml** file and extend its code as shown in Code Listing 16, focusing on the **Grid** with **x:Name="NavigationBar"** that is being added under the **ActivityIndicator** declaration.

Code Listing 16

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:views="clr-namespace:MobileAppWrapper.Views"
    x:Class="MobileAppWrapper.MainPage">

    <Grid>
        <Grid x:Name="LayoutRoot">
            <Grid.RowDefinitions>
                <RowDefinition />
                <RowDefinition Height="40"/>
            </Grid.RowDefinitions>
            <views:CustomWebView BindingContext="{Binding}"
                Source="{Binding Source}"
                x:Name="RootWebView"
                WebNavigated="RootWebView_Navigated"
                WebNavigating="RootWebView_Navigating">
```

```

        Navigated="RootWebView_Navigated"
        Navigating="RootWebView_Navigating"/>
<ActivityIndicator Color="Blue" x:Name="SpinnerGrid"
    IsVisible="False" IsRunning="True"
    WidthRequest="60" HeightRequest="60"/>

<Grid x:Name="NavigationBar" Grid.Row="1" IsVisible="False"
    BackgroundColor="White">
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>

    <!-- Home -->
    <VerticalStackLayout Grid.Column="0" Spacing="2"
        HorizontalOptions="Center">
        <Image Source="home.png" HeightRequest="24"
            WidthRequest="24"/>
        <Label Text="Home" FontSize="12"
            HorizontalTextAlignment="Center"/>
        <VerticalStackLayout.GestureRecognizers>
            <TapGestureRecognizer Tapped="OnHomeTapped"/>
        </VerticalStackLayout.GestureRecognizers>
    </VerticalStackLayout>

    <!-- Brochure -->
    <VerticalStackLayout Grid.Column="1" Spacing="2"
        HorizontalOptions="Center">
        <Image Source="document.png" HeightRequest="24"
            WidthRequest="24"/>
        <Label Text="Brochure" FontSize="12"
            HorizontalTextAlignment="Center"/>
        <VerticalStackLayout.GestureRecognizers>
            <TapGestureRecognizer Tapped="OnBrochureTapped"/>
        </VerticalStackLayout.GestureRecognizers>
    </VerticalStackLayout>

    <!-- My Profile -->
    <VerticalStackLayout Grid.Column="2" Spacing="2"
        HorizontalOptions="Center">
        <Image Source="profile.png" HeightRequest="24"
            WidthRequest="24"/>
        <Label Text="My Profile" FontSize="12"
            HorizontalTextAlignment="Center"/>
        <VerticalStackLayout.GestureRecognizers>
            <TapGestureRecognizer Tapped="OnMyProfileTapped"/>
        </VerticalStackLayout.GestureRecognizers>
    </VerticalStackLayout>
    </Grid>

```

```

        </VerticalStackLayout>

        <!-- Geolocation -->
        <VerticalStackLayout Grid.Column="3" Spacing="2"
            HorizontalOptions="Center">
            <Image Source="location.png" HeightRequest="24"
                WidthRequest="24"/>
            <Label Text="Location" FontSize="12"
                HorizontalTextAlignment="Center"/>
            <VerticalStackLayout.GestureRecognizers>
                <TapGestureRecognizer Tapped="OnGeolocationTapped"/>
            </VerticalStackLayout.GestureRecognizers>
        </VerticalStackLayout>
    </Grid>
</Grid>

</Grid>
</ContentPage>

```

The navigation bar is anchored at the bottom of the screen by assigning it to **Grid.Row="1"** within the layout structure. Its visibility is set as **false** for startup, and will be changed to **true** at runtime once the user has logged in. Each navigation item is built using a vertically stacked layout that combines an **Image** and a **Label** within a **VerticalStackLayout**. These elements are centered and spaced evenly to maintain a clean, intuitive design. To handle user actions, each button includes a **TapGestureRecognizer**, which triggers a corresponding event handler in the code-behind. Event handlers are defined as follows:

```

private void OnHomeTapped(object sender, TappedEventArgs e)
{
    RootWebView.Source = baseUrl;

#if IOS
    ((WebKit.WKWebView)RootWebView.Handler.PlatformView).LoadRequest(new
    Foundation.NSURLRequest(new Foundation.NSUrl(baseUrl)));
#endif
}

private void OnBrochureTapped(object sender, TappedEventArgs e)
{
    string url = $"{baseUrl}brochure";
    RootWebView.Source = url;

#if IOS
    ((WebKit.WKWebView)RootWebView.Handler.PlatformView).LoadRequest(new
    Foundation.NSURLRequest(new Foundation.NSUrl(url)));
#endif
}

private void OnMyProfileTapped(object sender, TappedEventArgs e)

```

```

        {
            string url = $"{baseUrl}myprofile";
            RootWebView.Source = url;
#if IOS
((WebKit.WKWebView)RootWebView.Handler.PlatformView).LoadRequest(new
Foundation.NSURLRequest(new Foundation.NSUrl(url)));
#endif
        }

```

The event handlers update the **Source** property of the **CustomWebView** to redirect to the desired page of the website. However, on iOS, the native **WKWebView** that powers the **WebView** needs to explicitly fire the **WebNavigating** and **WebNavigated** events that you defined in the previous chapter, and this is accomplished by invoking the **LoadRequest** method pointing to the page URL.

Controlling the navigation bar visibility

Most of the pages of the target website are available only to authenticated users, so it makes sense to display them only after the user has logged in. In real-world scenarios, you will need to implement robust logic to understand if the user has successfully logged in (for example, handling responses from the website), but for demonstration purposes in the sample application it is sufficient to determine the current URL. In fact, the application loads the login page at startup, and this has a specific URL (see the constructor of the main page), but when the user has logged in, the target URL is different.

To control the visibility, edit the **RootWebView_Navigated** event handler as follows:

```

private async void RootWebView_Navigated(object sender,
    WebNavigatedEventArgs e)
{
    RootWebView.Opacity = 1; // full page visibility
    SpinnerGrid.IsVisible = false;
    if (e.Url.ToLower() ==
        "https://your-domain-name.azurewebsites.net/")
    {
        // Navbar visible
        NavigationBar.IsVisible = true;
    }
}

```

Obviously, replace **your-domain-name** with the domain name of your target website. For now, you have everything you need to test a first version of the mobile app wrapper with the basic mobile experience. In the next chapters, you will extend the implementation with additional features.

Running the application

After much work, you are finally ready to run the sample application. Choose a target device of choice and press F5. At startup, the application will show the login page of the external website, as shown in Figure 19 (figures representing the app are based on both Android and iOS).

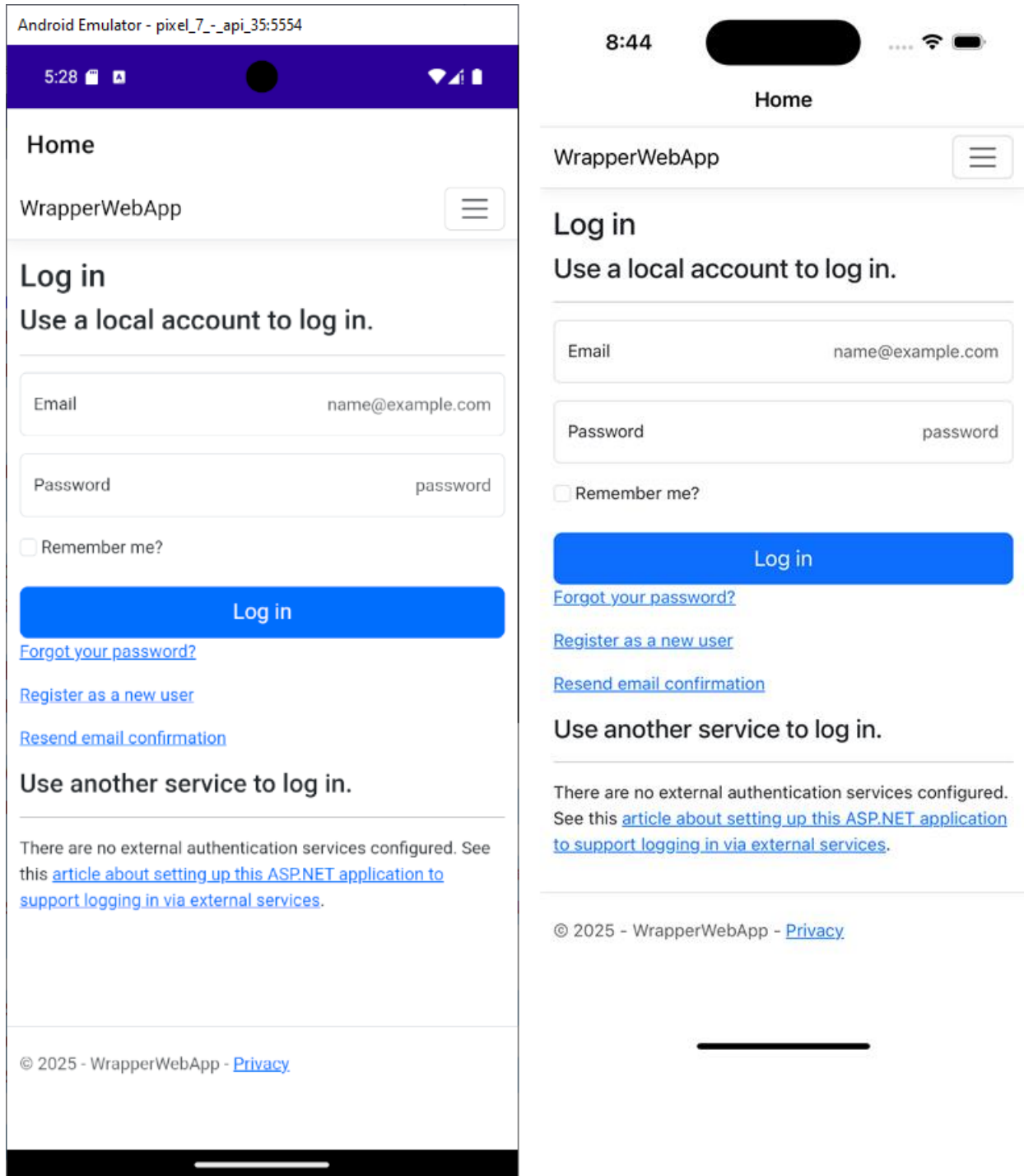


Figure 19: The application showing the login page of the external website

Tip: Though unlikely, it might happen that the webpage displays an error from SQL, saying that the database is not accessible. This can happen on development cloud configurations. If you get this message, simply restart the application. The database service will “wake up” automatically.

If you have not already registered, this is the moment to do it. Remember that the registration you made while debugging the web application locally does not work once it has been deployed to Azure. Take Figure 8 as a reference for a new registration. Once registered, you will be automatically logged into the website from the mobile app, and the navigation bar will appear (see Figure 20).

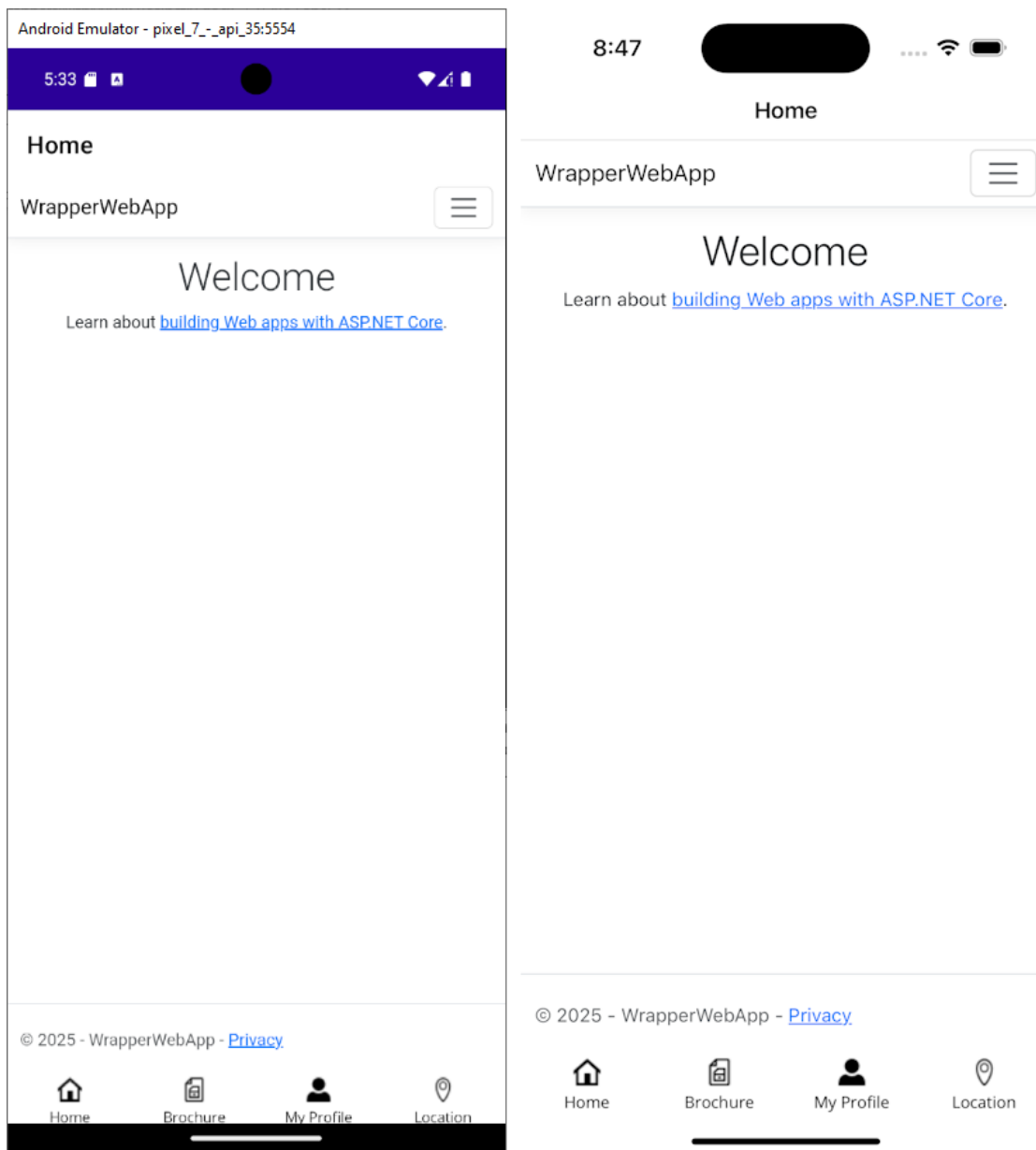


Figure 20: The navigation bar appears after successful login

You can click the navigation bar buttons to explore the linked pages on the website, and you can browse the website by using the hyperlinks in the webpages. You can now understand why a navigation bar is an important addition to the user experience when wrapping websites into a mobile app: it makes it more device-friendly to browse the target content.

Chapter summary

Adding a navigation bar to your mobile wrapper not only addresses one of the requirements from Google Play and the Apple App Store, but it also improves the usability of the app. It enables quick access to pages in the external website without relying on in-page links, aligning the user interface with mobile app standards.

In this chapter, you have implemented a navigation bar using .NET MAUI components, providing additional logic that is required for iOS by using the native `WKWebView.LoadRequest` method to ensure that navigation works reliably—especially when reloading identical URLs.

In the next chapter, you will implement a key feature from the mobile experience point of view: biometric authentication.

Chapter 8 Implementing Biometric Authentication

This chapter focuses on integrating native biometric capabilities (such as fingerprint or face recognition) into your .NET MAUI wrapper applications. This integration will allow users to log into your wrapped websites more securely and seamlessly, bypassing the need to manually enter credentials each time, and therefore offering an appropriate mobile experience and addressing another Google and Apple requirement for wrapped websites.

Installing NuGet packages

.NET MAUI does not include cross-platform APIs to interact with the biometric sensors of the various platforms. Luckily, you can install the [Oscore.Maui.Biometrics](#) NuGet package that ships for free. Use the NuGet Package Manager user interface to install the latest version of the library to the project and get ready to improve the user experience.

In the **MainPage.xaml** file, the only required change is to add the following event handler declaration to the **CustomWebView** control:

```
TextBoxValuesRetrieved="RootWebView_TextBoxValuesRetrieved"
```

This will be clarified shortly. Basically, this event is fired when the user manually enters credentials into the webpage for login. The user interface of the main page does not need any changes, and everything will be implemented in the C# logic.

Extending the CustomWebView control

The previously defined **CustomWebView** control must be extended with code that enables specific communication patterns required for biometric authentication, defining events that can be triggered by platform-specific JavaScript bridges. Code Listing 17 demonstrates this.

Code Listing 17

```
using Maui.Biometric;
using MobileAppWrapper.Model;

namespace MobileAppWrapper.Views
{
    public class CustomWebView : WebView
    {
        // Event triggered when username and password values
        // are retrieved from the webpage.
        public event EventHandler<TextBoxValuesEventArgs>
```

```

        TextBoxValuesRetrieved;
    public event EventHandler<WebNavigatedEventArgs>
        WebNavigated;
    public event EventHandler<WebNavigatingEventArgs>
        WebNavigating;

    public void OnTextBoxValuesRetrieved(string userValue,
        string pwdValue)
    {
        TextBoxValuesRetrieved?.Invoke(this,
            new TextBoxValuesEventArgs(userValue, pwdValue));
    }

    public void OnWebNavigated(WebNavigationEvent navigationEvent,
        WebViewSource source, string url, WebNavigationResult result)
    {
        WebNavigated?.Invoke(this,
            new WebNavigatedEventArgs(navigationEvent,
                source, url, result));
    }
    public void OnWebNavigating(WebNavigationEvent navigationEvent,
        WebViewSource source, string url)
    {
        WebNavigating?.Invoke(this,
            new WebNavigatingEventArgs(navigationEvent,
                source, url));
    }

    public async Task<bool> AuthenticateUserAsync()
    {
        var result = await BiometricAuthentication.Current.
            AuthenticateAsync(
                new AuthenticationRequest(
                    title: "Biometric authentication",
                    reason: "Login with biometric authentication"));
        return result.IsSuccessful;
    }
}

```

Here is an explanation of the relevant pieces:

- **TextBoxValuesRetrieved:** This custom event **EventHandler** is central to the biometric flow. It is raised by platform-specific code (via JavaScript bridges) when the username and password values are successfully extracted from the web login form.
- **OnTextBoxValuesRetrieved:** This method is called by the platform-specific **WebView** handlers to raise the **TextBoxValuesRetrieved** event. It passes the captured credentials to the **MainPage.xaml.cs** via an object of type **TextBoxValuesEventArgs**, defined shortly.

- **AuthenticateUserAsync**: This method encapsulates the call to **BiometricAuthentication.Current.AuthenticateAsync** from the **Oscore.Maui.Biometrics** library. It presents a biometric prompt to the user and returns **true** if authentication is successful, indicating that the user has verified their identity via biometric sensors.

The **TextBoxValuesEventArgs** class, which stores the user's name and password values entered via the website text boxes, is defined as follows:

```
public class TextBoxValuesEventArgs : EventArgs
{
    public string UsernameValue { get; }
    public string PwdValue { get; }

    public TextBoxValuesEventArgs(string userValue, string pwdValue)
    {
        UsernameValue = userValue;
        PwdValue = pwdValue;
    }
}
```

The next step is handling the event in the main page logic.

Handling authentication events

The **MainPage.xaml.cs** code-behind file manages the biometric authentication flow. It handles the **TextBoxValuesRetrieved** event from the **CustomWebView**, leverages methods from the **SecureStorage** class for secure credentials management, and initiates biometric prompts. This is a summary of the flow:

1. Users manually enter their credentials the first time they log in.
2. Some JavaScript code detects that the user has selected **Log In** and retrieves the content of the username and password text boxes.
3. Retrieved credentials are sent from the bridge class to the cross-platform code via the **OnTextBoxValuesRetrieved** method.
4. In the cross-platform code, the **CustomWebView** intercepts the **TextBoxValuesRetrieved** event and stores the credentials in the device's secure storage for later reuse.
5. The next time the user opens the application, the code will search for stored credentials in the secure storage and will launch the system biometric authentication prompt. If this succeeds, the application will invoke some JavaScript code to fill the username and password text boxes with the stored credentials and to programmatically click the **Log In** button.

Now that you have a clear understanding of the authentication flow, you can add the necessary logic.



Note: *In order to be consistent with the flow just described, you will now implement methods in the main page logic, though these methods invoke other*

methods that will be implemented later. If something seems confusing, be patient—everything will be clarified soon.

Defining methods for the biometric flow

The cross-platform code of the app, defined in the MainPage.xaml.cs file, handles the flow of the biometric authentication. More specifically, it defines three methods:

IsBiometricLoginEnabledAsync, which detects if the user has accepted enabling biometric authentication; **TryEnableBiometricLoginAsync**, which prompts the user with the system authentication options if biometric sensors are available and enabled on the device; and **RootWebView_TextBoxValuesRetrieved**, an event handler that receives credentials from the platform-specific code.

Code Listing 18 contains the full code for the page with the current updates, and explanations will follow.

Code Listing 18

```
using Maui.Biometric;
using MobileAppWrapper.Model;
using MobileAppWrapper.Services;
using MobileAppWrapper.ViewModels;
using System.Threading.Tasks;

namespace MobileAppWrapper
{
    public partial class MainPage : ContentPage
    {
        private string userName;
        private string pwd;
        private MainPageViewModel _viewModel;

        private readonly string baseUrl =
            "https://your-azure-domain.azurewebsites.net/";
        public MainPageViewModel ViewModel
        {
            get => _viewModel;
            set { _viewModel = value;
                OnPropertyChanged(); }
        }

        public MainPage()
        {
            InitializeComponent();
            ViewModel = new MainPageViewModel();

            this.BindingContext = ViewModel;
            ViewModel.Source =
                new UrlWebViewSource()
```

```

        { Url = $"{baseUrl}Identity/Account/Login" };
    }

    private async void RootWebView_TextBoxValuesRetrieved(object sender
    ,
        Model.TextBoxValuesEventArgs e)
    {
        userName = e.UsernameValue;
        pwd = e.PwdValue;
        if (userName != null && pwd != null)
        {
            await SecureStorage.SetAsync("username", userName);
            await SecureStorage.SetAsync("pwd", pwd);
            await TryEnableBiometricLoginAsync();
        }
    }

    public async Task<bool> IsBiometricLoginEnabledAsync()
    {
        var biometricEnabled =
            await SecureStorage.Default.GetAsync("biometric_enabled");
        try
        {
            bool result = bool.Parse(biometricEnabled);
            return result;
        }
        catch (Exception)
        {
            return false;
        }
    }

    public async Task<bool> TryEnableBiometricLoginAsync()
    {
        var availability =
            await BiometricAuthentication.Current.
                CheckAvailabilityAsync();

        if (!availability.IsAvailable)
            return false;

        var authResult =
            await BiometricAuthentication.Current.AuthenticateAsync(
                new AuthenticationRequest("Enable biometric login",
                    "Use biometrics to log in faster"));

        if (authResult.IsSuccessful)
        {

```



```

        await SecureStorage.Default.SetAsync("biometric_enabled",
            "true");
        return true;
    }
    else
    {
        await SecureStorage.Default.SetAsync("biometric_enabled",
            "false");
        return false;
    }
}

private async void RootWebView_Navigated(object sender,
    WebNavigatedEventArgs e)
{
    RootWebView.Opacity = 1; // full page visibility
    SpinnerGrid.IsVisible = false;

    if (e.Url.ToLower().Contains("/identity/account/login"))
    {
        var checkBiometricEnabled = await
            IsBiometricLoginEnabledAsync();
        if (!checkBiometricEnabled)
            await RootWebView.
                InjectCredentialsManagementToLoginPageAsync();
        else
        {
            await RootWebView.TryAutoLoginWithBiometricsAsync();
        }
    }
    else if (e.Url.ToLower() ==
        "https://your-azure-domain.azurewebsites.net/")
    {
        // Navbar visible
        NavigationBar.IsVisible = true;
    }
}

private async void RootWebView_Navigating(object sender,
    WebNavigatingEventArgs e)
{
    RootWebView.Opacity = 0.3; // Add some transparency while the s
    pinner is shown
    SpinnerGrid.IsVisible = true;
}

private void OnHomeTapped(object sender, TappedEventArgs e)
{
    RootWebView.Source = baseUrl;
}

```

```

#if IOS
((WebKit.WKWebView)RootWebView.Handler.PlatformV
iew).LoadRequest(new Foundation.NSURLRequest(new Foundation.NSUrl(baseUrl))
);
#endif
}

private void OnBrochureTapped(object sender, TappedEventArgs e)
{
    string url = $"{baseUrl}brochure";
    RootWebView.Source = url;
}

#if IOS
((WebKit.WKWebView)RootWebView.Handler.PlatformV
iew).LoadRequest(new Foundation.NSURLRequest(new Foundation.NSUrl(url)));
#endif
}

private void OnMyProfileTapped(object sender, TappedEventArgs e)
{
    string url = $"{baseUrl}myprofile";
    RootWebView.Source = url;
}

#if IOS
((WebKit.WKWebView)RootWebView.Handler.PlatformV
iew).LoadRequest(new Foundation.NSURLRequest(new Foundation.NSUrl(url)));
#endif
}

private void OnGeolocationTapped(object sender, TappedEventArgs e)
{
    string url = $"{baseUrl}geolocation";
}

#if IOS
((WebKit.WKWebView)RootWebView.Handler.PlatformV
iew).LoadRequest(new Foundation.NSURLRequest(new Foundation.NSUrl(url)));
#endif
}

}
}

```

Here is an explanation of the new methods and relevant objects:

- **RootWebView_TextBoxValuesRetrieved:** This event handler is triggered by the **CustomWebView** when JavaScript extracts the username and password from the web login form. It securely stores these credentials using **SecureStorage.SetAsync** and then calls **TryEnableBiometricLoginAsync** to prompt the user to enable biometric login for subsequent sessions.
- **IsBiometricLoginEnabledAsync:** This method checks the secure storage to determine if the user has previously enabled the biometric login feature.

- **TryEnableBiometricLoginAsync:** This method uses the `Oscore.Maui.Biometrics` library and its `BiometricAuthentication.Current` class to verify if biometric authentication is available on the device (**CheckAvailabilityAsync**). If available, it presents a biometric prompt to the user (**AuthenticateAsync**) with a descriptive title and reason. Based on the authentication result, it updates a flag in the secure storage (**biometric_enabled**) to remember the user's preference for future logins.

The next step is implementing code that injects JavaScript into the webpage to retrieve the content of the username and password text boxes.

Extending functionalities with extension methods

In order to keep the code of the `MainPage.xaml.cs` file clean, and to encapsulate complex JavaScript injection logic, it is a good idea to create extension methods to extend the `CustomWebView` class, allowing you to call them directly on `RootWebView` instance.

Add a new code file called **ExtensionMethods.cs** to the **Model** folder of the project and then add the content of Code Listing 19.

Code Listing 19

```
using Maui.Biometric;
using MobileAppWrapper.Views;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using static MobileAppWrapper.Scripts.AndroidScripts;
using static MobileAppWrapper.Scripts.IOSScripts;

namespace MobileAppWrapper.Model
{
    public static class ExtensionMethods
    {
        public static async Task
            InjectCredentialsManagementToLoginPageAsync(
                this CustomWebView webView)
        {
            #if ANDROID
                MainThread.BeginInvokeOnMainThread(async () =>
                {
                    await webView.
                        EvaluateJavaScriptAsync(AndroidGetCredentialsScript);
                });
            #elif IOS
                await webView.EvaluateJavaScriptAsync(iOSGetCredentialsScript);
            #endif
        }
    }
}
```

```

        public async static Task
            TryAutoLoginWithBiometricsAsync(
                this CustomWebView webView)
        {
            var biometricEnabled =
                await SecureStorage.Default.GetAsync("biometric_enabled");
            if (biometricEnabled != "true")
                return; // Biometric login not enabled

            var authResult = await BiometricAuthentication.Current.
                AuthenticateAsync(new AuthenticationRequest(
                    "Login with biometrics", "Biometric login"));

            if (!authResult.IsSuccessful)
                return; // User canceled or failed biometric auth

            var email = await SecureStorage.Default.GetAsync("username");
            var password = await SecureStorage.Default.GetAsync("pwd");

            if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(password))
                return; // No credentials saved

            // Inject JS to fill and submit form
            string script = $"
(function () {{
    function getElementByXPath(xpath) {{
        return document.evaluate(xpath, document, null, XPathResult.FIRST_ORDERED_NODE_TYPE, null).singleNodeValue;
    }}
    const interval = setInterval(() => {{
        const emailInput = getElementByXPath('//input[@id=""Input_Email""]');
        const passwordInput = getElementByXPath('//input[@id=""Input_Password""]');
        const loginButton = getElementByXPath('//button[@id=""login-submit""]');

        if (emailInput && passwordInput && loginButton) {{
            clearInterval(interval);
            emailInput.value = '{email}';
            passwordInput.value = '{password}';
            loginButton.click();
        }}
    }}, 500);
}})();";

            await webView.EvaluateJavaScriptAsync(script);
        }
    }
}

```

The **InjectCredentialsManagementToLoginPageAsync** extension method injects platform-specific JavaScript into the **CustomWebView**. This JavaScript snippet is designed to automate the login process on the webpage by repeatedly checking for the presence of specific input fields and a Log In button, then populating them with credentials and triggering a click. It's wrapped in an *immediately invoked function expression*, meaning it runs as soon as it's defined.

Inside the function, there is a helper function called **getElementByXPath**, which uses the **document.evaluate** method to locate elements on the page using **XPath** expressions. This is useful when elements cannot be easily targeted with standard selectors like **getElementById** or **querySelector**.

The core logic runs inside a **setInterval** loop that executes every 500 milliseconds. Each time it runs, it tries to find the email input box, password input box, and Log In button using their respective **XPath** queries. If all three elements are found, the interval is cleared to stop further execution. Then, it sets the value of the email and password fields to the provided credentials (the interpolated **{email}** and **{password}** strings). Finally, it simulates a click on the Log in button to submit the form. This kind of script is often used in browser automation or testing scenarios, but it should be used responsibly—especially when handling sensitive data like passwords.

When the user clicks this button, the JavaScript captures the entered username and password and sends them back to the C# code. The **TryAutoLoginWithBiometricsAsync** method implements the core autologin logic as follows:

1. It first verifies if biometric login is enabled (via secure storage).
2. If enabled, it initiates a biometric authentication prompt using **BiometricAuthentication.Current.AuthenticateAsync**.
3. If biometric authentication is successful, it retrieves the previously stored username and password from the secure storage.
4. Finally, it constructs and executes a JavaScript string. This script programmatically finds the login form's email, password, and Submit button elements, fills the input fields with the stored credentials, and simulates a click on the Log in button, effectively logging the user in without manual input. A **setInterval** loop is used to ensure the web elements are loaded before interaction.

The **WebView** and its subclasses, including the **CustomWebView** of the current project, expose the **EvaluateJavaScriptAsync** method, which allows for injecting and executing JavaScript code into the currently displayed webpage. As you have seen, there are different scripts to execute depending on the current OS, and in the next section, you will define them appropriately.



Note: As you can imagine, only an explanation of the relevant and specific JavaScript code can be provided here. Explaining JavaScript as a language would be out of scope here, since the main topic is .NET MAUI.

Notice how:

- Different scripts are executed depending on the target platform. These will be implemented shortly.

- On Android, the code ensures that the JavaScript injection happens on the main UI thread. On iOS, this is not necessary due to the different way of managing threads.

Now you have to define the actual scripts to inject.

Injecting JavaScript code

To enable the native .NET MAUI app to interact with the web login form, you need to inject JavaScript that can read the values from the input fields and trigger a communication back to C#. These scripts are platform-specific due to differences in how JavaScript bridges are handled.

First of all, add a new folder called **Scripts** to the project, and then add two new C# files called **AndroidScripts.cs** and **IOSScripts.cs** to the new folder. Then, you will go to each platform separately.

Android implementation

Let's start with Android, and add the code shown in Code Listing 20 to the new file.

Code Listing 20

```
namespace MobileAppWrapper.Scripts
{
    public static class AndroidScripts
    {
        public static string AndroidGetCredentialsScript = @"(function(){
function getElementByXPath(xpath){
    return document.evaluate(xpath,document,null,XPathResult.FIRST_ORDE
RED_NODE_TYPE,null).singleNodeValue;
}
const interval=setInterval(=>{
    const emailInput=getElementByXPath('//input[@id=\"\"Input_Email\"\"]'
);
    const passwordInput=getElementByXPath('//input[@id=\"\"Input_Passwor
d\"\"]');
    const loginButton=getElementByXPath('//button[@id=\"\"login-
submit\"\"]');
    if(emailInput&&passwordInput&&loginButton){
        clearInterval(interval);
        loginButton.addEventListener('click',(e)=>{
            jsBridge.retrieveCredentials(emailInput.value,passwordInput
.value);
        });
        console.log('Login button handler attached!');
    }else{
        console.log('Waiting for login elements...');
    }
},500);
```

```

    })();";
  }
}

```

The **AndroidGetCredentialsScript** variable contains the JavaScript code specifically tailored for Android. It uses **getElementByXPath** to locate the email input (**Input_Email**), password input (**Input_Password**), and Log in button (**login-submit**) elements on the webpage. A **setInterval** is employed to repeatedly check for the presence of these elements, ensuring the script interacts with a fully loaded form. Once found, an event listener is attached to the **loginButton**. When this button is clicked, the JavaScript calls **jsBridge.retrieveCredentials**. The **jsBridge** identifier represents the name of the C# bridge class defined previously, which you need to extend as follows:

```

[JavascriptInterface] // Exposes this method to JavaScript
[Export("retrieveCredentials")] // Specifies the JavaScript name for this
method
public void RetrieveCredentials(string userValue, string pwdValue)
{
    MainThread.BeginInvokeOnMainThread(() =>
    {
        // Cast VirtualView to CustomWebView and invoke its custom event.
        // This passes the captured credentials to the CustomWebView.

        ((CustomWebView)_handler.VirtualView)?.OnTextBoxValuesRetrieved(userValue,
            pwdValue);
    });
}

```

This method can be explained as follows:

- The **[JavascriptInterface]** attribute marks the **RetrieveCredentials** method as callable from JavaScript within the Android **WebView**.
- The **[Export("retrieveCredentials")]** attribute specifies the name by which the JavaScript code will call this method (for example, **jsBridge.retrieveCredentials**).
- The **RetrieveCredentials** method receives the username and password strings directly from JavaScript, which were captured by the injected **AndroidGetCredentialsScript**.
- The invocation to **MainThread.BeginInvokeOnMainThread** is crucial to dispatch UI-related operations (like raising an event that affects UI) back to the main UI thread.
- The invocation to **((CustomWebView)_handler.VirtualView)?.OnTextBoxValuesRetrieved** allows for accessing the **CustomWebView** instance associated with the **WebViewHandler** (represented by **VirtualView**) and calls its **OnTextBoxValuesRetrieved** method, effectively passing the captured credentials to the main application logic.

When the **RetrieveCredentials** method completes its flow, the control is returned to the instance of the **CustomWebView** in the .NET MAUI code that you saw in Code Listing 20.

iOS implementation

For iOS, the content of the script file will be the following, shown in Code Listing 21.



Note: Though not an explicit requirement, iOS scripts should always be on a single line. The reason is that the parsing engine of the native `WKWebView` might fail when parsing indented, multiline scripts. For the sake of clarity and readability, scripts for iOS will be provided as indented listings, but make sure you have them on a single line. If you find this confusing, you can use [Microsoft Copilot](#), pasting the JS code and asking the AI service to put it on a single line for you.

Code Listing 21

```
namespace MobileAppWrapper.Scripts
{
    public static class IOSScripts
    {
        public static string iOSGetCredentialsScript = @"(function () {
function getElementByXPath(xpath) {
    return document.evaluate(xpath, document, null, XPathResult.FIRST_0
RDERED_NODE_TYPE, null).singleNodeValue;
}
const interval = setInterval(() => {
    const emailInput = getElementByXPath("//input[@id='Input_Email']"
);
    const passwordInput = getElementByXPath("//input[@id='Input_Passwo
rd']");
    const loginButton = getElementByXPath("//button[@id='login-
submit']");
    if (emailInput && passwordInput && loginButton && !loginButton._han
dlerAttached) {
        clearInterval(interval);
        loginButton._handlerAttached = true;
        loginButton.addEventListener('click', function (e) {
            const currentUsernameValue = emailInput.value;
            const currentPwdValue = passwordInput.value;
            var formData = {
                function: 'retrieveCredentials',
                data: currentUsernameValue + ',' + currentPwdValue
            };
            invokeCSharpAction(JSON.stringify(formData));
        });
        console.log('Login button handler attached!');
    } else {
        console.log('Waiting for login elements...');
    }
}, 500);
})();";
}
```



```
}
```

The **iOSGetCredentialsScript** variable contains the JavaScript code specifically for iOS. Similar to the Android script, it uses **getElementByXPath** and a **setInterval** to locate the login elements. It includes a flag, **loginButton._handlerAttached**, to ensure the event listener is attached only once. When the Log in button is clicked, it constructs a JSON object (**formData**) containing the function name (**retrieveCredentials**) and the comma-separated username and password. This **formData** is then converted to a string and passed to **invokeCSharpAction**, which is the method exposed by the iOS **WKScriptMessageHandler** to allow JavaScript to call C# methods. Now that you have the script, you have to edit the iOS bridge class, more specifically, the **DidReceiveScriptMessage** method, as follows:

```
public void DidReceiveScriptMessage(WKUserContentController
userContentController, WKScriptMessage message)
{
    try
    {
        var body = message.Body.ToString();

        JavascriptData _data =
        JsonConvert.DeserializeObject<JavascriptData>(body);

        if (_data.function == "retrieveCredentials")
        {
            var values = _data.data.Split(',');
            ((CustomWebView)_handler.VirtualView)?.
                OnTextBoxValuesRetrieved(values[0], values[1]);
        }
    }
    catch (Exception ex)
    {
        Debug.WriteLine(ex.Message);
    }
}
```

Here is a breakdown of the relevant code in this method:

- The **message.Body.ToString()** invocation extracts the message payload from JavaScript.
- The **JsonConvert.DeserializeObject<JavascriptData>(body)** deserializes the JSON string received from JavaScript into a **JavascriptData** object, which represents the structured data sent from the iOS JavaScript.
- If the JavaScript message is a request to retrieve credentials, the **_data.data.Split(',')** invocation splits the comma-separated username and password string received from JavaScript.
- The **((CustomWebView)_handler.VirtualView)?.OnTextBoxValuesRetrieved(values[0], values[1])** line accesses the **CustomWebView** instance and calls its **OnTextBoxValuesRetrieved** method, passing the extracted credentials to the main application logic.

Now you have everything you need to implement biometric authentication over the wrapped website from the code perspective. But before running the application, you need to request permission to use the biometric sensors on each OS.

Adding manifest permissions

By default, applications do not have permission to use biometric sensors, and this makes sense for privacy reasons. You must explicitly add permission requests in the application manifest. For Android, double-click the **ApplicationManifest.xml** file located in the **Platforms\Android** folder. When the Manifest editor appears (see Figure 21), select the **USE_BIOMETRIC** permission.

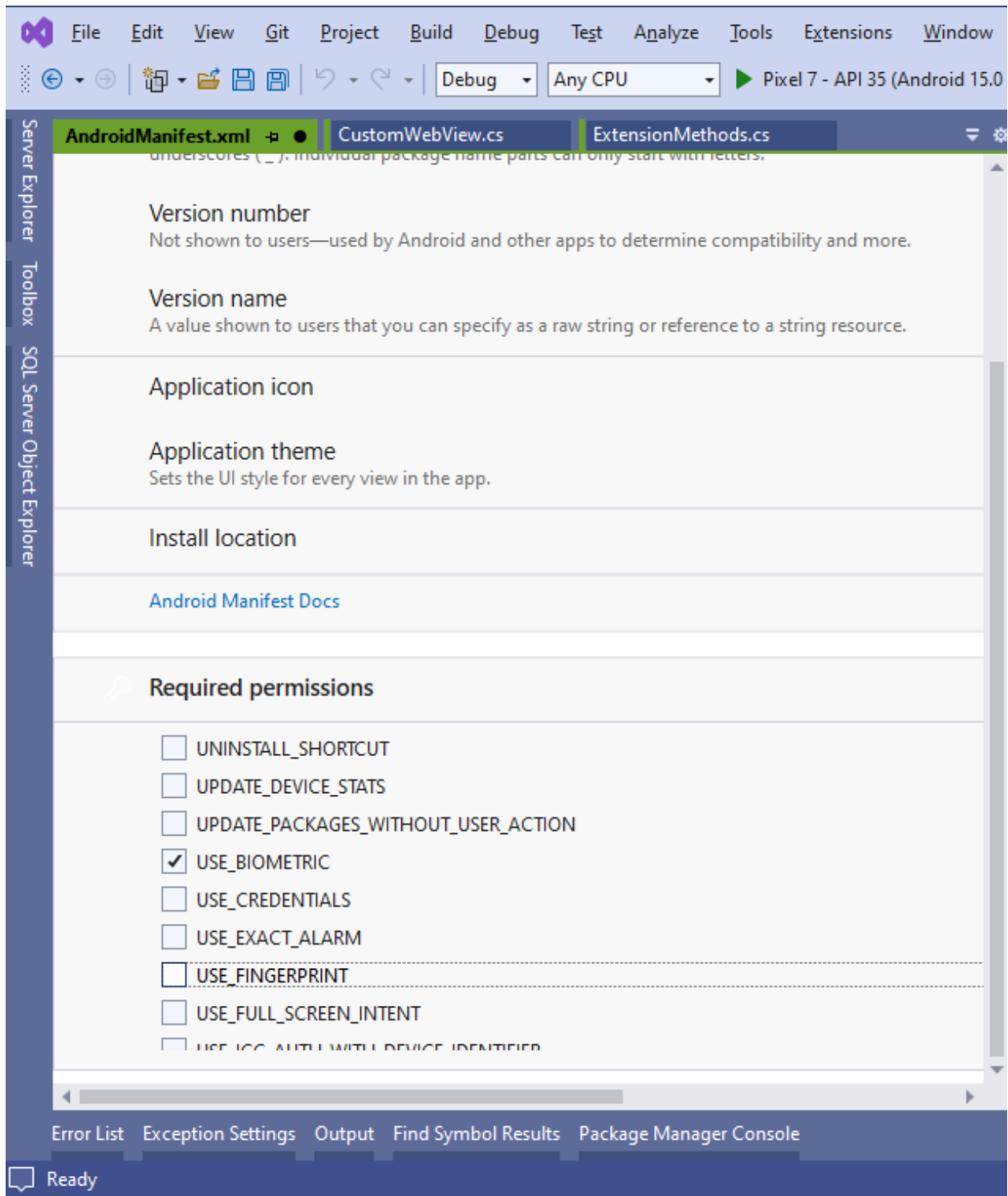


Figure 21: Specifying permissions in the Android manifest

When done, save the file and close it. For iOS, right-click the **Info.plist** file located in **Platforms\iOS**, and then select **Open With**. In the dialog box, select **XML (Text Editor)**. When the XML editor shows the content of the **Info.plist** file, add the following lines:

```
<key>NSFaceIDUsageDescription</key>  
<string>Required to implement biometric authentication</string>
```

The `NSFaceIDUsageDescription` permission tells the system that the application must ask the user for permission to use the biometric sensors, explaining the reasons specified in the string. The explanation for using the biometric sensors is extremely important and must be very clear; otherwise, the Apple App Store could reject the app submission. Now that you have added permission requests for both systems, you can test the application.

Running the application

In the Visual Studio toolbar, select the target device of choice and press F5 to start the application in debug mode.



Tip: The way the app asks the user permission for biometric authentication varies depending on the target system. Generally speaking, this happens via system dialogs.

When the external website appears in the application's main page, on iOS the app will ask your permission to enable biometric authentication, whereas on Android you first need to enter your credentials and select **Log in**. This is demonstrated in Figure 22.

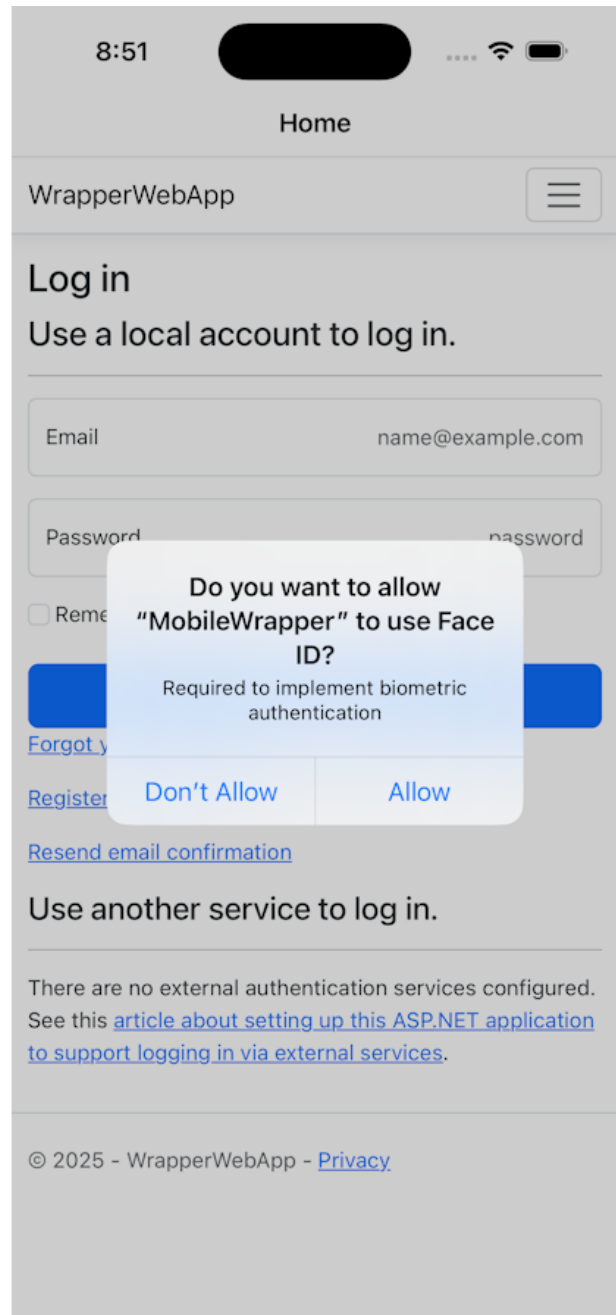
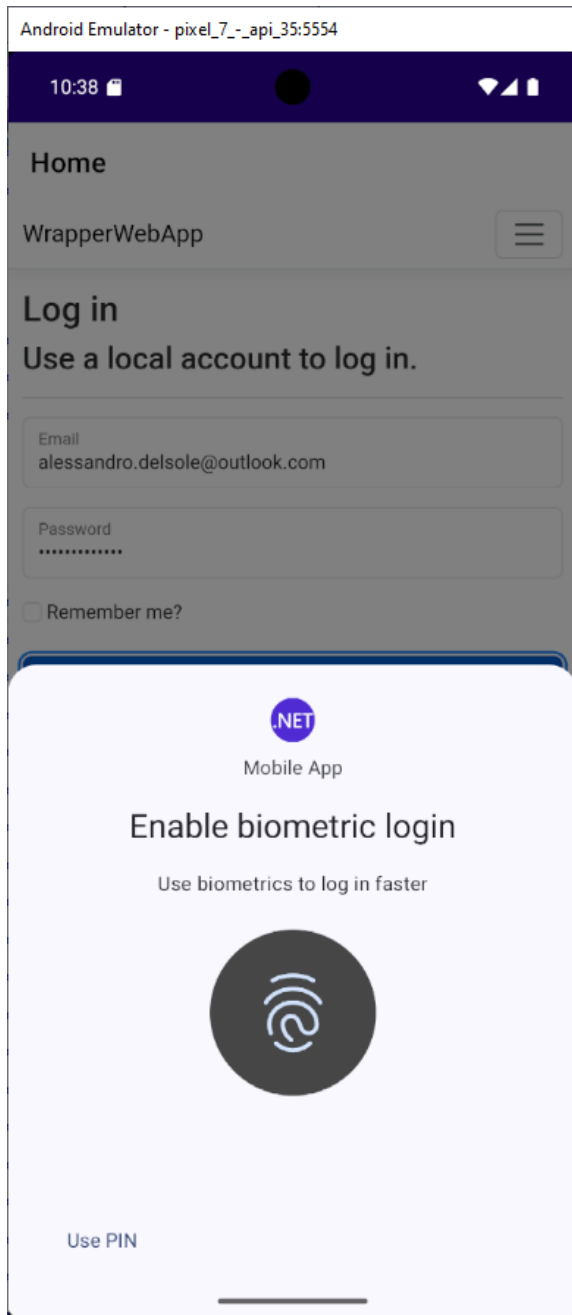


Figure 22: Enabling biometric authentication

If you now restart the application, you will see that it prompts you to log in with biometric authentication without entering your credentials manually, as shown in Figure 23.

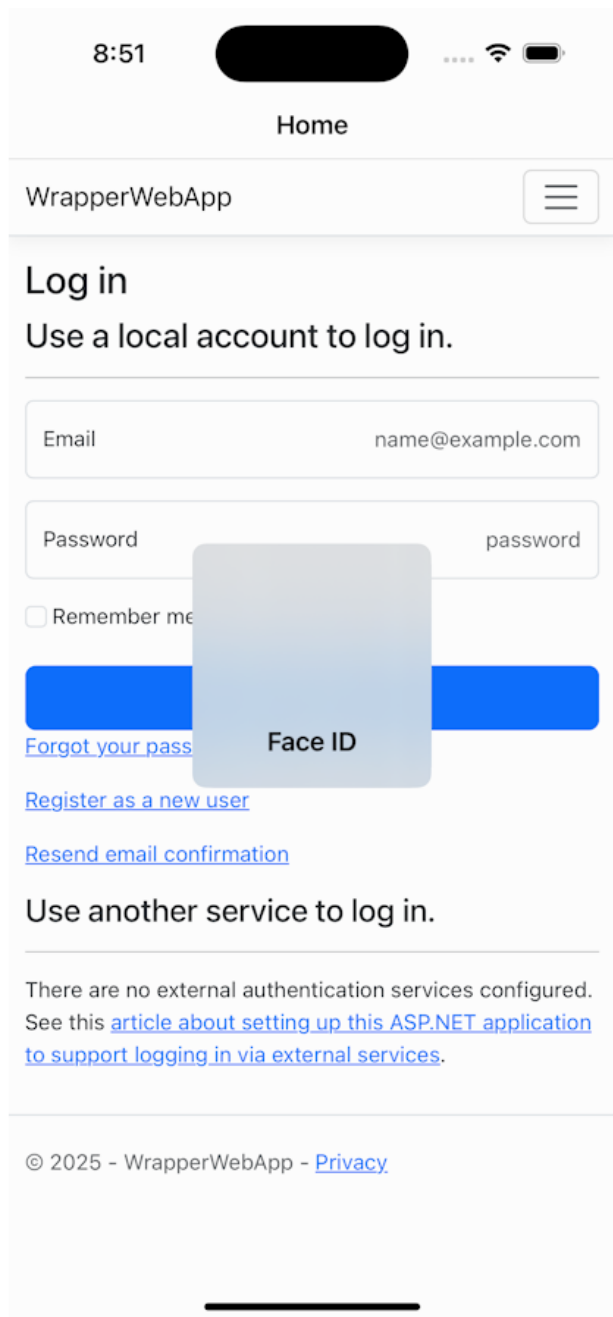
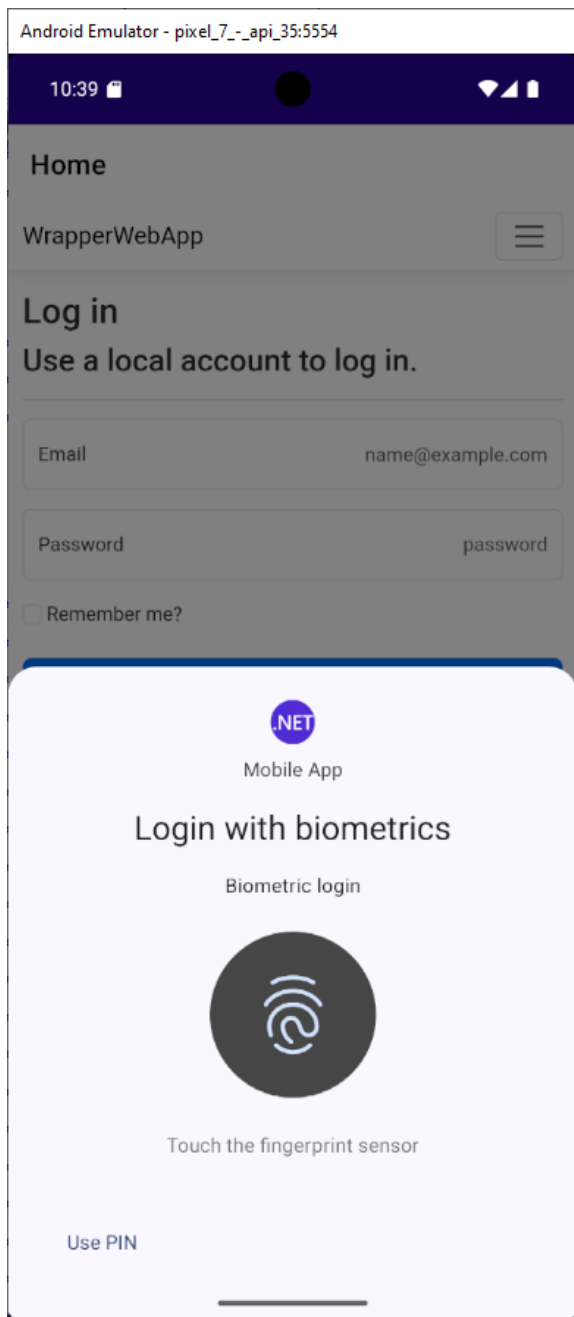


Figure 23: Logging into the app with biometric authentication

It has been a lot of work, but you can immediately understand the value of adding biometric authentication to the mobile user experience.

Chapter summary

This chapter provided a detailed guide to implementing biometric authentication within your .NET MAUI wrapped web application. This involved installing the `Oscore.Maui.Biometrics` NuGet package for device biometric interaction, extending the `CustomWebView` to enable custom event-driven communication, and injecting platform-specific JavaScript to capture login credentials from the web form upon submission.

You have also seen how to leverage the secure storage for securely storing and retrieving user credentials, and how to implement autologin functionality that prompts for biometric authentication, and if successful, automatically fills and submits the web login form. Finally, you learned to extend platform-specific bridge classes to receive data from JavaScript and trigger native events.

In the next chapter, you will implement code that enforces the user input based on data types.

Chapter 9 Handling Mobile Input Properly

The most modern mobile browsers and the most recent versions of the native Android `WebView` and iOS `WKWebView` components automatically detect the data type assigned to an input box inside a website, and they automatically show the appropriate keyboard. For example, if an input box is designed to enter a numeric value, the system will show the numeric keypad when the input box gets focus. However, if you plan to distribute your mobile wrappers to older devices and to ensure a consistent behavior across platforms, it is best practice to explicitly enforce the assignment of the proper keypad, depending on the target data type. This chapter explains how to accomplish this, based on the My Profile page of the website that includes a phone number field.



Tip: *The concepts described in this chapter apply to all the data types for which the device should display a specific keypad or component, such as the numeric keypad for numbers, the email keypad for email addresses, and the calendar for date/time fields.*

Defining the scripts

The My Profile page in the target website has a field called `PhoneNumber`, and the goal is to make sure that the telephone keyboard is displayed when the user enters text into this field. When the My Profile page is displayed, a JavaScript script must be injected into the page to retrieve the instance of the input box and explicitly assign the target keyboard to it.

For Android, add the following variable to the `AndroidScripts` class:

```
public static string AndroidReinforceTelephoneScript = @"var interval =
setInterval(() => {
    var phoneInput = document.getElementById('user-phone-number');
    if (phoneInput) {
        clearInterval(interval);
        phoneInput.type = 'tel';
        phoneInput.setAttribute('inputmode', 'tel');
    }
}, 500);";
```

For iOS, the script is the same, but it goes on one line and must be added to the `IOSScripts` class as follows:

```
public static string iOSReinforceTelephoneScript = @"var interval =
setInterval(() => {
    var phoneInput = document.getElementById('user-phone-number');
    if (phoneInput) {
        clearInterval(interval);
        phoneInput.type = 'tel';
    }
}, 500);";
```



```

        phoneInput.setAttribute('inputmode', 'tel');
    }
}, 500);";

```

This script uses `setInterval` to check every 500 milliseconds whether an input field with the ID `user-phone-number` has loaded in the DOM. This is to ensure that the assignment happens only if the element is available. Once it appears, the interval is cleared to stop further checking, and the input field is configured by setting its type to `tel` and its `inputmode` to `tel`, so that the native component will display a telephone keypad.

The next step is adding an extension method to the `ExtensionMethods` class, to extend the `CustomWebView` as follows:

```

        public async static Task InjectTelephoneKeyboardAsync(this
            CustomWebView webView)
        {
#if ANDROID
            await
webView.EvaluateJavaScriptAsync(AndroidReinforceTelephoneScript);
#elif IOS
            await webView.EvaluateJavaScriptAsync(
                iOSReinforceTelephoneScript);
#endif
        }

```



Tip: Differently from what you did for the biometric authentication, in this case, there is no need to invoke the code execution on the main UI thread, because you are not interacting with .NET MAUI visual elements, but only with elements in the HTML page.

Now you can conditionally invoke this method if the displayed page is My Profile.

Injecting the scripts

The scripts must be injected if the `CustomWebView` leads to the My Profile page of the website. This is accomplished by appending the following code to the if block of the `RootWebView_Navigated` event handler:

```

else if (e.Url.ToLower().Contains("/myprofile"))
    await RootWebView.InjectTelephoneKeyboardAsync();

```

There is nothing else you have to do, and you are ready to test your work.

Running the application

Press F5 to start the application in debug mode inside the target device of choice. Log in and open the **My Profile** page. If you go to enter a value in the **PhoneNumber** field, you will now see the telephone keypad appear, as shown in Figure 24.

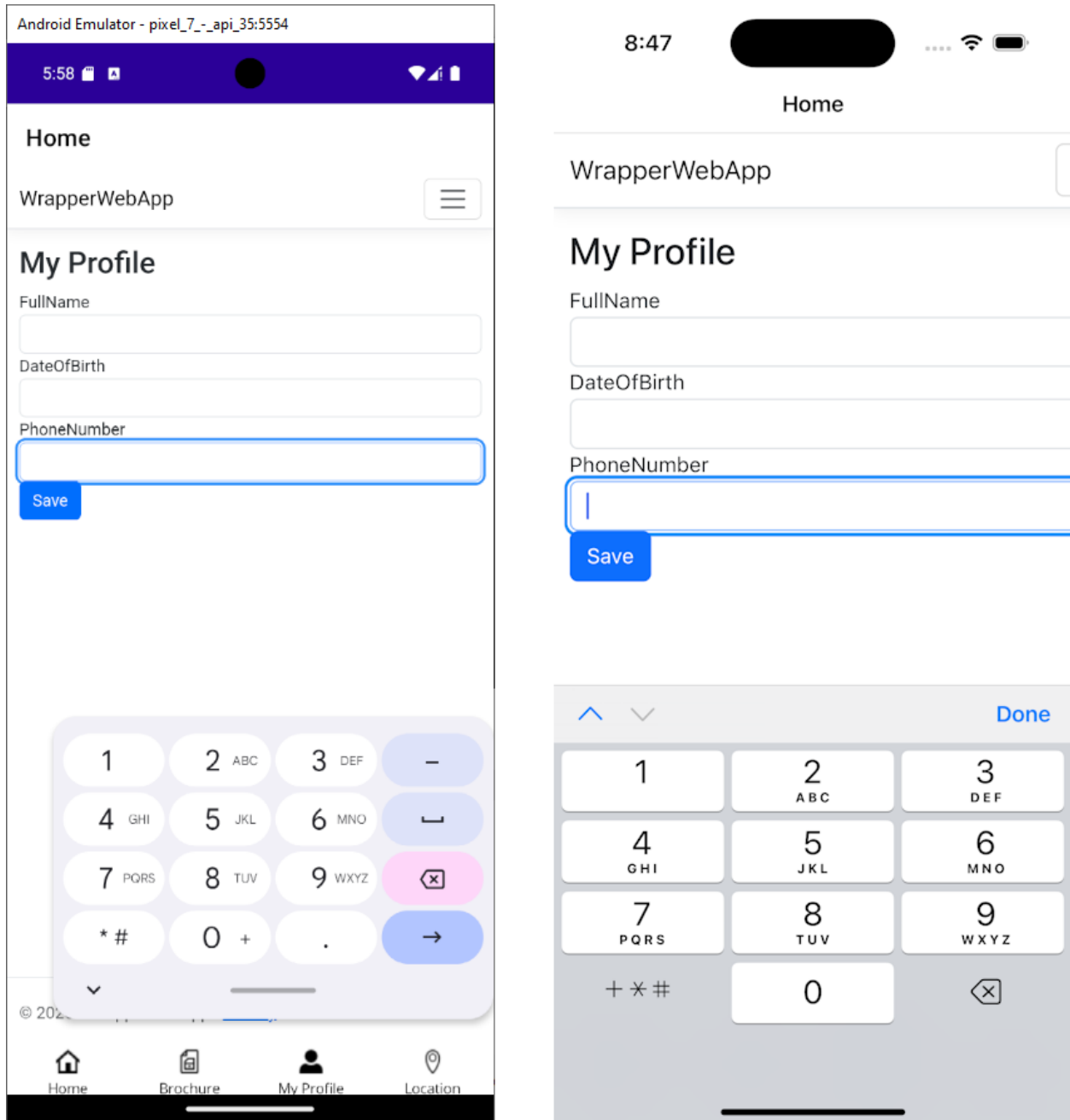


Figure 24: The application showing the specialized keypad

It is best practice to follow this approach to ensure a consistent behavior across platforms and devices, especially older ones. If you plan to distribute your app only to the most modern systems, you can always check if this step can be avoided.

Chapter summary

This chapter explained how to make sure that specialized keypads are displayed depending on the data type of the target input field. This requires a one-way JavaScript injection and no interaction with the bridge classes, so it is a very simple step.

In the next chapter, you will work on something more complex: implementing printing features in a mobile-oriented way.

Chapter 10 Printing and Sharing Documents

Printing and sharing documents in a mobile app wrapper work very differently from the process on a desktop. In fact, a website displayed in a desktop browser automatically uses the system printing service and provides an option to directly download a document. On mobile devices, printing requires manually instantiating the OS specific service; also, instead of directly downloading a document, a robust mobile experience requires you to prompt users with the system user interface that allows for sharing the document or opening it in their favorite application.

This chapter will guide you through the process of allowing users to print content displayed within the **WebView** and share documents directly from your wrapped web application using native device capabilities. You will learn how to intercept web-based print requests and link clicks and then trigger the appropriate platform-specific native actions.

Implementing printing capabilities

You might recall from Chapter 5 that the Privacy Policy page of the external website offers a Print button that displays the content of the full page. On the desktop, printing the page is very easy because of the following JS code:

```
window.print();
```

This automatically shows the printer dialog for the current system (it can be any desktop system), giving the user the opportunity to print the page. However, on mobile devices, that code has no effect. This means that you have to:

1. Write platform-specific C# code that instantiates the current OS printing service and retrieves the content of the **WebView** for printing.
2. Intercept the user's click on the **Print** button.
3. Redefine the behavior of the button by replacing its original code with new JavaScript code that invokes methods in the bridge class, whose job is to invoke the platform-specific code.

In the next paragraphs, you will learn how to implement all the aforementioned points so that your mobile wrapper can offer the proper printing experience.

Defining the platform-specific scripts

To handle printing natively, you must inject JavaScript that identifies the Print button or intercepts the **window.print()** call and then communicates this intent to the native MAUI application.

For iOS, you can add the following script to the **IOSScripts.cs** file (as always, on a single line):

```

public static string iOSPrintPageScript = @"var iPrint = 0;
var intervalPrint = setInterval(function () {
    var print_button = document.querySelector('#printButton');
    console.log('Found:', print_button);
    if (document.readyState === 'complete' && print_button != null) {
        clearInterval(intervalPrint);
        print_button.replaceWith(print_button.cloneNode(true));
        print_button = document.querySelector('#printButton');
        print_button.addEventListener('click', function (event) {
            event.preventDefault();
            var formData = {
                function: 'printPrivacy',
                data: ''
            };
            var jsonConvertedData = JSON.stringify(formData);
            invokeCSharpAction(jsonConvertedData);
        });
        console.log('Print button handler attached.');
```

This JavaScript snippet is designed, like the others, to interact with the DOM once it is fully loaded. It uses a `setInterval` function to continuously check for the existence of the `printButton` element. As soon as this element is detected, the interval is cleared to stop further checks. To override any existing event listeners attached to the button, the code replaces the original `printButton` function with a cloned version using `print_button.replaceWith(print_button.cloneNode(true))`. This cleans the button of any prior behaviors.

A new click event listener is then attached to the new button and `event.preventDefault()` prevents the default browser print action from being executed. Then, the script constructs a `formData` object with a function key set to `'printPrivacy'` and an empty data field to signal the intent to print a privacy page. This object is then serialized and sent to the bridge iOS class via `invokeCSharpAction(JSON.stringify(formData))`, exactly like in all the scripts seen so far.

For Android, the script to be added to the `AndroidScripts.cs` file is the following:

```

    public static string AndroidPrintPageScript = @"(function() {
function getElementByXPath(xpath) {
    return document.evaluate(xpath, document, null,
XPathResult.FIRST_ORDERED_NODE_TYPE, null).singleNodeValue;
}

const interval = setInterval(() => {
```

```

const printButton = getElementByXPath('//button[@id=""printButton""]');

if (printButton) {
    clearInterval(interval);

    // Remove the inline onclick handler
    printButton.removeAttribute('onclick');

    // Attach our custom listener
    printButton.addEventListener('click', function(e) {
        e.preventDefault();
        e.stopPropagation(); // Prevent other handlers
        jsBridge.printPrivacy(); // Call your MAUI-native code
    });

    console.log('Print button intercepted and inline handler removed.');
} else {
    console.log('Waiting for print button...');
}
}, 500);
})();";

```

Similar to its iOS counterpart, the Android script invokes `setInterval` to repeatedly search for the `printButton` element, but it uses `getElementByXPath` to retrieve it, since this approach works better on the Android `WebView`. Once the button is successfully located, the polling loop is stopped with `clearInterval(interval)`. To prevent the default browser-based print behavior, any existing click event handlers on the button are removed using `printButton.removeAttribute('onclick')`.

A new event listener is then attached to the button, which invokes both `e.preventDefault()` and `e.stopPropagation()` to suppress the standard action and stop the event from bubbling up the DOM. Instead of triggering the browser's native print function, the script calls `jsBridge.printPrivacy()`, which invokes the related method in the Android bridge class.

The next step is calling the native printing services of each platform.

Accessing the platform-specific printing services

The platform-specific scripts defined in the previous section need to initiate the printing services on each OS through code in the bridge classes. The code is divided by platform for better organization.

Accessing the Android printing services

For Android, you will need to add the following method to the `JSBridge` class:

```

public void PrintWebViewContent(string jobName)
{
    var activity = Platform.CurrentActivity;

```

```

    if (activity == null)
        return;

    // Get the current WebView instance
    var currentWebView = _handler.PlatformView;
    if (currentWebView == null)
        return;

    currentWebView.EvaluateJavascript("console.log('print');",
        new ValueCallback((result) =>
        {
            var printManager =
                (PrintManager)activity.
                GetSystemService(
                    global::Android.Content.Context.PrintService);
            var originalAdapter =
                currentWebView.CreatePrintDocumentAdapter(jobName);
            var customAdapter = new CustomPrintDocumentAdapter(
                currentWebView, originalAdapter, jobName);

            printManager.Print(jobName, customAdapter,
                new PrintAttributes.Builder().Build());
        }));

```

This method contains the core Android printing logic. More specifically, **Android.Print.PrintManager** is the system service responsible for handling print jobs on Android. It is obtained by invoking **activity.getSystemService(global::Android.Content.Context.PrintService)**. The **Android.Webkit.WebView.CreatePrintDocumentAdapter(jobName)** method is crucial in the printing process because it creates a **PrintDocumentAdapter**, which is responsible for laying out the **WebView**'s content for printing, handling pagination, and rendering to the print spooler.

The invocation to **Android.Print.PrintAttributes.Builder().Build()** defines the properties of the print job via **PrintAttributes**, such as page size, orientation, and margins. A default builder is used here. The native print dialog is finally initiated by invoking **printManager.Print(jobName, customAdapter, printAttributes)**, allowing the user to select a printer and configure print settings. The **customAdapter** (which wraps the **originalAdapter** from the **WebView**) provides the content to be printed.

This **PrintWebViewContent** method is invoked by JavaScript via the following bridge method:

```

[JavascriptInterface]
[Export("printPrivacy")]
public void PrintPrivacy()
{
    MainThread.BeginInvokeOnMainThread(() =>
    {
        PrintWebViewContent("PrivacyPolicy");
    });
}

```

Notice how the method runs the printing process on the main thread to avoid exceptions. Also, you can definitely use it to print multiple pages, so you can supply a different job name according to your needs.

Accessing the iOS printing services

For iOS, the `PrintWebViewContent` method is still added to the `JSBridge` class and has the following implementation:

```
public void PrintWebViewContent(string jobName)
{
    documentName = jobName;
    webView = _handler.PlatformView;
    if (webView == null)
        return;

    var printFormatter =
        webView.ViewPrintFormatter;
    var printInfo = UIPrintInfo.PrintInfo;
    printInfo.OutputType =
        UIPrintInfoOutputType.General;
    printInfo.JobName = jobName;

    var printController =
        UIPrintInteractionController.
        SharedPrintController;
    printController.PrintInfo = printInfo;
    printController.PrintFormatter = printFormatter;
    string script = string.Empty;

    printController.Present(true, (controller,
        completed, error) =>
    {
        if (!completed && error != null)
        {
            Console.WriteLine($"Error printing: " +
                $"{error.LocalizedDescription}");
        }
    });
}
```


The **PrintWebViewContent** method encapsulates the essential logic for printing, and it utilizes **UIKit.UIPrintInfo.PrintInfo**, a singleton class that provides metadata for the print job, such as type, orientation, and paper size. The output type is set to **UIKit.UIPrintInfoOutputType.General** to specify general document printing. To format the content for printing, it accesses **WKWebView.ViewPrintFormatter**, which returns an object of type **UIViewPrintFormatter** capable of rendering the content of the **WKWebView** appropriately. The actual print operation is managed by **UIKit.UIPrintInteractionController.SharedPrintController**, a shared controller that interacts with the native iOS print system. The **printController.Present** method invocation then shows the animated print dialog and includes a completion handler that responds to the print job outcome, whether it's a success, canceled by the user, or an error.

The next step is to add the following code to the main if block of the **DidReceiveScriptMessage** method:

```
else if (_data.function == "printPrivacy")
{
    PrintWebViewContent("Privacy Policy");
}
```

When the JavaScript code is executed, invoking the **printPrivacy** function, the bridge class will invoke the **PrintWebViewContent** method defined previously, initiating the native iOS printing service.

Injecting the scripts

The next step is adding an extension method to the **ExtensionMethods** class that injects the JavaScript for printing, depending on the current platform. Here's the code:

```
public async static Task PrintPageAsync(this CustomWebView webView)
{
#if ANDROID
    await webView.EvaluateJavaScriptAsync(AndroidPrintPageScript);
#elif IOS
    await webView.EvaluateJavaScriptAsync(iOSPrintPageScript);
#endif
}
```

It simply injects the proper platform-specific script into the instance of the **CustomWebView** class. In the **MainPage.xaml.cs** file, you will extend the main if block of the **RootWebView_Navigated** event with the following code:

```
else if (e.Url.ToLower().Contains("privacy"))
    await RootWebView.PrintPageAsync();
```

The script will be injected when the target page is the Privacy Policy page. You have completed the work to implement native printing. Before running the application to see the result, you will now move to implementing another important feature in the mobile experience: sharing documents.

Sharing documents

On desktop environments, when you open a document or file inside the browser, this will (generally) either display the document or start the file download if it is not possible to directly display its content due to an unsupported file type, so that the user can use a specific application to open it.

On the website that you created as the foundation for the examples of this book, the `Brochure.cshtml` page provides a hyperlink that you can click to display a PDF document called `brochure.pdf`, which will be opened in a separate tab of the browser.

On mobile devices, the user experience is designed to behave differently. In fact, a mobile application should prompt users with the file-sharing UI so that they can decide which app they want to use to open a document or file.



Note: *In addition, the `WebView` in `.NET MAUI` does not support opening links to separate tabs. It only supports navigation in the current instance, so you must modify the behavior of the website.*

This involves a few steps that are common to both Android and iOS:

1. Find and replace the JavaScript that displays the document in the webpage with JavaScript that intercepts the click and retrieves the document URL.
2. Write the new JavaScript to invoke a method in the bridge class that receives the document (or file) URL.
3. The method in the bridge class fires an event, whose handler includes the document URL. Such an event is intercepted by the main app page, which initiates the file-sharing UI on the device.
4. The user can select the app they want to use to display, edit, or store the document file.

In the next sections, you will implement all the necessary steps to provide the appropriate user experience in your mobile wrappers. The goal is to intercept the click event over the hyperlink that displays the PDF document in the `Brochure.cshtml` page of the external, sample website, and to prompt users with the file sharing user interface.

Defining the platform-specific scripts

You need to define some JavaScript code that will be injected into the `Brochure.cshtml` page when it's opened, which overrides the behavior of the PDF document hyperlink.

Starting with Android, in the **`AndroidScripts.cs`** file add the following C# code:

```
public static string AndroidPdfShareScript = @"(function() {  
function getElementByXPath(xpath) {  
return document.evaluate(xpath, document, null,  
XPathResult.FIRST_ORDERED_NODE_TYPE, null).singleNodeValue;  
}  
})
```

```

const interval = setInterval(() => {
    const pdfLink = getElementByXPath('//a[contains(@href, "'.pdf'")]');

    if (pdfLink) {
        clearInterval(interval);

        pdfLink.addEventListener('click', function(e) {
            e.preventDefault();

            const pdfUrl = new URL(pdfLink.getAttribute('href'),
window.location.origin).href;

            jsBridge.openPdf(pdfUrl);
        });

        console.log('PDF link click interception attached!');
    } else {
        console.log('Waiting for PDF link...');
    }
}, 500);
})();";

```

This JavaScript code continuously scans the webpage every 500 milliseconds using a `setInterval` loop to make sure the required elements are rendered, and it searches for an `<a>` element whose `href` attribute contains the string `.pdf`. It leverages an `XPath` expression via the helper function `getElementByXPath` to perform this precise search. Once the element is found, the script halts the interval using `clearInterval` and proceeds to bind a custom click event handler to the matched link.

When this link is clicked, the handler prevents the default navigation behavior using `e.preventDefault()`, then constructs a full, absolute URL to the PDF by combining the link's `href` with the current page's origin using the URL constructor. The resulting URL is passed to the native Android bridge class via the `jsBridge.openPdf` function (to be defined later in C#). Throughout the process, `console.log` messages track progress and provide insight into whether a target link has been found and whether interception has been successfully initialized.

On iOS, you will need to add the following code to the `IOSScripts.cs` file:

```

public static string iOSPdfShareScript = @"(function () {
    function getElementByXPath(xpath) {
        return document.evaluate(xpath, document, null,
XPathResult.FIRST_ORDERED_NODE_TYPE, null).singleNodeValue;
    }
    const interval = setInterval(() => {
        const pdfLink = getElementByXPath('//a[contains(@href, "'.pdf'")]');
        if (pdfLink) {
            clearInterval(interval);
            pdfLink.addEventListener('click', function (e) {
                e.preventDefault();

```

```

        const pdfUrl = new URL(pdfLink.getAttribute('href'),
window.location.origin).href;
        var formData = {
            function: 'openPdf',
            data: pdfUrl
        };
        invokeCSharpAction(JSON.stringify(formData));
    });
    console.log('PDF link click interception attached!');
} else {
    console.log('Waiting for PDF link...');
}
}, 500);
})();";

```

The behavior of the code is the same as Android. The difference is that, for iOS, the method that handles the URL in the native bridge class is invoked via the `invokeCSharpAction` function, passing the target function name and the document URL as a parameter.

Now that you have defined the scripts, you can move on to writing code that handles the returned document URL in the bridge class.



Tip: Notice how the JavaScript code is not limited to retrieving the specified document URL, but it applies to all .pdf documents on the page. This allows you to use the same script to retrieve any kind of file URL in the webpage.

Handling document URLs

The flow to handle the document URL after clicking on the hyperlink is the following:

1. A method in the bridge class receives the document URL from the JavaScript code.
2. This method fires an event that will be intercepted by the app's main page, passing the document URL.
3. The main page displays the file sharing user interface, allowing users to decide which external app they want to use to display the document.

Passing the document URL while firing an event requires a specific class that derives from **EventArgs**, defined as follows (to be added to the **Model** folder of the project):

```

public class PdfEventArgs : EventArgs
{
    public string PdfUrl { get; }
    public PdfEventArgs(string pdfUrl)
    {
        PdfUrl = pdfUrl;
    }
}

```

This class is very simple: its purpose is to just store the document URL.

Next, you need to extend the code of the **CustomWebView** with the following event:

```
public event EventHandler<PdfEventArgs>
    PdfRequested;
```

And the following method that fires the event, passing the document URL as an argument:

```
public void OnPdfRequested(string pdfUrl)
{
    PdfRequested?.Invoke(this, new PdfEventArgs(pdfUrl));
}
```

This method will be invoked in the bridge classes once they receive the document URL from JavaScript.

Extending the Android bridge class

For Android, the **JSBridge** class is extended with the following method:

```
[JavascriptInterface]
[Export("openPdf")]
public void OpenPdf(string pdfUrl)
{
    ((CustomWebView)_handler.VirtualView)?.OnPdfRequested(pdfUrl);
}
```

The code is very simple—when the JavaScript code invokes this method, the **OnPdfRequested** event is fired, passing the document URL to subscribers (in this particular case, the app's main page).

Extending the iOS bridge class

On iOS, the **JSBridge** class requires adding the following condition to the main **if** block of the **DidReceiveScriptMessage** method:

```
else if (_data.function == "openPdf")
    ((CustomWebView)_handler.VirtualView)?.OnPdfRequested(_data.data);
```

As always, the code parses the content of the **data** object from the JavaScript payload, the document URL in this case, which is passed as an argument to the event.

Injecting the scripts

Like in the previous mobile feature implementations, you need to first extend the **ExtensionMethods** class with a method that invokes the appropriate platform-specific script. The new method is defined as follows:

```
public static async Task HandlePdfAsync(this CustomWebView webView)
{
```

```

#if ANDROID
    MainThread.BeginInvokeOnMainThread(async () => {
        await webView.EvaluateJavaScriptAsync(AndroidPdfShareScript);
    });
#elif IOS
    await webView.EvaluateJavaScriptAsync(iOSPdfShareScript);
#endif
    }

```

Notice how, for Android, the code ensures that the script runs on the main UI thread. At this point, in the **MainPage.xaml.cs** file, you can add the following condition to the main **if** block of the **RootWebView_Navigated** event handler:

```

else if (e.Url.ToLower().Contains("/brochure"))
    await RootWebView.HandlePdfAsync();

```

This ensures the script is injected and executed when the **Brochure.cshtml** page is displayed.

Displaying the OS sharing user interface

.NET MAUI exposes cross-platform APIs to display the device-specific file sharing user interface with a few lines of code. More specifically, the **Share** class exposes methods to share links, files, text, and simple data. For the current example, what you need is the following, simple code:

```

private async void RootWebView_PdfRequested(object sender, PdfEventArgs e)
{
    if (!string.IsNullOrEmpty(e.PdfUrl))
    {
        await Share.Default.RequestAsync(new ShareTextRequest
        {
            Uri = e.PdfUrl,
            Title = "Open PDF"
        });
    }
}

```

The **PdfRequest** event, fired from the bridge classes and intercepted here, includes the document URL. The **RequestAsync** method initiates the file sharing UI, and the **ShareTextRequest** object specifies the target of the file sharing UI. In this case, the target is a URL represented by a title.

Before running the application, you need to add the following event handler reference to the **CustomWebView** object in the **MainPage.xaml** file:

```
PdfRequested="RootWebView_PdfRequested"
```

Now you have everything you need, and you can run the application to test both printing and document sharing features.

Running the application

You are now ready to test both printing and document-sharing features. Press F5 to start debugging the application on your favorite device and platform. When the external website is loaded on the main page, log in either with biometric authentication or by supplying your credentials. Note that the Privacy Policy page does not require you to be authenticated, but the Brochure page does.

Testing the printing service

When you're ready, open the **Privacy Policy** page and click the **Print Page** button. At this point, the printing service on the device will be initiated, and the page preview will be displayed, as shown in Figure 25.

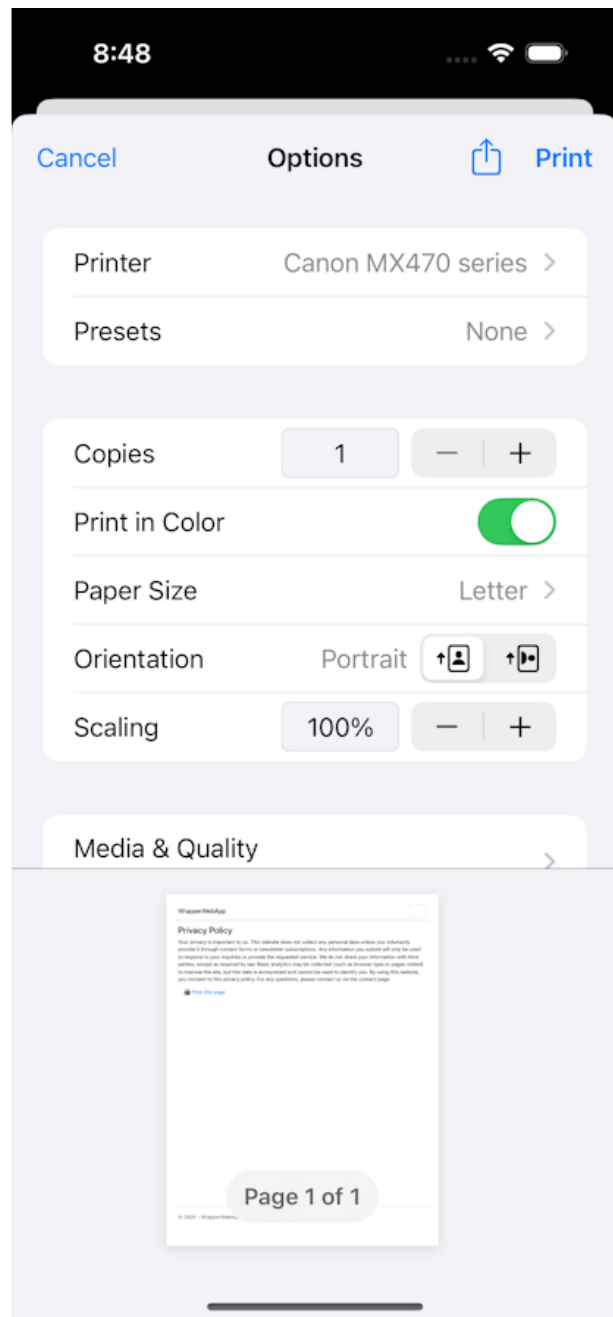
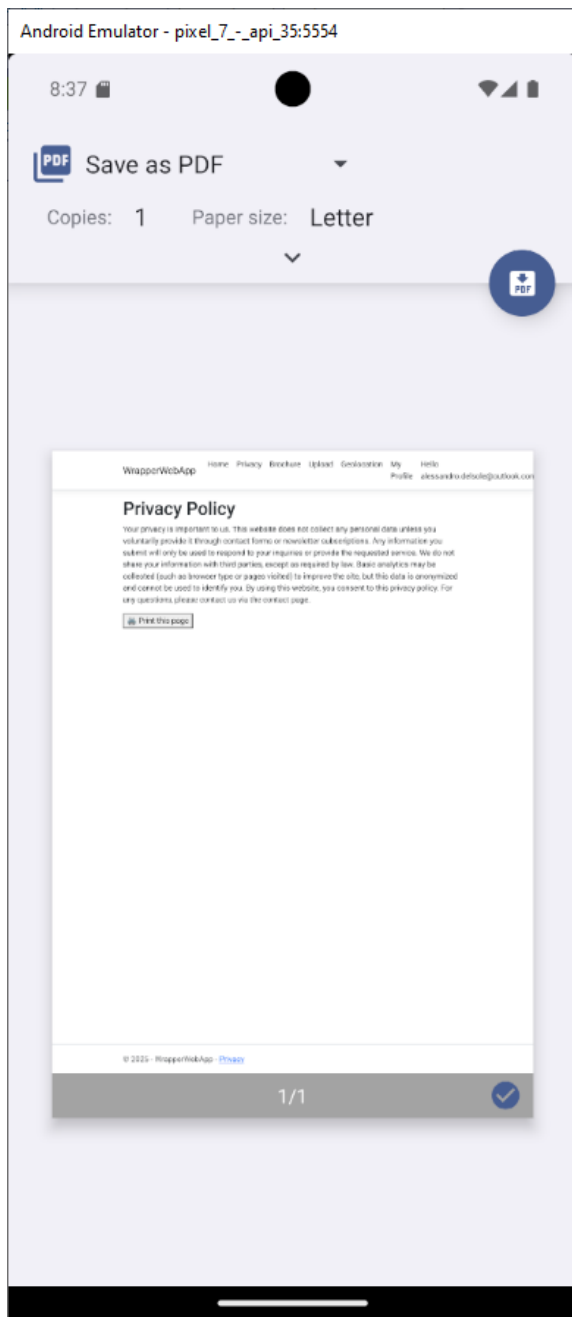


Figure 25: The printing service running on both platforms

On Android, if no printer is available, you can still print the page as a PDF, at least for testing purposes. On iOS, printing as a PDF is not available as an option, and you need a printer connected to the same network. You can still preview the printing result if no printer is available.

Testing the document sharing feature

If you now open the **Brochure** page, either via the mobile navigation bar or the website hyperlink, and click the document hyperlink, you will see how the application prompts users with the option to choose the app to display the document, as shown in Figure 26. You will likely have different options, depending on which apps you have installed on your device.

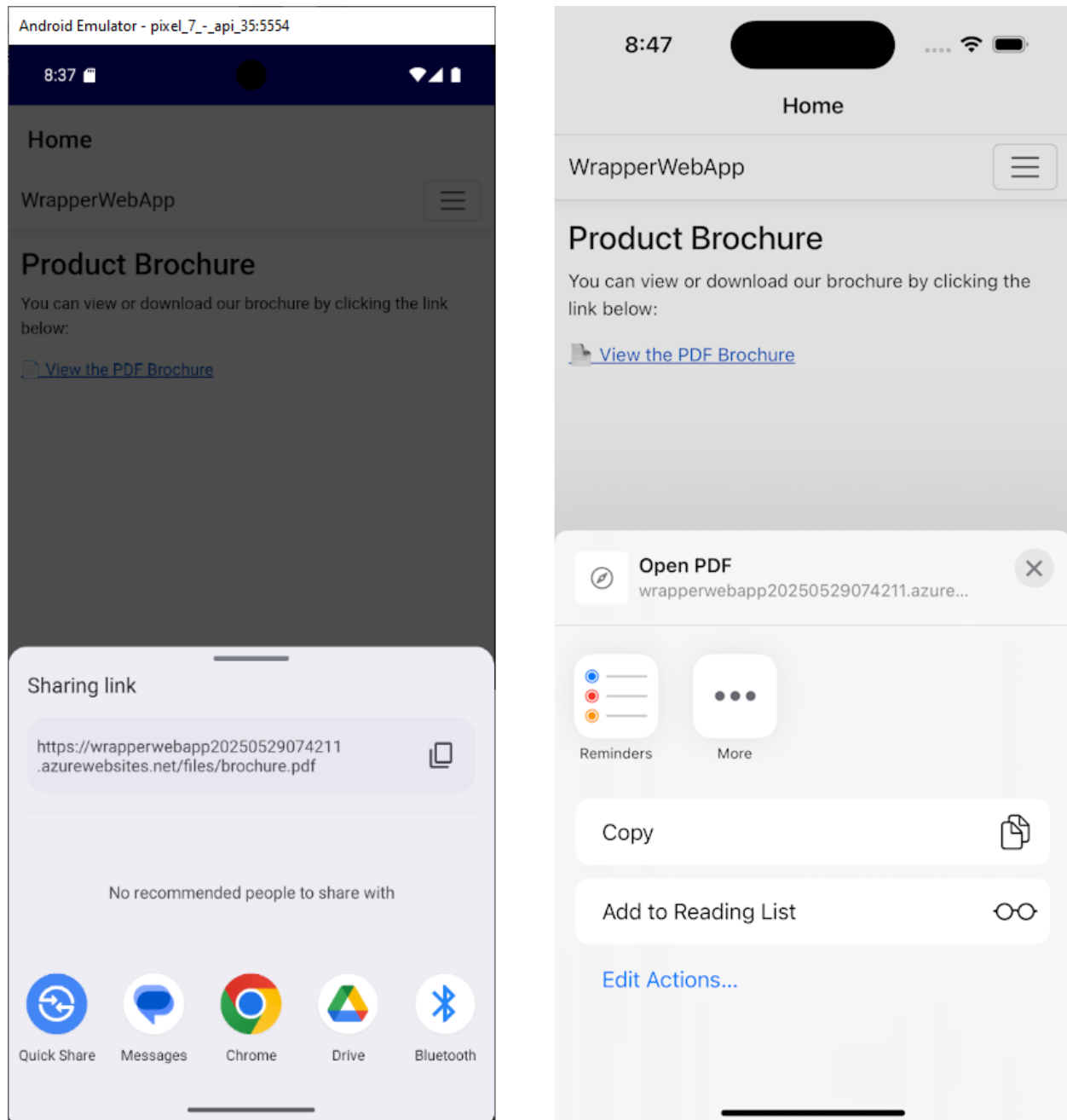


Figure 26: The file sharing user interface

In this way, you have improved the mobile experience of your app, satisfying the Google Play and Apple App Store requirements of offering native features over web wrappers.

Chapter summary

In this important chapter, you have seen how to leverage the platform-specific printing services to override the default browser behavior, making it possible for your mobile wrappers to print web content. Next, you saw how to implement another important mobile feature, the file-sharing user interface over documents, overriding hyperlink clicks in the browser. In both cases, you injected JavaScript code that redefines button and hyperlink clicks, sending the necessary information to the native bridge classes for data manipulation and event handling. With specific regard to the file sharing feature, you have seen how the .NET MAUI's **Share** API makes it extremely simple to share content in a cross-platform approach.

In the next chapter, you will make a step forward by implementing another important feature, which is sensor integration.

Chapter 11 Working with Sensors and Permissions

It is very common for mobile applications to integrate data from device sensors, and consequently, appropriately managing permissions becomes essential. This chapter demonstrates how to access a device's geolocation capabilities from your wrapped web application. You will learn how to intercept web-based requests for location data, prompt the user for necessary native permissions, retrieve the location using .NET MAUI's cross-platform APIs, and manipulate the resulting data.



Note: *Devices have many sensors, and it is not possible to demonstrate the interaction with each of them in this book. However, the content presented here is structured in a way that can be easily reused with other sensors as well, so that you have a solid foundation on this topic.*

Intercepting web geolocation requests with JavaScript

If you look back at the external website implementation, you can recall how web applications typically use the `navigator.geolocation.getCurrentPosition()` JavaScript API to request location data. Such an API has no effect when the page is displayed in Android's native **WebView**, while it partially works on the native iOS **WKWebView**, meaning that it can display the location data, but manipulating the data has limitations.

To provide native control and ensure proper permission handling, you must inject JavaScript into the .NET MAUI **WebView** that overrides this standard web API. This allows your application to intercept the request and handle it using device-specific geolocation services. To accomplish this, you first need to define the JavaScript code that will be injected into the **CustomWebView**.

Defining the Android scripts

To override the JavaScript location API from .NET MAUI, add the following code to the **AndroidScripts.cs** file:

```
public static string AndroidGetLocationScript = @"
setInterval(() => {
    if (document.readyState === 'complete') {
        navigator.geolocation.getCurrentPosition = function(successCallback,
errorCallback) {
            jsBridge.getLocation()
                .then(function(json) {
                    const pos = JSON.parse(json);
                    successCallback({
                        coords: {
```

```

        latitude: pos.latitude,
        longitude: pos.longitude,
        accuracy: pos.accuracy
    },
    timestamp: Date.now()
  ));
}
}
}, 300);";
}

```

This JavaScript code uses a `setInterval` function to repeatedly check if the document is fully loaded. Once loaded, it overrides the `navigator.geolocation.getCurrentPosition` function. When the web content calls this function, instead of using the browser's default behavior, it calls `jsBridge.getLocation()`. This `jsBridge` object is the mechanism for JavaScript to invoke the appropriate methods from the native bridge class. The `then` and `catch` blocks handle the asynchronous response from the native C# method, parsing the JSON location data and passing it back to the original web `successCallback` or `errorCallback`.

Defining the iOS scripts

For iOS, add the following code to the `IOSScripts.cs` file:

```

public static string iOSGetLocationScript = @"setInterval(() => {
    if (document.readyState === 'complete') {
        navigator.geolocation.getCurrentPosition = function (successCallback,
errorCallback) {
            try {
                const callbackId = 'cb_' + Date.now();
                window._geoCallbacks = window._geoCallbacks || {};
                window._geoCallbacks[callbackId] = {
                    successCallback,
                    errorCallback
                };
                var formData = {
                    function: 'getLocation'
                };
                invokeCSharpAction(JSON.stringify(formData));
            } catch (error) {
                if (typeof errorCallback === 'function') {
                    errorCallback({
                        code: 1,
                        message: error.message
                    });
                }
            }
        };
    }
});

```

```

    }
    };
}
}, 3000);";

```

Similar to the Android script, this script overrides `navigator.geolocation.getCurrentPosition`. When this function is called by the web content, it generates a unique `callbackId` and stores the original `successCallback` and `errorCallback` functions in a global `window.geoCallbacks` object. It then constructs a `formData` object with `function: 'getLocation'` and sends it to the native bridge class via the usual `invokeCSharpAction()` function. This allows the native iOS code to retrieve the location and then use the stored callbacks to manipulate location data.

Requesting user permissions

When a mobile app wants to access device sensors that might have an impact on a user's privacy, it is mandatory to request permissions and to explain the reasons for accessing sensor data. This means that before accessing location data, the sample application must request and obtain the necessary permissions from the user and the operating system. More specifically, the most recent operating system versions allow the user to select if the location can be used only when the application is running or also for background tasks.



Note: *In this chapter, you will prompt users for both permissions only for demonstrational purposes. In real-world scenarios, you will carefully select which permissions you really need.*

Declaring permissions on Android

For Android, you must first open the manifest editor, as you already did for biometric authentication. Double-click the `AndroidManifest.xml` file. When the editor appears, make sure to select `ACCESS_COARSE_LOCATION` and `ACCESS_FINE_LOCATION`, as shown in Figure 27.

Required permissions	
☐	ACCESS_BLOBS_ACROSS_USERS
☐	ACCESS_CHECKIN_PROPERTIES
☒	ACCESS_COARSE_LOCATION
☒	ACCESS_FINE_LOCATION

Figure 27: Declaring location permissions on Android

You will still need to write some code to ask for permissions, but this is done later in a cross-platform approach.

Declaring permissions on iOS

For iOS, open the **Info.plist** file with the built-in XML editor and add the following lines:

```
<key>NSLocationWhenInUseUsageDescription</key>
<string>We need your location to show relevant content.</string>
<key>NSLocationAlwaysUsageDescription</key>
<string>We need your location to show relevant content.</string>
<key>NSLocationAlwaysAndWhenInUseUsageDescription</key>
<string>We need your location to show relevant content.</string>
```

The key names are self-explanatory, and the important thing is to specify the reason you need to access location services. A description that is not detailed enough might result in the app's rejection from the Apple App Store.

Requesting permissions in C#

Before accessing location data, your mobile application must request and obtain the necessary permissions from the user and the operating system. Luckily enough, .NET MAUI exposes the cross-platform API that makes this step very easy.

Now, add the following method to the **MainPage.xaml.cs** file:

```
public async Task<bool> RequestGeolocationPermissionAsync()
{
    var statusWhenInUse =
        await Permissions.CheckStatusAsync<Permissions.LocationWhenInUse>();

    if (statusWhenInUse != PermissionStatus.Granted)
    {
        statusWhenInUse =
            await Permissions.RequestAsync<Permissions.LocationWhenInUse>();
    }

    var statusAlways =
        await Permissions.CheckStatusAsync<Permissions.LocationAlways>();

    if (statusAlways != PermissionStatus.Granted)
    {
        statusAlways =
            await Permissions.RequestAsync<Permissions.LocationAlways>();
    }

    return statusWhenInUse ==
        PermissionStatus.Granted &&
```

```

        statusAlways == PermissionStatus.Granted;
    }

```

The **Permissions** class represents the cross-platform API that manages permissions from shared code. The **RequestGeolocationPermissionAsync** method handles the native permission requests. The **Permissions.CheckStatusAsync<Permissions.LocationWhenInUse>** method checks the current status of the "Location When In Use" permission. This permission allows the app to access location data only when the app is actively in use (foreground).

The **Permissions.RequestAsync<Permissions.LocationWhenInUse>** method prompts the user to grant this permission if not previously granted. Similarly, the **Permissions.CheckStatusAsync<Permissions.LocationAlways>** method checks the status of the "Location Always" permission, which allows the app to access location data even when it is in the background. This permission is more sensitive and often requires stronger justification to the user and the app store.

The **Permissions.RequestAsync<Permissions.LocationAlways>** method prompts the user to grant the permission if not previously granted. The method returns **true** only if both "Location When In Use" and "Location Always" permissions are granted, ensuring comprehensive location access.



Tip: While requesting permissions from shared code should be enough, it is best practice to reinforce permission handling in native code, too. You will do this shortly in the bridge classes, and the good news is that you will use the same MAUI API.

The appropriate point for requesting permissions is before the target webpage is displayed, so in the **Navigating** event of the **CustomWebView**. The code for the event handler now looks like this:

```

private async void RootWebView_Navigating(object sender,
    WebNavigatingEventArgs e)
{
    RootWebView.Opacity = 0.3;
    SpinnerGrid.IsVisible = true;
    bool geoPermission = false;
    bool photoPermission = false;
    if (e.Url.ToLower().Contains("/geolocation"))
    {
        geoPermission = await RequestGeolocationPermissionAsync();
        if (!geoPermission)
            e.Cancel = true;
    }
}

```

As you can see, if the user does not grant permission, the navigation is canceled.

Injecting JavaScript code

In order to simplify the approach to location handling, this time you will first write the code that injects the scripts, and then you will see how to manage the location results in the bridge classes. With this in mind, and having already defined the scripts, add the following code to the `ExtensionMethods` class:

```
        public async static Task InjectLocationScriptAsync(this CustomWebView
webView)
        {
#if ANDROID
            await webView.EvaluateJavaScriptAsync(AndroidGetLocationScript);
#elif IOS
            await webView.EvaluateJavaScriptAsync(iOSGetLocationScript);
#endif
        }
```

You know very well how this works, so no further explanation is needed. The actual injection happens in the `RootWebView_Navigated` event handler, if the user has previously granted permissions, so you need to add the following condition to the main `if` block in the method:

```
else if (e.Url.ToLower().Contains("/geolocation"))
    await RootWebView.InjectLocationScriptAsync();
```

Now you will move to implementing platform-specific code in the native bridge classes.

Managing location data in the bridge classes

The platform-specific bridge classes are responsible for receiving the `getLocation` call from the injected JavaScript and using MAUI's Geolocation API to retrieve the actual device location.

Android implementation

For Android, add the following method to the `JSBridge` class:

```
[JavascriptInterface]
[Export("getLocation")]
public void GetLocation()
{
    MainThread.BeginInvokeOnMainThread(async () =>
    {
        try
        {
            var status =
                await Permissions.RequestAsync<Permissions.LocationWhenInUse>();
            if (status != PermissionStatus.Granted)
                return;
        }
    });
}
```



```

        var statusAlways =
            await Permissions.RequestAsync<Permissions.LocationAlways>();
        if (status != PermissionStatus.Granted)
            return;

        var location =
            await Geolocation.GetLastKnownLocationAsync()
            ?? await Geolocation.GetLocationAsync(
                new GeolocationRequest(GeolocationAccuracy.Medium));

        if (location == null)
            return;

        ((CustomWebView)_handler.VirtualView)?.
            OnLocationRetrieved(location.Latitude,
                location.Longitude, location.Accuracy);
    }
    catch (Exception ex)
    {
        // handle exceptions here...
    }
});
}

```

Before attempting to retrieve location data, the code first enforces permissions checks for both the **LocationWhenInUse** and **LocationAlways** scenarios via the **Permissions.RequestAsync** method. It then tries to fetch cached location data using **Geolocation.GetLastKnownLocationAsync**, providing a fast response without engaging the device's GPS. If no valid location is available through that method, it proceeds to actively request a new reading by invoking the **Geolocation.GetLocationAsync** method, with a **GeolocationRequest** set to medium accuracy.

Once the location is successfully obtained, either from cache or via a fresh query, the result is forwarded to the mobile application's UI layer by invoking the **OnLocationRetrieved** method (to be defined shortly) on the associated **CustomWebView** instance, transmitting the latitude, longitude, and accuracy parameters.

iOS implementation

For iOS, add the following condition to the main **if** block of the **DidReceiveScriptMessage** method:

```

else if(_data.function == "getLocation")
{
    try
    {
        MainThread.BeginInvokeOnMainThread(async () =>
        {

```

```

        var status =
            await Permissions.RequestAsync<Permissions.LocationWhenInUse>();
        if (status != PermissionStatus.Granted)
            return;
        var statusAlways =
            await Permissions.RequestAsync<Permissions.LocationAlways>();
        if (status != PermissionStatus.Granted)
            return;

        var location =
            await Geolocation.GetLastKnownLocationAsync()
                ?? await Geolocation.GetLocationAsync(
                    new GeolocationRequest(GeolocationAccuracy.Medium));

        if (location == null)
            return;

        ((CustomWebView)_handler.VirtualView)?.
            OnLocationRetrieved(location.Latitude, location.Longitude,
                                location.Accuracy);
    });
}
catch (Exception ex)
{
    // Handle exceptions here
}
}

```

Sensor integration and management happen on the main UI thread, and this is why the code execution begins by invoking **MainThread.BeginInvokeOnMainThread**. Before retrieving any location data. Similar to what you did for Android, the app verifies once again that the necessary permissions, **LocationWhenInUse** and **LocationAlways**, are granted by invoking the **Permissions.RequestAsync** method.

The retrieved location information is then accessed through MAUI's cross-platform APIs—the **Geolocation.GetLastKnownLocationAsync** method is used to quickly retrieve cached data, while **Geolocation.GetLocationAsync** with medium accuracy is invoked to obtain fresh data if no recent location is available. Once the location data is acquired, it is relayed back to the application's main page via the **OnLocationRetrieved** method, passing along latitude, longitude, and accuracy details.

Handling events and managing location data

The code in the bridge classes triggers an event when they receive the location information from JavaScript by invoking a method called **OnLocationRetrieved**. This method must be added to the **CustomWebView** class as follows:

```

public void OnLocationRetrieved(double latitude,

```

```

        double longitude, double? accuracy)
    {
        LocationRetrieved?.Invoke(this,
            new LocationEventArgs(latitude, longitude, accuracy));
    }

```

It requires the following event definition:

```

public event EventHandler<LocationEventArgs>
    LocationRetrieved;

```

The **LocationEventArgs** object is used to pass the location information to the event subscribers, and is defined as follows:

```

public class LocationEventArgs: EventArgs
{
    public double Latitude { get; }
    public double Longitude { get; }
    public double? Accuracy { get; }

    public LocationEventArgs(double latitude,
        double longitude, double? accuracy)
    {
        Latitude = latitude;
        Longitude = longitude;
        Accuracy = accuracy;
    }
}

```

In the **MainPage.xaml** file, add the following line to the **RootWebView** declaration so that it subscribes to the **LocationRetrieved** event:

```

LocationRetrieved="RootWebView_LocationRetrieved"

```

In the **MainPage.xaml.cs** file, add the following event handler:

```

private async void RootWebView_LocationRetrieved(object sender,
    LocationEventArgs e)
{
    string location = $"Latitude: {e.Latitude}, Longitude " +
        $"{e.Longitude}, Accuracy: {e.Accuracy}";
    await DisplayAlert("Location", location, "OK");
}

```

When the location information is sent from the bridge class to subscribers, the main page displays a native alert with the relevant information. Displaying an alert is just for the sake of simplicity, but this is the point where you can manipulate location information within the mobile app from shared C# code.

Running the application

If you run the application on your favorite device now, you can log in and open the **Geolocation** page. The first time you do this, you will see the permission request prompt, as shown in Figure 28.

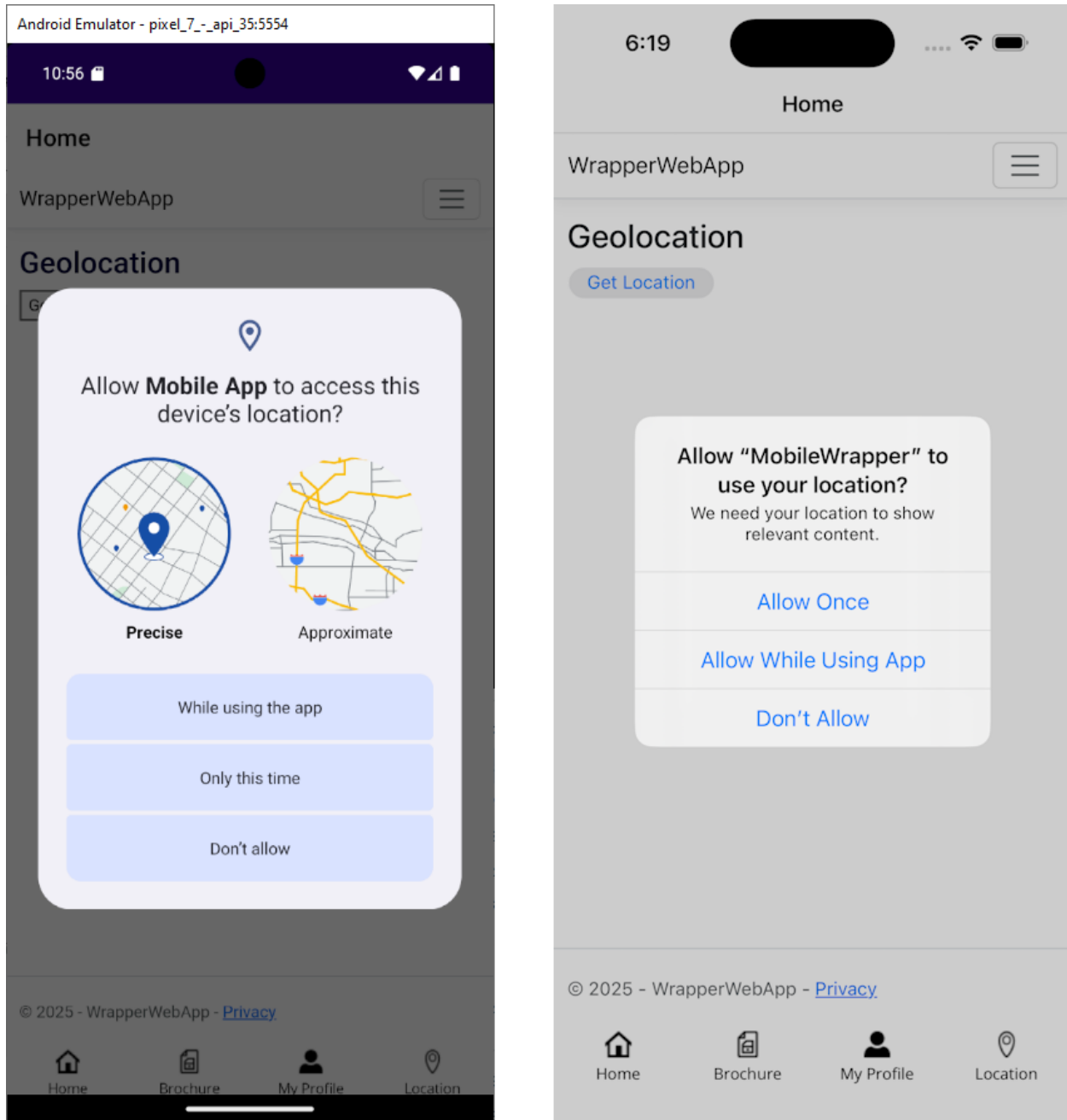


Figure 28: The application requesting location permission

Select **While using the app**. When the Geolocation page appears, click the **Get Location** button, and you will see the native alert displaying the retrieved information, as shown in Figure 29.

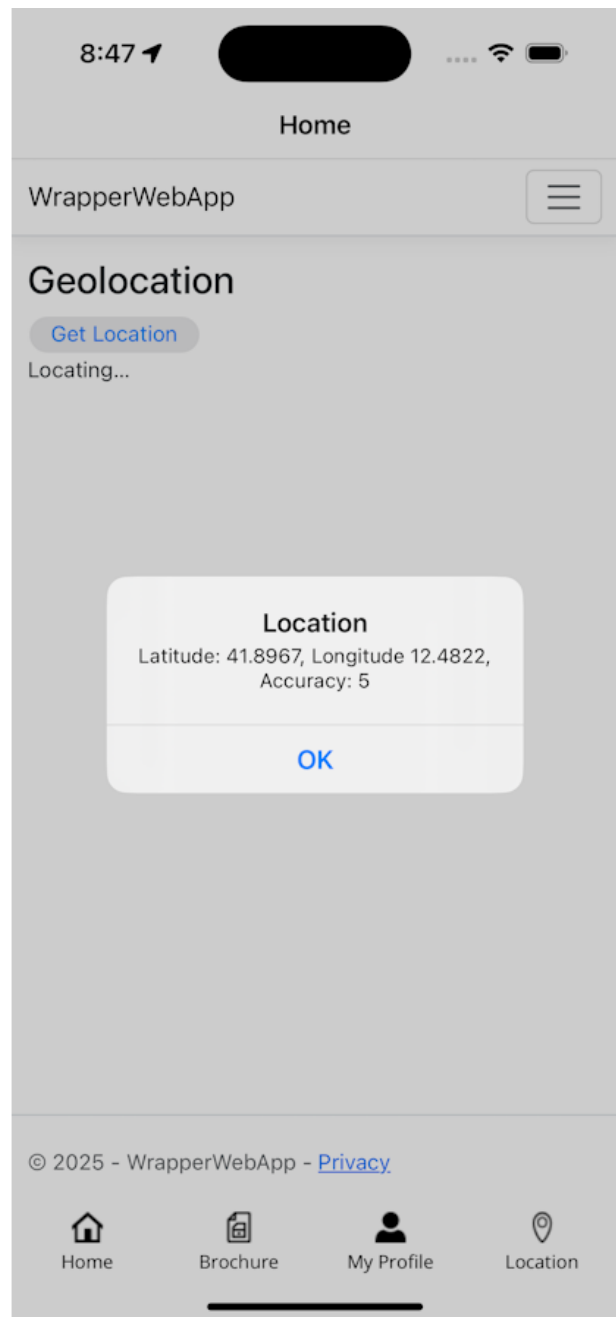
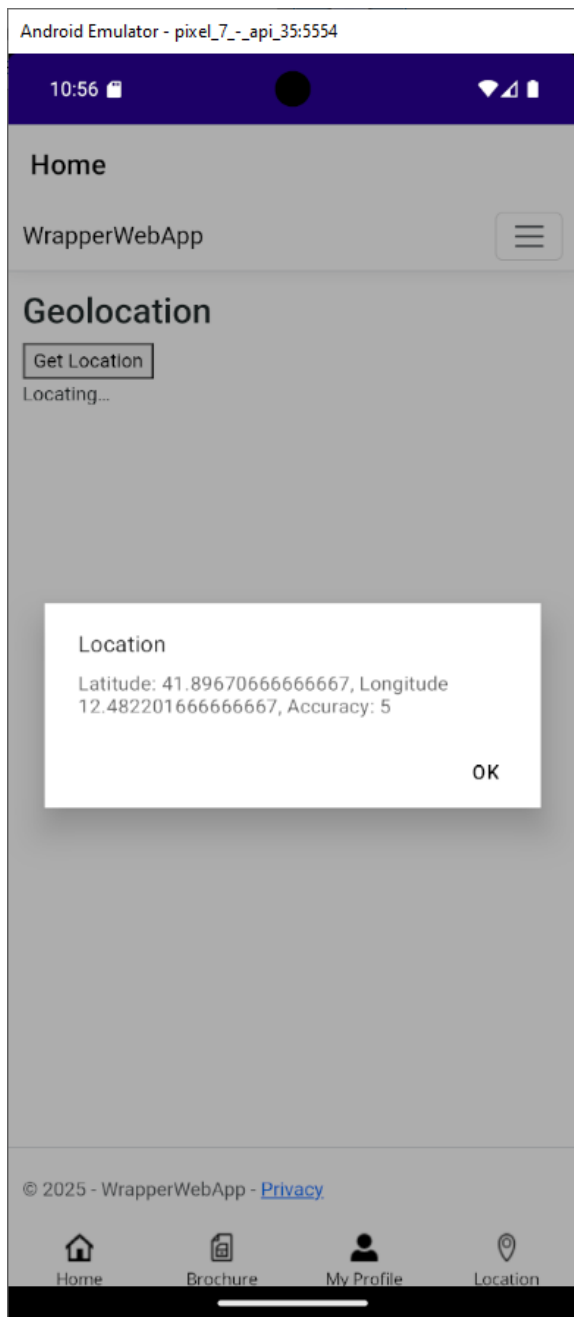


Figure 29: Manipulating location information

As mentioned previously, the native alert is used for demonstrational purposes, but this is the point where you can manipulate location information, like calculating a route.

A common approach for sensors

At the beginning of this chapter, you read that it is obviously not possible to explain how to use all the device sensors in this book, but you also read that you would have gone through a common approach. This can be summarized as follows:

1. Declaring permissions in the manifests for the given sensors.
2. Writing code to ask for permissions from users.
3. Defining JavaScript code that accesses sensor data and sends data to the C# bridge classes.
4. Implementing methods in the bridge classes that receive sensor data and fire an event, whose argument is the received data.
5. Implementing code in the main page, or other subscriber pages, that receive and manipulate sensor data using .NET MAUI cross-platform API.

In this way, you have a checklist to follow for whichever sensor you want to access.

Chapter summary

Chapter 11 provided a detailed explanation of integrating device sensors and handling permissions within your wrapped web application. You learned how to seamlessly override the web's `navigator.geolocation.getCurrentPosition()` API using platform-specific JavaScript injection, redirecting location requests to the native MAUI application.

Furthermore, you implemented robust permission handling using the **Permissions** API to ensure user consent for location access, and you saw how to retrieve actual location data using the **Geolocation** API. Finally, the retrieved location data was passed back to the **CustomWebView** and displayed in a native alert, demonstrating a complete end-to-end integration of native sensor capabilities with your wrapped web content.

Chapter 12 Handling Network Connections

A critical aspect of delivering a robust mobile application experience is gracefully handling changes in network connectivity. Mobile devices frequently encounter situations where internet access is lost or restored, and an effective application must respond appropriately to these changes.

This chapter focuses on implementing a mechanism to detect network status, inform the user about the lack of connection, and provide a convenient way to access platform-specific network settings to restore connectivity. This enhances the user experience by preventing the application from appearing unresponsive or broken when offline, satisfying another requirement from Google Play and the Apple App Store. This is also the first time that you'll implement a mobile feature without directly interacting with the content of the external website.

Handling network connections: Understanding the flow

Especially with mobile wrappers, which continuously display the content of an external website via the internet, it is important that you, as a developer, check for network status changes and handle them properly. In a .NET MAUI application wrapping an external website, the flow can include:

1. Declaring a service interface that will be implemented by platform-specific objects and that defines a method that will open the device settings for network connections.
2. Implementing the interface via native classes that open the device settings for network connections based on the current platform.
3. Registering the service interface with the .NET MAUI startup objects.
4. Subscribing for network status changes via the **Connectivity** cross-platform API in the app's main page.
5. Showing visual elements that allow for changing network settings if the internet connection is lost.
6. Hiding these visual elements if the internet connection is restored.

In the next sections, you will implement all the aforementioned features so that your application will gracefully handle network status changes.

Defining and implementing a network service

To provide a cross-platform way to open device-specific network settings, a service interface and its platform-specific implementations are required. This service is registered with the dependency injection container in the **MauiParam** class.

So, you first need a new project folder called **Services** with a new code file called **INetworkService.cs** inside that defines the following interface:

```
public interface INetworkSettingsService
```

```
{
    void OpenNetworkSettings();
}
```

The **OpenNetworkSettings** method serves as a contract, ensuring that any class implementing this interface will provide a way to open the operating system's network settings, regardless of the underlying platform. Then, in the **MauiProgram.cs** file, you register this dependency inside the **CreateMauiApp** method, right after the invocation to the **CreateBuilder** method:

```
#if ANDROID
    builder.Services.AddSingleton<INetworkSettingsService,
        Platforms.Android.NetworkSettingsService>();
#elif IOS
    builder.Services.AddSingleton<INetworkSettingsService,
        Platforms.iOS.NetworkSettingsService>();
#endif
```

The conditional compilation directives are used to register the appropriate platform-specific implementation of **INetworkSettingsService**. Both the **NetworkSettingsService** classes will be defined shortly. The **AddSingleton** method registers the service as a singleton, meaning a single instance will be created and reused throughout the application's lifetime. For Android, the network service implementation is represented by a new class called **NetworkSettingsService**, declared inside a new **NetworkSettingsService.cs** file to be added to the **Platforms\Android** folder. Code Listing 22 shows the code for this class.

Code Listing 22

```
public class NetworkSettingsService : INetworkSettingsService
{
    public void OpenNetworkSettings()
    {
        var intent = new Intent(global::Android.Provider.
            Settings.ActionWirelessSettings);
        intent.SetFlags(ActivityFlags.NewTask);
        global::Android.App.Application.
            Context.StartActivity(intent);
    }
}
```

The **NetworkSettingsService** class implements **INetworkSettingsService**, with its **OpenNetworkSettings** method that creates a new **Android.Content.Intent**. An intent generally represents a system action. The **global::Android.Provider.Settings.ActionWirelessSettings** constant specifies the system action to open the wireless settings screen.

The **ActivityFlags.NewTask** value is added to ensure that the system settings activity is opened as a stand-alone page. In this way, the settings page is not added to the application's internal navigation stack. Finally, **global::Android.App.Application.Context.StartActivity(intent)** launches the device's network settings activity.

For iOS, you add a new code file called **NetworkSettingsService.cs** to the **Platforms\iOS** folder. This declares the **NetworkSettingsService** class, as shown in Code Listing 23.

Code Listing 23

```
public class NetworkSettingsService : INetworkSettingsService
{
    public void OpenNetworkSettings()
    {
        var wifiUrl = new NSURL("prefs:root=WIFI");
        var app = UIApplication.SharedApplication;

        if (app.CanOpenUrl(wifiUrl))
        {
            app.OpenUrl(wifiUrl);
        }
        else
        {
            var appPrefsUrl = new NSURL("App-Prefs:root=WIFI");
            app.OpenUrl(appPrefsUrl);
        }
    }
}
```

The **OpenNetworkSettings** method on iOS attempts to open the Wi-Fi settings using a URL scheme. **Foundation.NSUrl** is used to create a URL object from the string **prefs:root=WIFI**, which is a system URL scheme used to deep link into system settings. The **UIKit.UIApplication.SharedApplication** object provides access to the shared application object the **app.CanOpenUrl(wifiUrl)** method checks if the device can open the specified URL scheme. If successful, the next call to **app.OpenUrl(wifiUrl)** opens the Wi-Fi settings. If the direct scheme fails, an alternative scheme **App-Prefs:root=WIFI** is attempted, which typically opens the application's settings within the iOS Settings app, from where the user can then navigate to Wi-Fi settings.

Designing a user interface for network status change

To visually inform the user about a lack of internet connection, or more generally about a network status change, it is a good idea to implement a dedicated visual element in the application's main page that overlays the page content. This overlay will appear when the network is unavailable, disabling the main content and providing an option to open network settings.

With this in mind, add the following XAML as a child of the root **Grid** of the page:

```
<Grid x:Name="NoNetworkGrid" IsVisible="False" InputTransparent="False">
    <Grid.RowDefinitions>
        <RowDefinition Height="4*" />
        <RowDefinition Height="6*" />
    </Grid.RowDefinitions>
</Grid>
```

```

</Grid.RowDefinitions>
<Grid HorizontalOptions="Fill" VerticalOptions="Fill"
    InputTransparent="False" />
<Border BackgroundColor="White" Stroke="White" Grid.Row="1">
    <Border.StrokeShape>
        <RoundRectangle CornerRadius="40,40,0,0" />
    </Border.StrokeShape>
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>
        <Label Text="No internet connection"
            Margin="0,30,0,0" HorizontalTextAlignment="Center"
            FontAttributes="Bold" FontSize="Title" />
        <Label Grid.Row="1" Margin="15,30,15,0"
            Text="Internet connection not available.
                Select a network from system settings."
            FontSize="Medium" TextColor="Black"
            HorizontalTextAlignment="Center" />
        <Button Grid.Row="2" Margin="40,30,40,0"
            Text="Open Settings" CornerRadius="20"
            BackgroundColor="Blue" TextColor="White"
            x:Name="NetworkSettingsButton"
            Clicked="NetworkSettingsButton_Clicked" />
    </Grid>
</Border>
</Grid>

```

The **NoNetworkGrid** is added directly under the main **Grid**. This positioning allows it to overlay the entire application content when visible, and its visibility is set as **False**. Assigning **InputTransparent="False"** helps to ensure it can receive touch events when active. This new grid defines two rows—the first row provides a transparent space to dim the background content, and the second row contains a **Border** element with rounded top corners. This element contains a **Grid** with a child **Label** to display a "No internet connection" message, and a child **Button** labeled "Open Settings." The click event handler for the button is called **NetworkSettingsButton_Clicked**, which will trigger the opening of OS network settings, to be defined in the next section.

Handling connection events

Now that you have defined the user interface to inform users about the network status change, including the lack of internet connection, it is time to handle connection events and display or hide these visual elements. You need to hold a list of services that can be defined in the **App.xaml.cs** file as follows:

```
public static IServiceProvider Services { get; set; }
```

```

public App(IServiceProvider serviceProvider)
{
    InitializeComponent();
    Services = serviceProvider;
}

```

You are overriding the constructor definition using the .NET MAUI's dependency injection engine to receive the collection of services created in the **MauiProgram** class. The retrieved collection is assigned to a static property called **Services**, of type **IServiceProvider**. In the **MainPage.xaml.cs** file, you need to retrieve the platform-specific instance of the **NetworkSettingsService** class. To accomplish this, first add the following field to keep a service reference:

```

private readonly INetworkSettingsService? _networkSettingsService;

```

Then, extend the page's constructor as follows:

```

public MainPage()
{
    InitializeComponent();
    ViewModel = new MainPageViewModel();

    this.BindingContext = ViewModel;
    ViewModel.Source =
        new UrlWebViewSource()
        { Url = $"{baseUrl}Identity/Account/Login" };

    _networkSettingsService =
        App.Services.GetService<INetworkSettingsService>();
    Connectivity.Current.
        ConnectivityChanged += Current_ConnectivityChanged;
}

```

The **App.Services.GetService** method retrieves the **INetworkSettingsService** instance from the dependency injection container, whereas the next line subscribes to the **ConnectivityChanged** event from the **Connectivity** class to react to network status changes.

The event handler for the network status change looks like the following:

```

private void Current_ConnectivityChanged(object? sender,
    ConnectivityChangedEventArgs e)
{
    if (e.NetworkAccess != NetworkAccess.Internet)
        DisableUIForNoInternet();
    else
        EnableUIForInternet();
}

```

The **Current_ConnectivityChanged** event handler receives an object of type **ConnectivityChangedEventArgs**, which exposes the **NetworkAccess** property. If its value is not **NetworkAccess.Internet**, it means there is no internet connection, and a method called **DisableUIForNoInternet** is called to disable the main content and show the appropriate user interface. Otherwise, the **EnableUIForInternet** is invoked to restore the previous status. Both methods are simply defined as follows:

```
// Internet is available: hide the "no network" popup

private void EnableUIForInternet()
{
    Dispatcher.Dispatch(() =>
    {
        RootWebView.IsEnabled = true;
        LayoutRoot.IsEnabled = true;
        LayoutRoot.Opacity = 1;
        NoNetworkGrid.IsVisible = false;
    });
}

// Internet is not available: show the "no network" popup
private void DisableUIForNoInternet()
{
    Dispatcher.Dispatch(() =>
    {
        RootWebView.IsEnabled = false;
        LayoutRoot.IsEnabled = false;
        LayoutRoot.Opacity = 0.3;
        NoNetworkGrid.IsVisible = true;
    });
}
```

The **EnableUIForInternet** method is responsible for restoring the main user interface when internet connectivity is available. It invokes **Dispatcher.Dispatch** to ensure that UI updates occur on the main thread. It enables the **RootWebView** and **LayoutRoot** for interaction, sets their opacity to 1 (fully visible), and hides the **NoNetworkGrid** visual element.

The **DisableUIForNoInternet** method is called when internet connectivity is lost. It also uses **Dispatcher.Dispatch** for the same reason. It disables the **RootWebView** and **LayoutRoot** to prevent user interaction with the underlying content, dims **LayoutRoot** by setting its **Opacity** property to 0.3, and makes the **NoNetworkGrid** visible, presenting the “no internet” message and the Open Settings button.

Finally, the **NetworkSettingsButton_Clicked** event handler in **MainPage.xaml.cs** uses the injected service to open the platform's network settings. The code for this event handler is the following:

```
private void NetworkSettingsButton_Clicked(object sender,
                                           EventArgs e)
{

```

```
        _networkSettingsService?.OpenNetworkSettings();  
    }
```

The **NetworkSettingsButton_Clicked** event handler is invoked when the Open Settings button on the **NoNetworkGrid** is tapped. It invokes the **OpenNetworkSettings** from the platform-specific implementation of the **NetworkSettingsService** class, which executes the platform-specific code to launch the device's network settings, allowing the user to troubleshoot and restore their internet connection.

Now you have completed all the work and you are ready to test the application and see how it reacts to network status changes.

Running the application

When you're ready, press F5 to start debugging the application. Notice that the iOS simulator used for this book does not have an option to enable airplane mode or to disconnect the system from the network. You will need a physical device to do this. On Android, it's possible inside the emulator, so this feature is demonstrated with an only figure from Android (see Figure 30).

When the external website is displayed, leave the application open in the background and turn off both your Wi-fi and data connections. At this point, you will see how the application will react to the network status change by displaying the appropriate user interface, as shown in Figure 30.

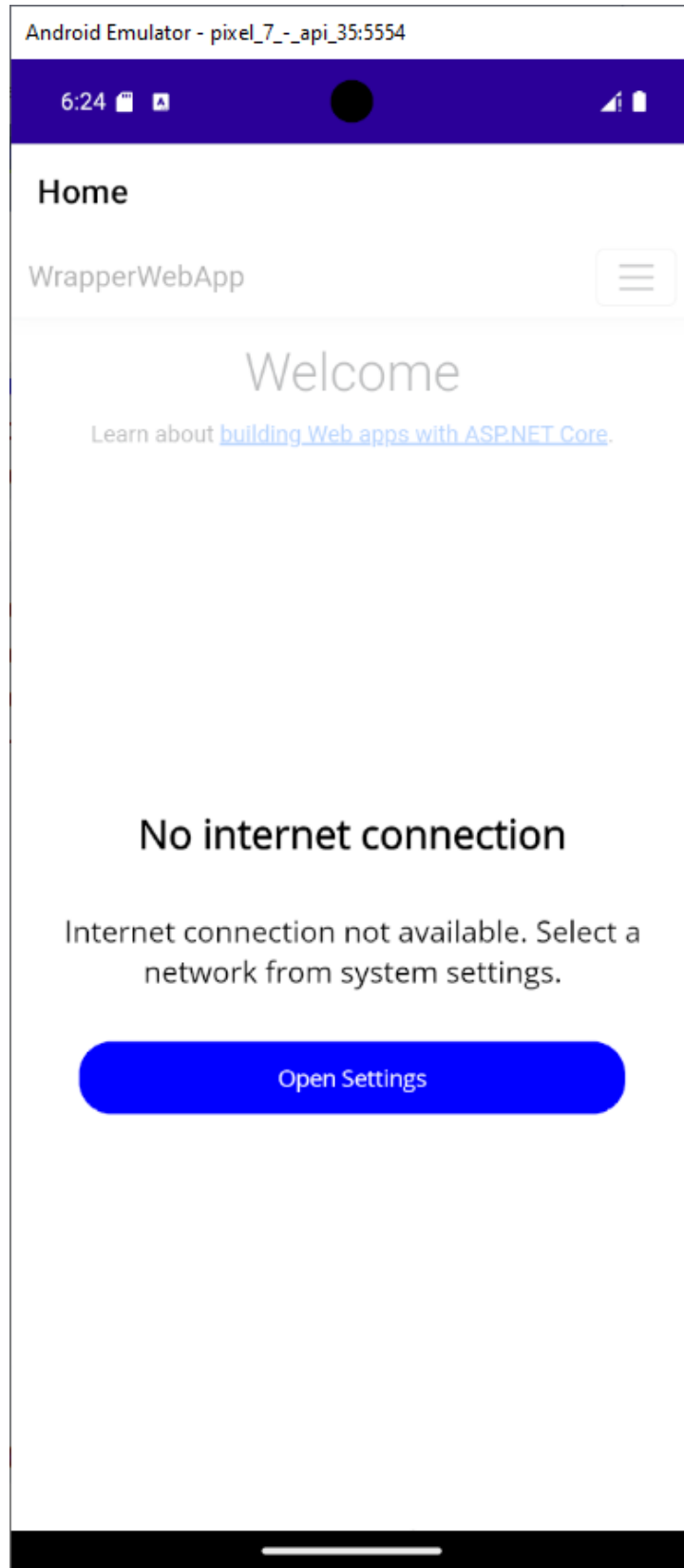


Figure 30: The application showing the “no network” user interface

You can click the button to open the device's network settings and restore the Wi-Fi or mobile data connections. When done, you will see how the pop-up notification will disappear. It is best practice when developing mobile apps to use this feature, and it is even more important with mobile app wrappers that typically require an internet connection to work against an external website.

Future enhancements

You could consider extending the current sample app with additional features. For example, you could integrate more sensors, or you could implement a webpage that allows for uploading files to the page and integrate this with NET MAUI, using the File Picker API to retrieve a file on the mobile device and send it to the server. You could consider extending the sample app to use the camera if the target website accepts pictures. You really have plenty of options to improve the mobile experience of your applications.

Hints about submitting apps to Google Play and the Apple App Store

You will follow exactly the same steps you know to submit an app for review and approval to the two major app stores. There is really no change, and the only thing you will want to make sure to do is provide the appropriate description for each device feature that you use. If you are not familiar with this, the Microsoft documentation provides appropriate guidance for both [Android](#) and [iOS](#).

Chapter summary

This chapter provided a comprehensive approach to improving the mobile experience by robustly handling internet connectivity changes. You learned how to implement a cross-platform **INetworkSettingsService** interface and its platform-specific Android and iOS implementations to programmatically open device network settings.

Furthermore, you integrated a dynamic UI overlay on the main page that visually communicates the absence of an internet connection and provides a direct pathway for users to resolve the issue. By leveraging the MAUI's **Connectivity** API to detect real-time network status changes, the application now offers a user-friendly experience, ensuring that the user always has control over the app.

Conclusion

Throughout this book, you have embarked on a comprehensive journey into the world of building mobile web wrappers with .NET MAUI. The path you have followed, from understanding the fundamental concepts of web wrappers to implementing advanced features like biometric authentication and robust network handling, illustrates a powerful paradigm in modern application development. You have seen how .NET MAUI provides a compelling framework to bridge the gap between the agility of web technologies and the rich capabilities of native mobile platforms. This approach offers a strategic advantage, allowing you to leverage existing web investments while still delivering a polished, integrated mobile experience.

By wrapping your web content in a native shell, you gain the ability to distribute your application through app stores, access device-specific features like sensors and biometrics, and provide a user interface that feels inherently mobile. The seamless integration of web content with native UI elements, such as custom navigation bars and dynamic connectivity overlays, transforms a simple website into a fully-fledged mobile application, enhancing user engagement and satisfaction.

Ultimately, the power of .NET MAUI in this context lies in its flexibility. It enables developers to choose the right balance between web and native components, optimizing for development speed, cross-platform reach, and a superior user experience. As you continue to build and evolve your mobile applications, the principles and techniques explored in this book will serve as a valuable foundation, empowering you to create dynamic, responsive, and secure mobile experiences that truly stand out.